

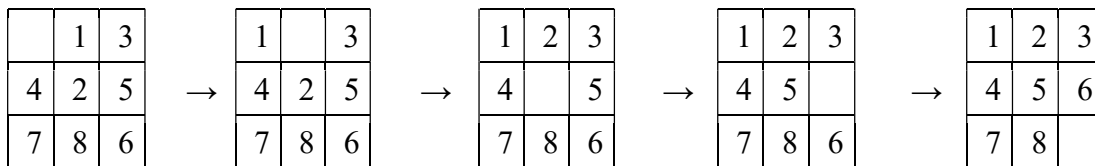
## CSE 318: Offline Assignment

### Solve n-Puzzle

Objective: Write a program to solve the n-puzzle problem using the A\* search algorithm, where

$$n = k - 1, \quad k = 3, 4, 5 \dots$$

Problem description: The n-puzzle is a puzzle invented and popularized by Noyes Palmer Chapman in the 1870s. It is played on a  $k \times k$  grid with  $n = k - 1$  square blocks labeled 1 through  $n$  and a blank square. Your goal is to rearrange the blocks so that they are in order, using as few moves as possible. You are permitted to slide blocks horizontally or vertically into the blank square. The following shows a sequence of legal moves from an initial board (left) to the goal board (right).



Now, we describe a solution to the problem that illustrates the A\* search algorithm. We define a search node of the game to be a board, the number of moves made to reach the board, and the previous search node. First, insert the initial search node (the initial board, 0 moves, and a null previous search node) into a priority queue (open list). Then delete from the priority queue the search node with the minimum priority (the most promising node), insert the node into the closed list, and insert onto the priority queue all neighboring search nodes (that can be reached by one move) which are not in the closed list. Repeat this procedure until the search node dequeued corresponds to the final goal board. The success of this approach hinges on the choice of priority function for a search node. In A\* search algorithm, the priority function  $f(n)$  represents an estimated cost of the path from the initial state to the goal state through node  $n$ .  $f(n)$  is calculated as  $f(n) = g(n) + h(n)$ , where  $g(n)$  is the cost to reach node  $n$  from the initial state and  $h(n)$  is an estimated cost from node  $n$  to the goal state. The function  $h(n)$  is often called a heuristic. Finding optimal path by A\* search algorithm depends on the properties of the heuristic used.

In this assignment we will consider two priority functions:

1. **Hamming distance:** The number of blocks in the wrong position (not including the blank)
2. **Manhattan distance:** The sum of the Manhattan distances (sum of the vertical and horizontal distance) of the blocks to their goal positions.

Let us assume that our initial board is

|   |   |   |
|---|---|---|
| 7 | 2 | 4 |
| 6 |   | 5 |
| 8 | 3 | 1 |

Our goal board is

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 |   |

From the following table we can calculate Hamming distance = 7

| Digits                                | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---------------------------------------|---|---|---|---|---|---|---|---|
| Is placed correctly (1 = no, 0 = yes) | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 |

From the following table we can calculate Manhattan distance = **16**

| Digits          | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|-----------------|---|---|---|---|---|---|---|---|
| Row distance    | 2 | 0 | 2 | 1 | 0 | 0 | 2 | 0 |
| Column distance | 2 | 0 | 1 | 2 | 1 | 2 | 0 | 1 |
| Total distance  | 4 | 0 | 3 | 3 | 1 | 2 | 2 | 1 |

|   |   |   |
|---|---|---|
| 8 | 1 | 3 |
| 4 |   | 2 |
| 7 | 6 | 5 |

|   |   |   |
|---|---|---|
| 8 | 1 | 3 |
| 4 | 2 |   |
| 7 | 6 | 5 |

|   |   |   |
|---|---|---|
| 8 | 1 | 3 |
| 4 |   | 2 |
| 7 | 6 | 5 |

|   |   |   |
|---|---|---|
| 8 | 1 | 3 |
| 4 | 2 | 5 |
| 7 | 6 |   |

Previous node

Search node

Neighbor 1



Neighbor 2

Neighbor 3

We can see a search node, it's previous node and generated neighbors above. We must disallow neighbor 2 to avoid generating the previous node as neighbor.

Remember that, to calculate the priority function of the open list, you need to add the cost to reach the current node with the value of heuristic function. To solve the puzzle from a given search node on the priority queue, the total number of moves we need to make (including those already made) is at least its priority, using either the Hamming or Manhattan priority function. (For Hamming distance, this is true

because each block that is out of place must move at least once to reach its goal position. For Manhattan priority, this is true because each block must move its Manhattan distance from its goal position. Note that we do not count the blank square when computing the Hamming or Manhattan priorities. Consequently, when the goal board is dequeued, we have discovered not only a sequence of moves from the initial board to the goal board, but one that makes the fewest number of moves (can you prove this fact?).

Detecting unsolvable puzzles: Not all initial boards can lead to the goal board by a sequence of legal moves, as the following board,

|   |   |   |
|---|---|---|
| 8 | 1 | 2 |
|   | 4 | 3 |
| 7 | 6 | 5 |

To detect such situations, use the fact that boards are divided into two equivalence classes with respect to the reachability; those which lead to the goal board and those which cannot lead to the goal board. Moreover, we can identify in which equivalence class a board belongs without attempting to solve it if we use inversion. Given a board, an inversion is any pair of blocks  $i$  and  $j$  where  $i < j$  but  $i$  appears after  $j$  when considering the board in row-major order (row 0, followed by row 1, and so forth).

If the grid size  $k$  is an odd integer, then each legal move changes the number of inversions by an even number. Thus, if a board has an odd number of inversions, then it cannot lead to the goal board by a sequence of legal moves because the goal board has an even number of inversions (zero). The converse is also true, if a board has an even number of inversions, then it can lead to the goal board by a sequence of legal moves.

If the grid size  $k$  is an even integer, puzzle instance is solvable if

- the blank is on an even row counting from the bottom (second-last, fourth-last, etc.) and number of inversions is odd.
- the blank is on an odd row counting from the bottom (last, third-last, fifth-last, etc.) and number of inversions is even.
- For all other cases, the puzzle instance is not solvable.

Courtesy:

[How to check if an instance of 8 puzzle is solvable? - GeeksforGeeks](#)

[How to check if an instance of 15 puzzle is solvable? - GeeksforGeeks](#)

.

Tasks:

1. Implement A\* search algorithm using the two mentioned heuristics. Your first input will be the grid size  $k$ . Your second input will be the initial board position. Represent the blank using an asterisk '\*'.
  - a. Show the optimal cost to reach the goal state and the steps. You can show the board states at command prompt. Use your best judgement to decide your output layout.
  - b. For two different heuristics, print number of explored nodes and expanded nodes.

# **Chapter 4**

## **Informed/Heuristic Search**

# Outline

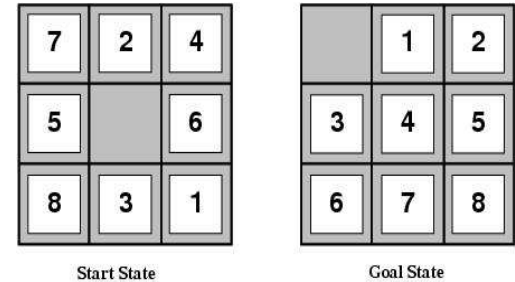
---

- Limitations of uninformed search methods
- Informed (or heuristic) search uses problem-specific heuristics to improve efficiency
  - Best-first
  - $A^*$
  - RBFS
  - SMA\*
  - Techniques for generating heuristics
- Can provide significant speed-ups in practice
  - e.g., on 8-puzzle
  - But can still have worst-case exponential time complexity
- Reading:
  - Chapter 4, Sections 4.1 and 4.2

# Limitations of uninformed search

---

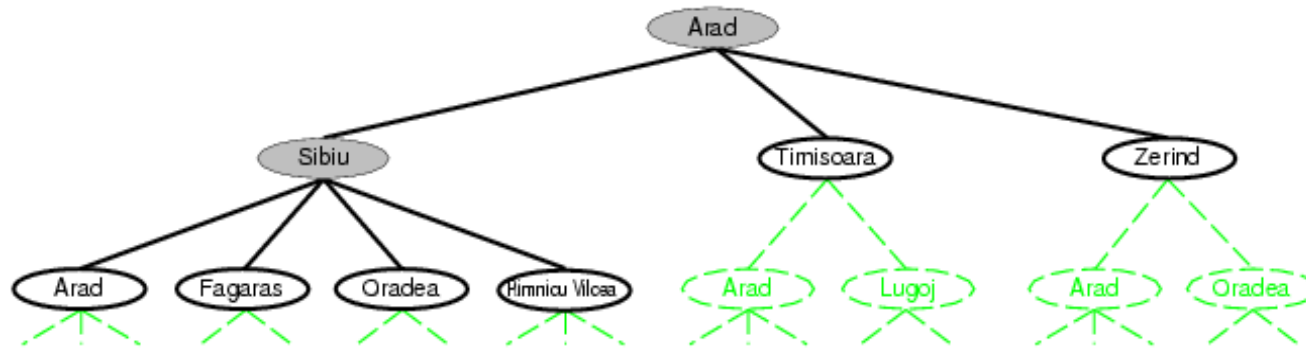
- 8-puzzle
  - Avg. solution cost is about 22 steps
  - branching factor  $\sim 3$
  - Exhaustive search to depth 22:
    - $3.1 \times 10^{10}$  states
  - E.g.,  $d=12$ , IDS expands 3.6 million states on average



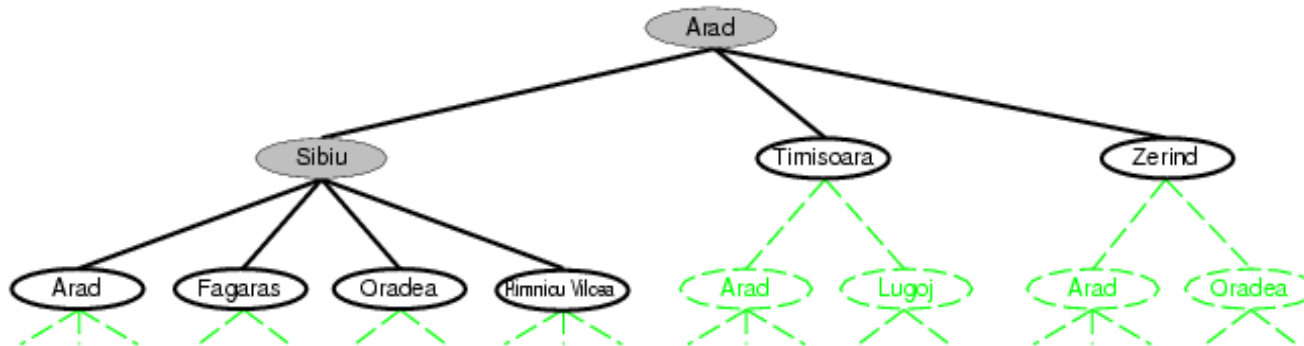
[24 puzzle has  $10^{24}$  states (much worse)]

# Recall tree search...

---



# Recall tree search...



```
function TREE-SEARCH(problem, strategy) returns a solution
  initialize the search tree using the initial state of problem
  loop do
```

```
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
```

This “strategy” is what differentiates different search algorithms



# Best-first search

---

- Idea: use an **evaluation function**  $f(n)$  for each node
  - estimate of "desirability"
  - Expand most desirable unexpanded node
- Implementation:
  - Order the nodes in fringe by  $f(n)$  (by desirability, lowest  $f(n)$  first)
- Special cases:
  - uniform cost search (from last lecture):  $f(n) = g(n) = \text{path to } n$
  - greedy best-first search
  - $A^*$  search
- Note: evaluation function is an estimate of node quality
  - => More accurate name for "best first" search would be "seemingly best-first search"

# Heuristic function

---

- Heuristic:
  - Definition: “using rules of thumb to find answers”
- Heuristic function  $h(n)$ 
  - Estimate of (optimal) cost from  $n$  to goal
  - $h(n) = 0$  if  $n$  is a goal node
  - Example: straight line distance from  $n$  to Bucharest
    - Note that this is not the true state-space distance
    - It is an estimate – actual state-space distance can be higher
  - Provides problem-specific knowledge to the search algorithm

# Heuristic functions for 8-puzzle

---

- 8-puzzle
  - Avg. solution cost is about 22 steps
  - branching factor  $\sim 3$
  - Exhaustive search to depth 22:
    - $3.1 \times 10^{10}$  states.
  - A good heuristic function can reduce the search process.
- Two commonly used heuristics
  - $h_1$  = the number of misplaced tiles
    - $h_1(s) = 8$
  - $h_2$  = the sum of the distances of the tiles from their goal positions (Manhattan distance).
    - $h_2(s) = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$

|   |   |   |
|---|---|---|
| 7 | 2 | 4 |
| 5 |   | 6 |
| 8 | 3 | 1 |

Start State

|   |   |   |
|---|---|---|
|   | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |

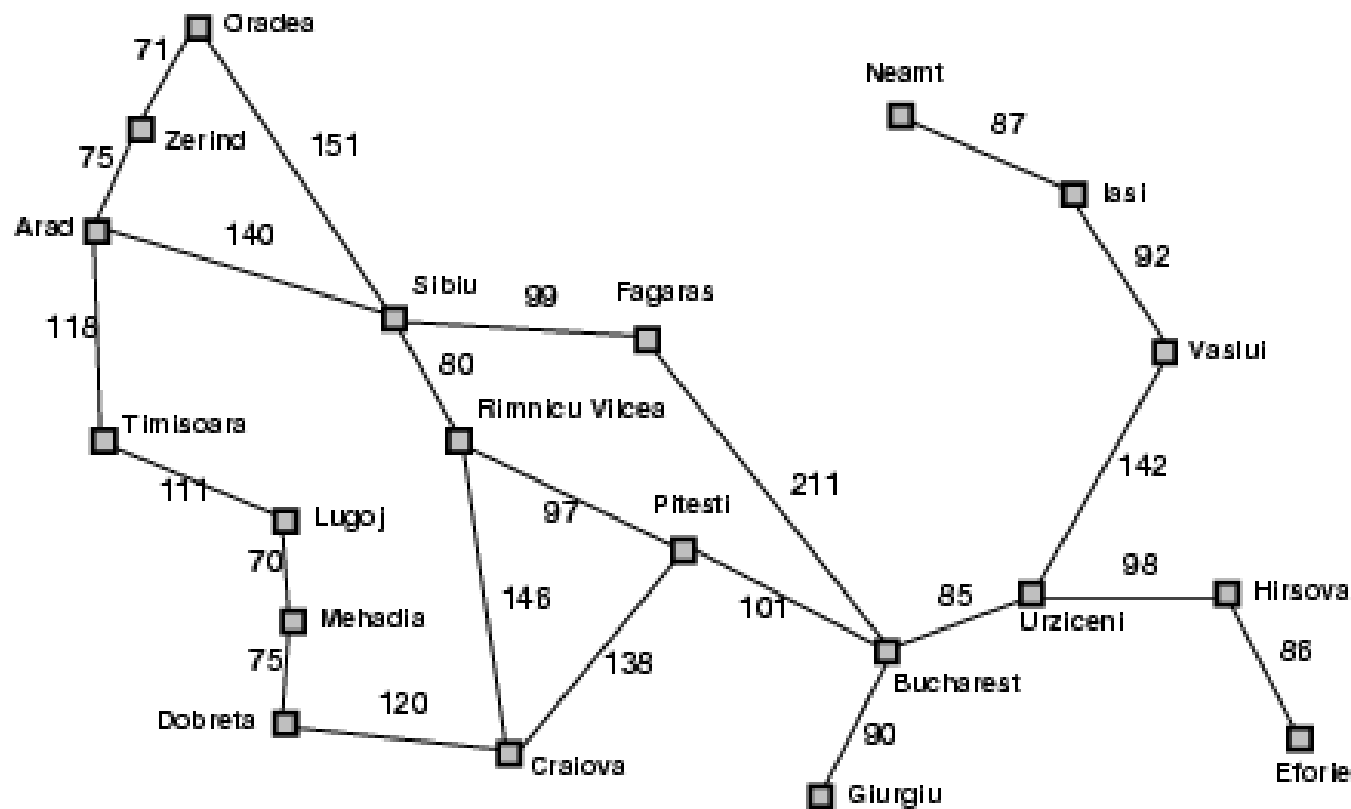
Goal State

# Greedy best-first search

---

- Special case of best-first search
  - Uses  $h(n)$  = heuristic function as its evaluation function
  - Expand the node that appears closest to goal

# Romania with step costs in km

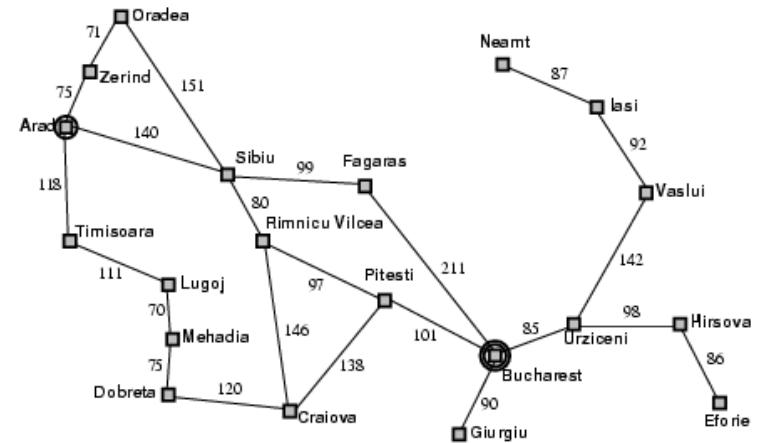


Straight-line distance  
to Bucharest

|                |     |
|----------------|-----|
| Arad           | 366 |
| Bucharest      | 0   |
| Craiova        | 160 |
| Dobreta        | 242 |
| Eforie         | 161 |
| Fagaras        | 176 |
| Giurgiu        | 77  |
| Hirsova        | 151 |
| Iasi           | 226 |
| Lugoj          | 244 |
| Mehadia        | 241 |
| Neamt          | 234 |
| Oradea         | 380 |
| Pitesti        | 10  |
| Rimnicu Vilcea | 193 |
| Sibiu          | 253 |
| Timisoara      | 329 |
| Urziceni       | 80  |
| Vaslui         | 199 |
| Zerind         | 374 |

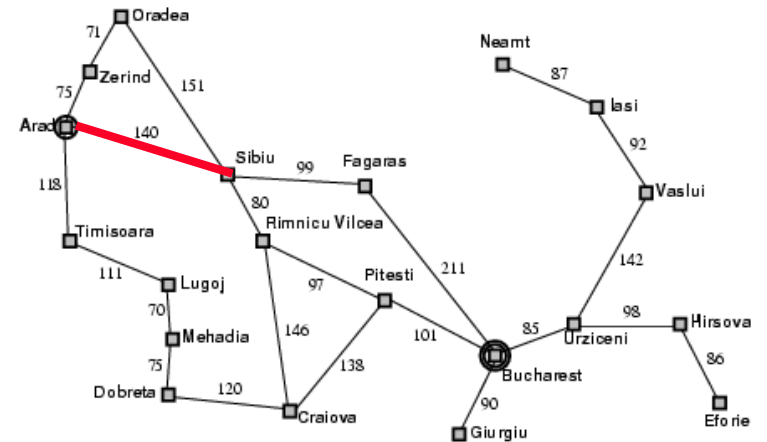
# Greedy best-first search example

---

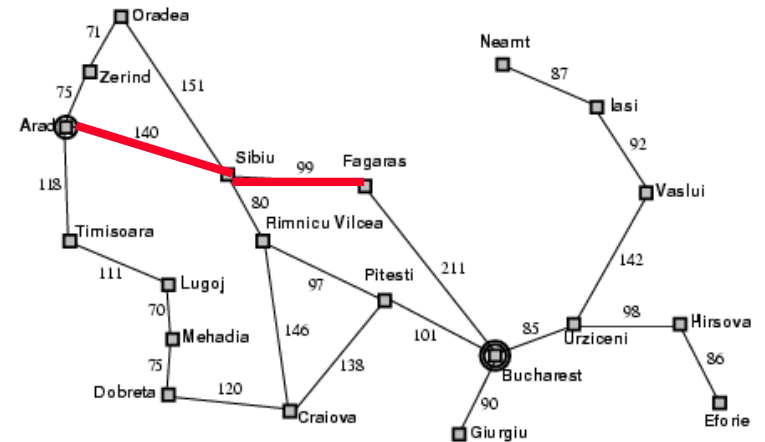
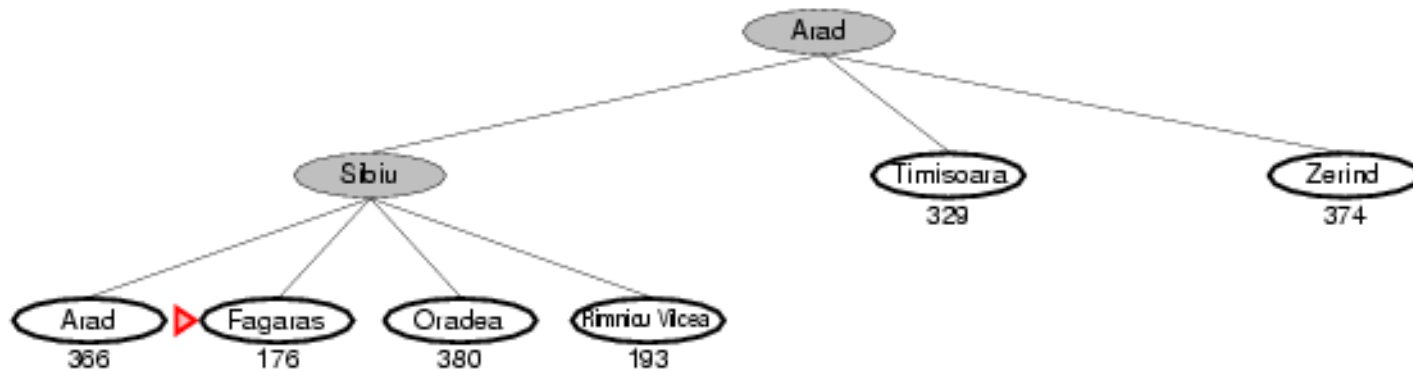


# Greedy best-first search example

---

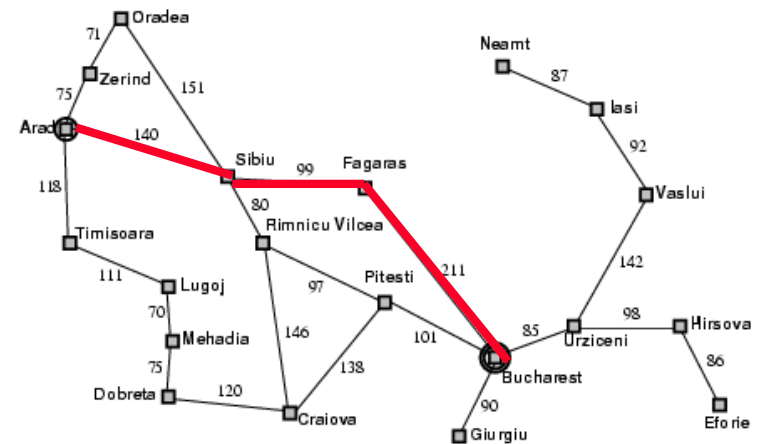
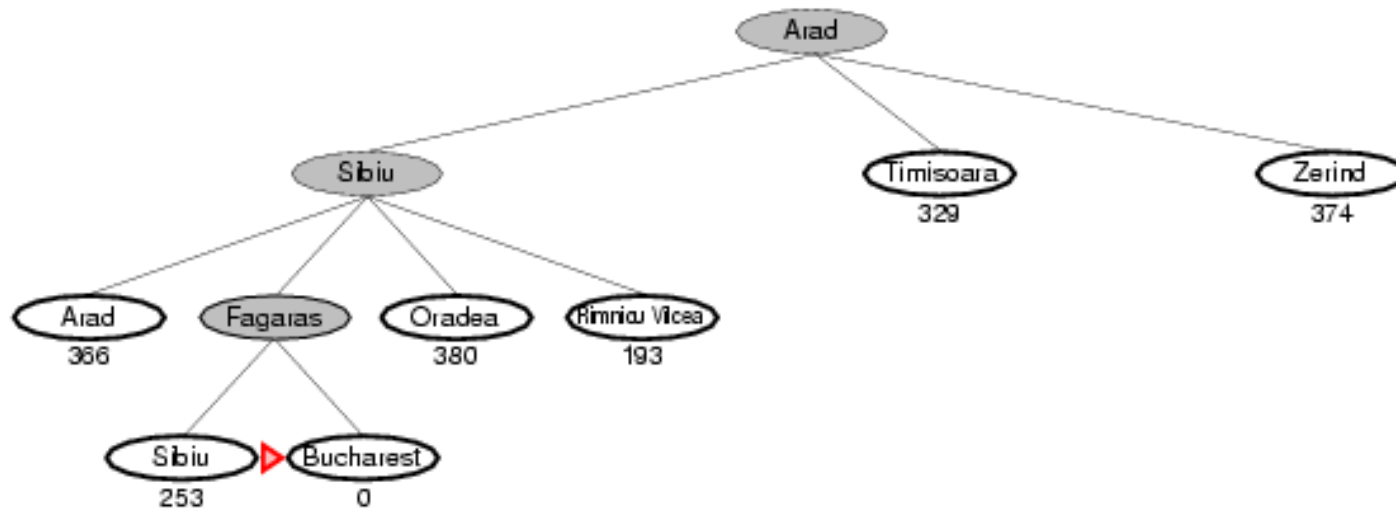


# Greedy best-first search example



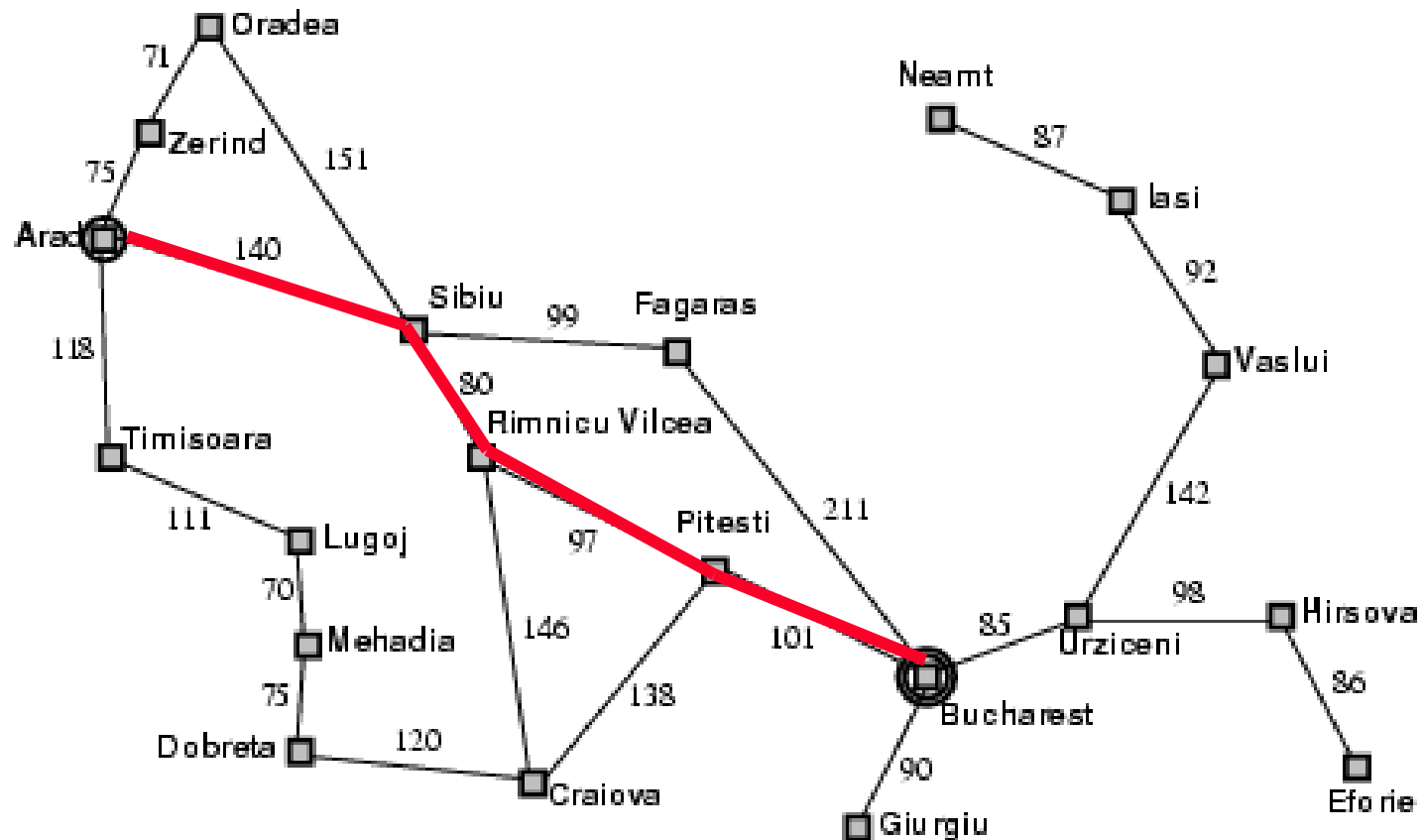


# Greedy best-first search example



# Optimal Path

---



# Properties of greedy best-first search

---

- Complete?
  - Not unless it keeps track of all states visited
    - Otherwise can get stuck in loops (just like DFS)
- Optimal?
  - No – we just saw a counter-example
- Time?
  - $O(b^m)$ , can generate all nodes at depth  $m$  before finding solution
  - $m$  = maximum depth of search space
- Space?
  - $O(b^m)$  – again, worst case, can generate all nodes at depth  $m$  before finding solution

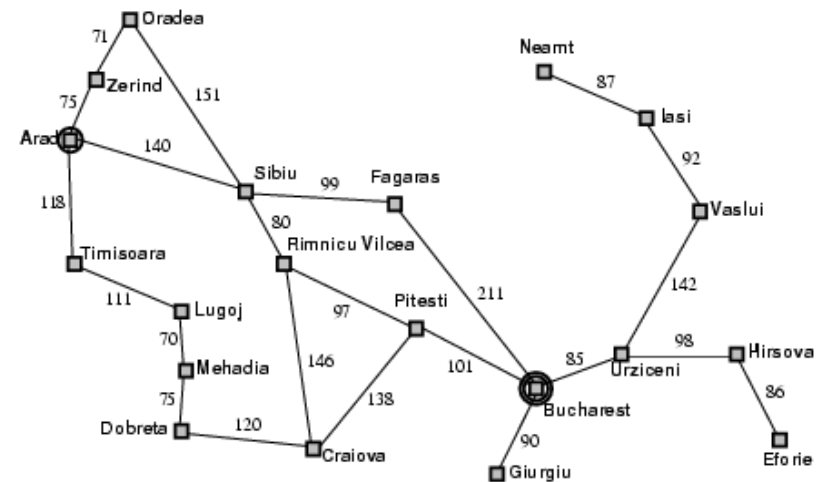
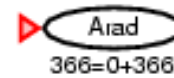
# A\* Search

---

- Expand node based on estimate of total path cost through node
- Evaluation function  $f(n) = g(n) + h(n)$ 
  - $g(n)$  = cost so far to reach  $n$
  - $h(n)$  = estimated cost from  $n$  to goal
  - $f(n)$  = estimated total cost of path through  $n$  to goal
- Efficiency of search will depend on quality of heuristic  $h(n)$

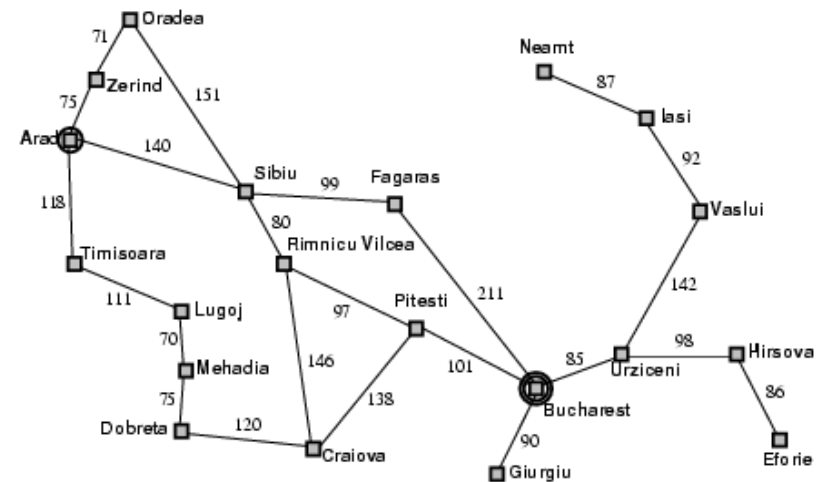
# A\* search example

---

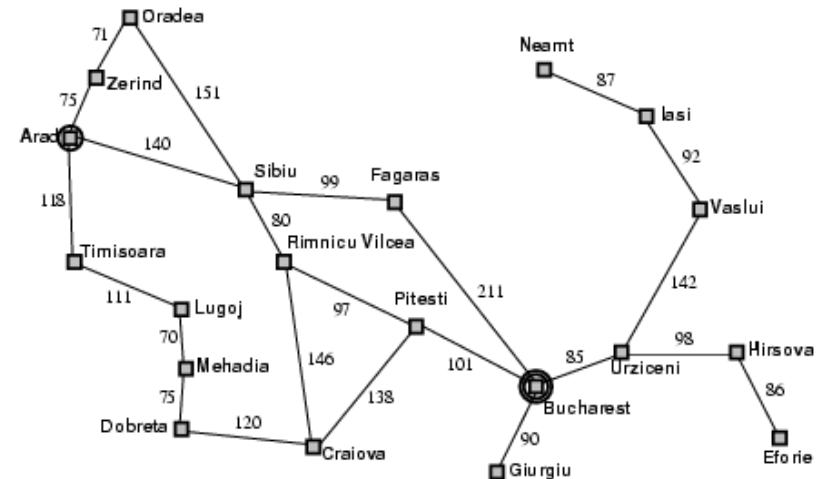
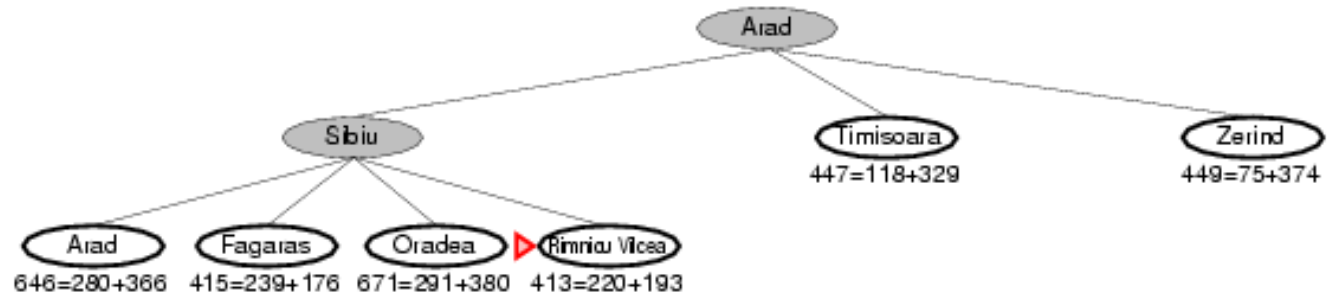


# A\* search example

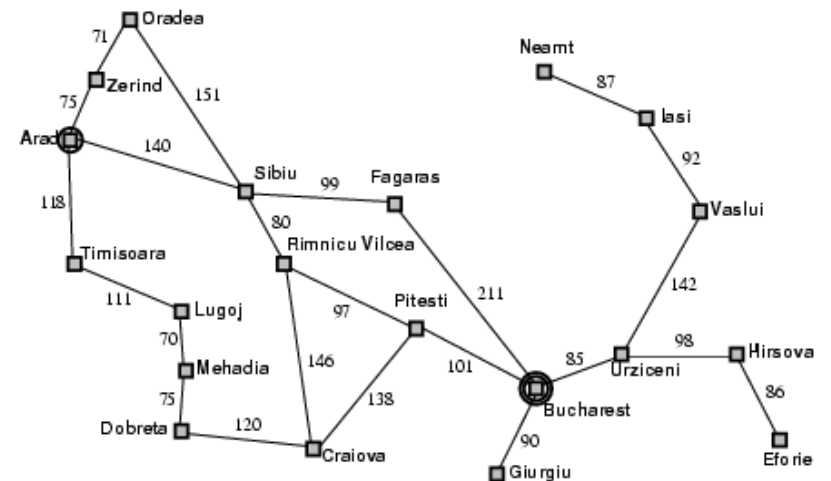
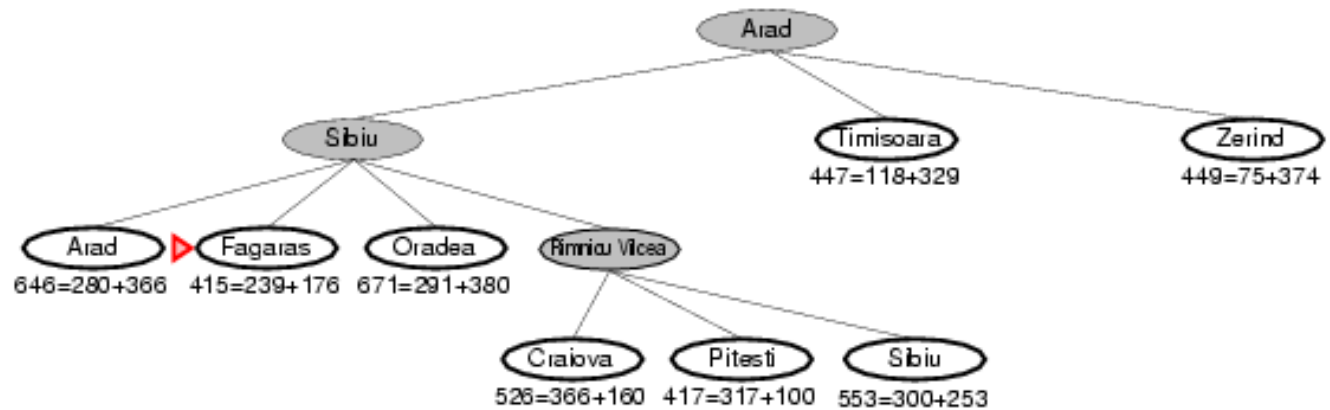
---



# A\* search example

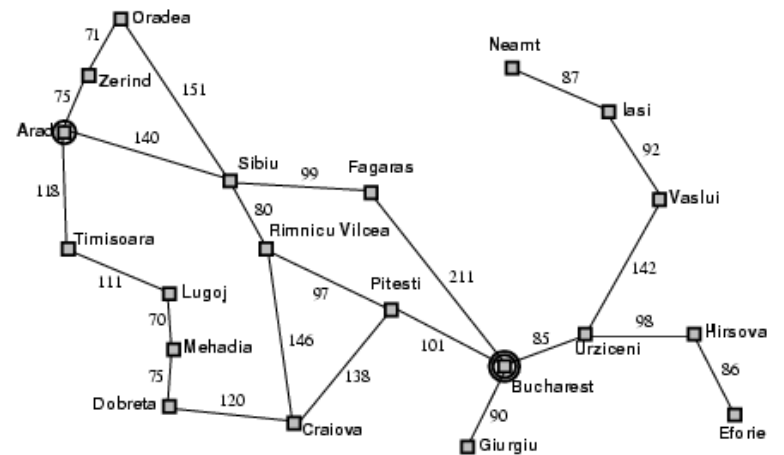
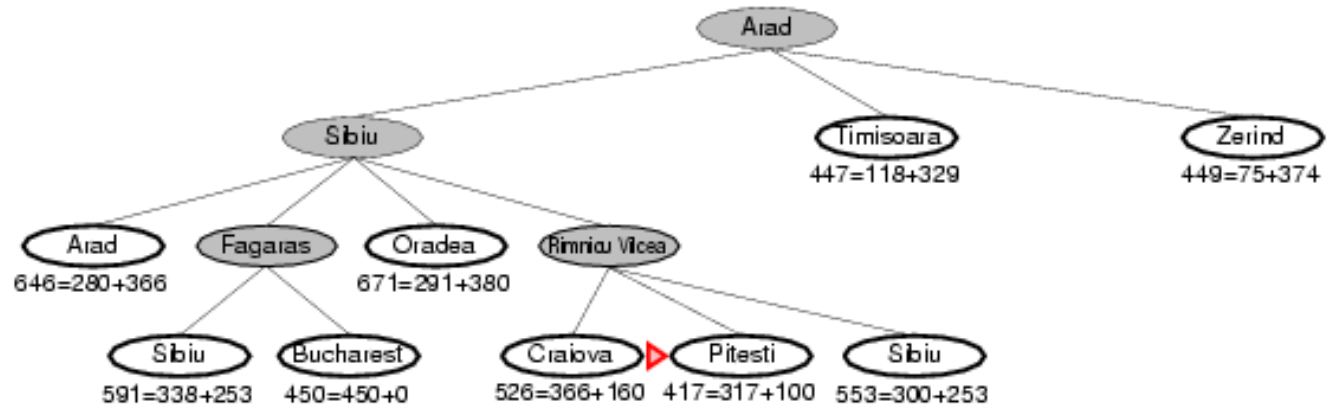


# A\* search example

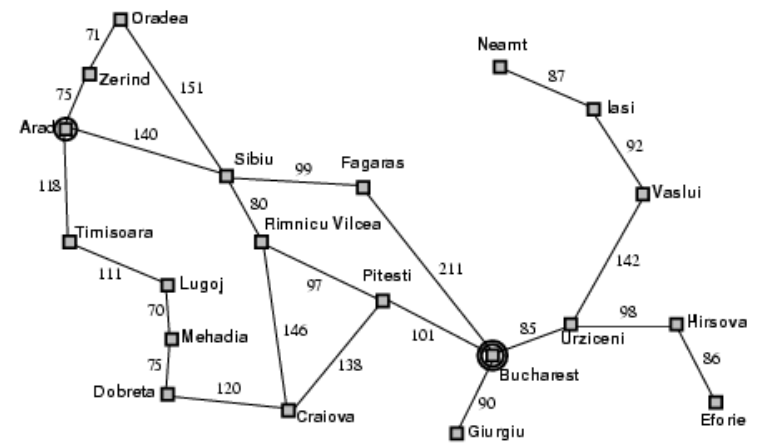
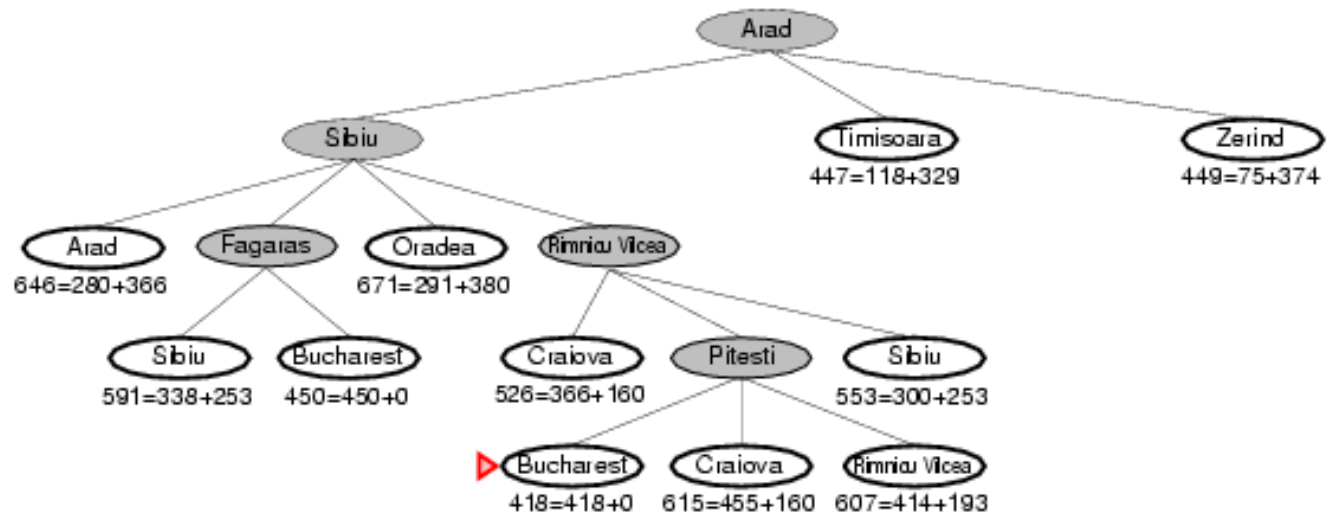




# A\* search example



# A\* search example



# Admissible heuristics

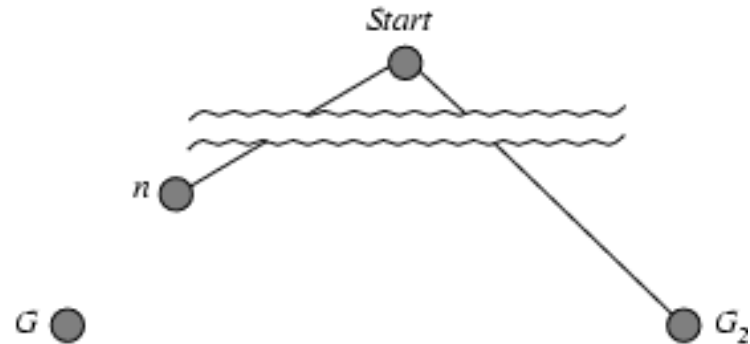
---

- A heuristic  $h(n)$  is **admissible** if for every node  $n$ ,  $h(n) \leq h^*(n)$ , where  $h^*(n)$  is the **true** cost to reach the goal state from  $n$ .
- An admissible heuristic **never overestimates** the cost to reach the goal, i.e., it is **optimistic**
- Example:  $h_{SLD}(n)$  *is admissible*
  - never overestimates the actual road distance
- **Theorem:**
  - If  $h(n)$  is admissible,  $A^*$  using TREE-SEARCH is optimal

## Optimality of $A^*$ (proof)

---

- Suppose some suboptimal goal  $G_2$  has been generated and is in the fringe. Let  $n$  be an unexpanded node in the fringe such that  $n$  is on a shortest path to an optimal goal  $G$ .

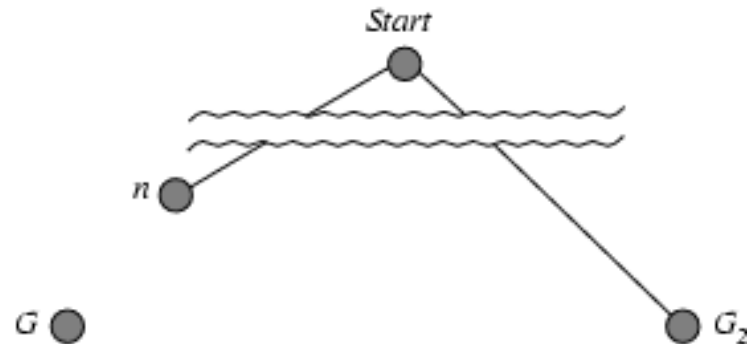


- $f(G_2) = g(G_2)$  since  $h(G_2) = 0$
- $g(G_2) > g(G)$  since  $G_2$  is suboptimal
- $f(G) = g(G)$  since  $h(G) = 0$
- $f(G_2) > f(G)$  from above

## Optimality of A\* (proof)

---

- Suppose some suboptimal goal  $G_2$  has been generated and is in the fringe. Let  $n$  be an unexpanded node in the fringe such that  $n$  is on a shortest path to an optimal goal  $G$ .



- $f(G_2) > f(G)$  from above
- $h(n) \leq h^*(n)$  since  $h$  is admissible
- $g(n) + h(n) \leq g(n) + h^*(n)$
- $f(n) \leq f(G)$

Hence  $f(G_2) > f(n)$ , and  $A^*$  will never select  $G_2$  for expansion

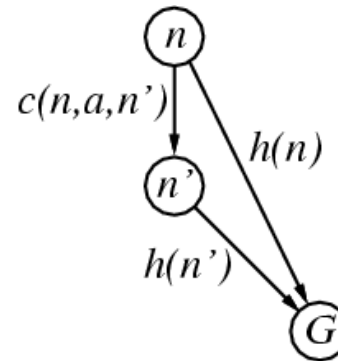
# Optimality for graphs?

---

- Admissibility is not sufficient for graph search
  - In graph search, the optimal path to a repeated state could be discarded if it is not the first one generated
  - Can fix problem by requiring consistency property for  $h(n)$
- A heuristic is **consistent** if for every successor  $n'$  of a node  $n$  generated by any action  $a$ ,

$$h(n) \leq c(n,a,n') + h(n')$$

(aka "monotonic")



- admissible heuristics are generally consistent

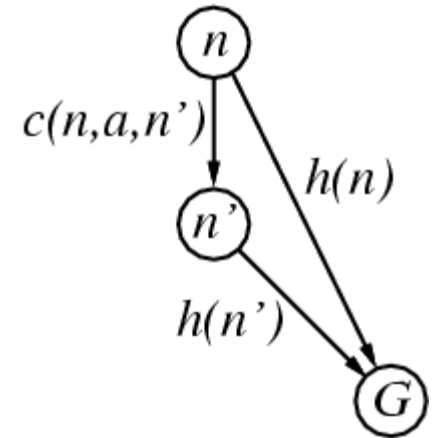
# **A\* is optimal with consistent heuristics**

---

- If  $h$  is consistent, we have

$$\begin{aligned}f(n') &= g(n') + h(n') \\&= g(n) + c(n, a, n') + h(n') \\&\geq g(n) + h(n) \\&= f(n)\end{aligned}$$

i.e.,  $f(n)$  is non-decreasing along any path.

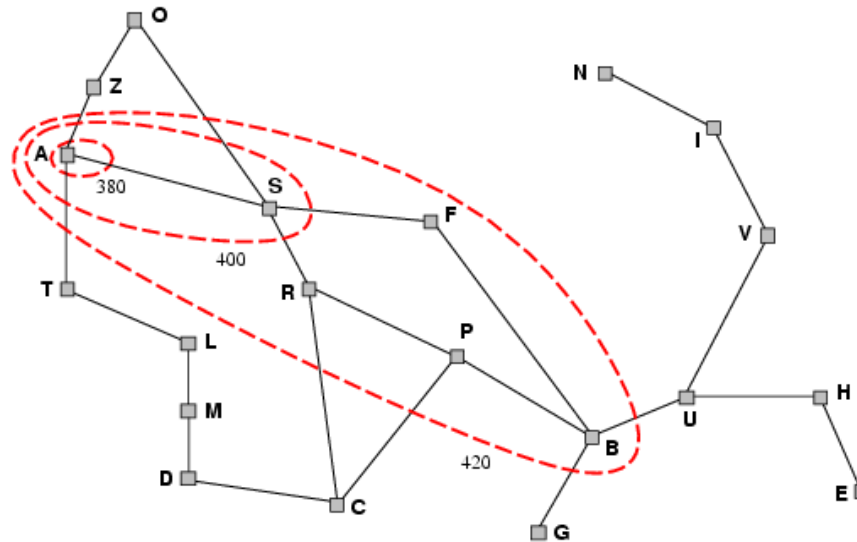


Thus, first goal-state selected for expansion must be optimal

- Theorem:
  - If  $h(n)$  is consistent, A\* using GRAPH-SEARCH is optimal
  -

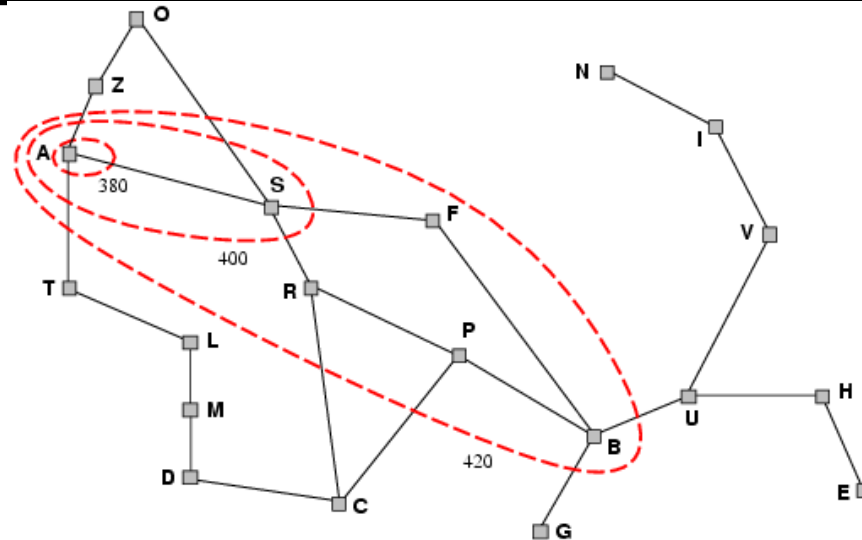
# Contours of A\* Search

- A\* expands nodes in order of increasing  $f$  value
- Gradually adds " $f$ -contours" of nodes
- Contour  $i$  has all nodes with  $f=f_i$ , where  $f_i < f_{i+1}$





# Contours of A\* Search



- With uniform-cost ( $h(n) = 0$ , contours will be circular
- With good heuristics, contours will be focused around optimal path
- A\* will expand all nodes with cost  $f(n) < C^*$

# Properties of A\*

---

- Complete?
  - Yes (unless there are infinitely many nodes with  $f \leq f(G)$  )
- Optimal?
  - Yes
  - Also optimally efficient:
    - No other optimal algorithm will expand fewer nodes, for a given heuristic
- Time?
  - Exponential in worst case
- Space?
  - Exponential in worst case

## Comments on A\*

---

- A\* expands all nodes with  $f(n) < C^*$ 
  - This can still be exponentially large
- Exponential growth will occur unless error in  $h(n)$  grows no faster than  $\log(\text{true path cost})$ 
  - In practice, error is usually proportional to true path cost (not log)
  - So exponential growth is common

# Memory-bounded heuristic search

---

- In practice  $A^*$  runs out of memory before it runs out of time
  - How can we solve the memory problem for  $A^*$  search?
- Idea: Try something like depth first search, but let's not forget everything about the branches we have partially explored.

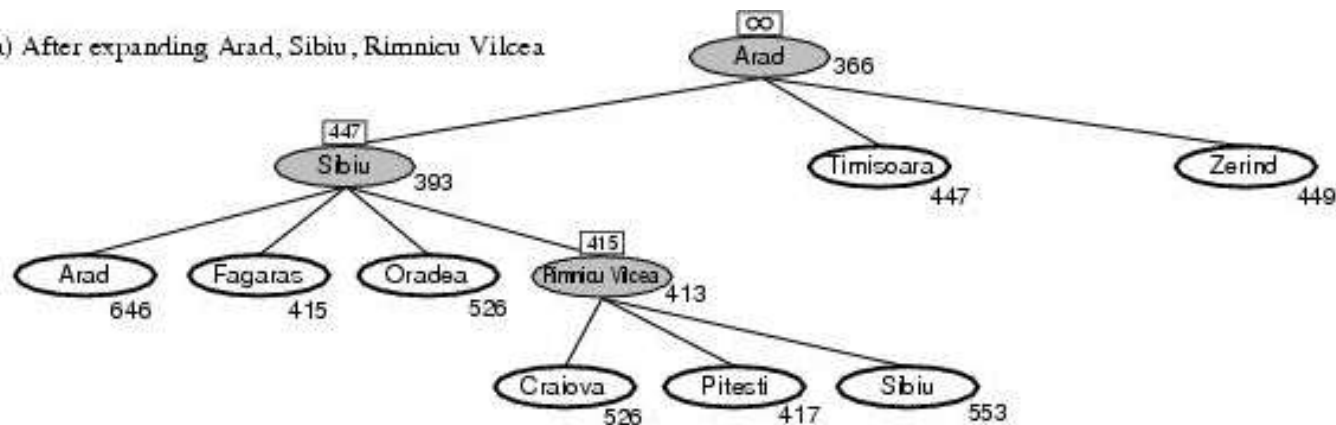
# Recursive Best-First Search (RBFS)

---

- Similar to DFS, but keeps track of the f-value of the best alternative path available from any ancestor of the current node
- If current node exceeds f-limit -> backtrack to alternative path
- As it backtracks, replace f-value of each node along the path with the best  $f(n)$  value of its children
  - This allows it to return to this subtree, if it turns out to look better than alternatives

# Recursive Best First Search: Example

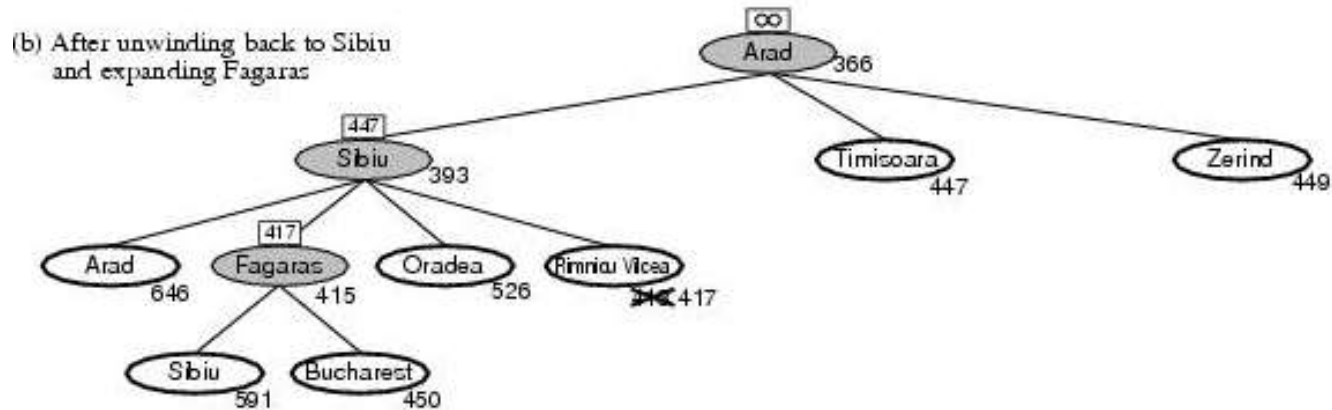
(a) After expanding Arad, Sibiu, Rimnicu Vilcea



- Path until Rumnicu Vilcea is already expanded
- Above node;  $f$ -limit for every recursive call is shown on top.
- Below node:  $f(n)$
- The path is followed until Pitesti which has a  $f$ -value worse than the  $f$ -limit.

## RBFS example

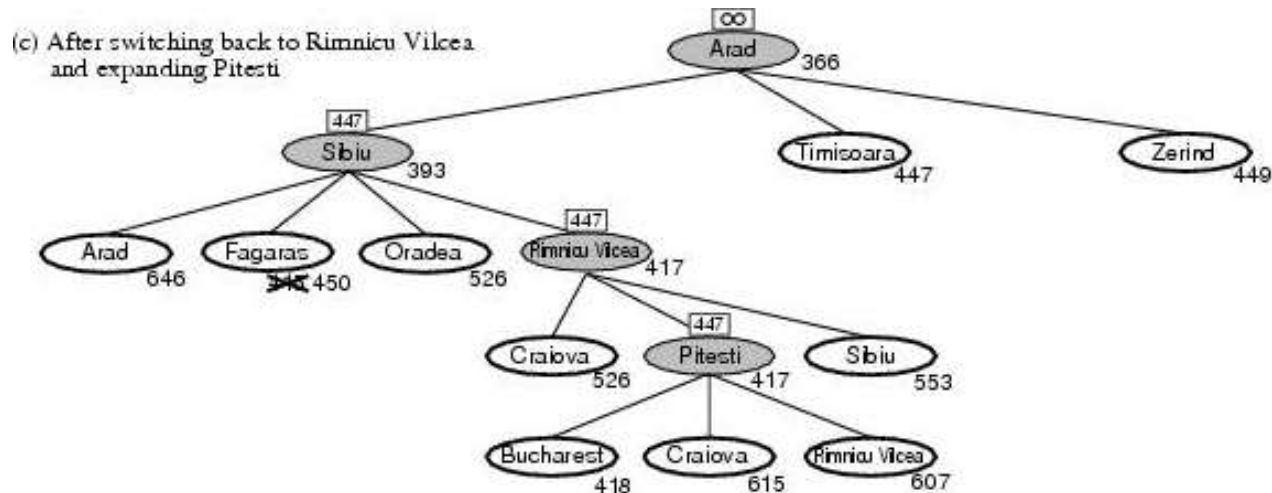
---



- Unwind recursion and store best  $f$ -value for current best leaf Pitesti  

$$result, f[best] \leftarrow \text{RBFS}(\text{problem}, \text{best}, \min(f\_limit, \text{alternative}))$$
- $best$  is now Fagaras. Call RBFS for new  $best$ 
  - $best$  value is now 450

## RBFS example



- Unwind recursion and store best  $f$ -value for current best leaf Fagaras  

$$result, f[best] \leftarrow \text{RBFS}(\text{problem}, best, \min(f\_limit, alternative))$$
- $best$  is now Rimnicu Viclea (again). Call RBFS for new  $best$ 
  - Subtree is again expanded.
  - Best  $alternative$  subtree is now through Timisoara.
- Solution is found since because  $447 > 418$ .



# RBFS properties

---

- Like  $A^*$ , optimal if  $h(n)$  is admissible
- Time complexity difficult to characterize
  - Depends on accuracy of  $h(n)$  and how often best path changes.
  - Can end up “switching” back and forth
- Space complexity is  $O(bd)$ 
  - Other extreme to  $A^*$  - uses ***too little*** memory.

## **(Simplified) Memory-bounded A\* (SMA\*)**

- This is like A\*, but when memory is full we delete the worst node (largest f-value).
- Like RBFS, we remember the best descendant in the branch we delete.
- If there is a tie (equal f-values) we delete the oldest nodes first.
- simplified-MA\* finds the optimal *reachable* solution given the memory constraint.
- Time can still be exponential.

# Heuristic functions

---

- 8-puzzle
  - Avg. solution cost is about 22 steps
  - branching factor  $\sim 3$
  - Exhaustive search to depth 22:
    - $3.1 \times 10^{10}$  states.
  - A good heuristic function can reduce the search process.
- Two commonly used heuristics
  - $h_1$  = the number of misplaced tiles
    - $h_1(s)=8$
  - $h_2$  = the sum of the distances of the tiles from their goal positions (manhattan distance).
    - $h_2(s)=3+1+2+2+2+3+3+2=18$

|   |   |   |
|---|---|---|
| 7 | 2 | 4 |
| 5 |   | 6 |
| 8 | 3 | 1 |

Start State

|   |   |   |
|---|---|---|
|   | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |

Goal State

# Notion of dominance

---

- If  $h_2(n) \geq h_1(n)$  for all  $n$  (both admissible) then  $h_2$  **dominates**  $h_1$   
 $h_2$  is better for search
- Typical search costs (average number of nodes expanded) for 8-puzzle problem

$d=12$       IDS = 3,644,035 nodes  
                  $A^*(h_1) = 227$  nodes  
                  $A^*(h_2) = 73$  nodes

$d=24$       IDS = too many nodes  
                  $A^*(h_1) = 39,135$  nodes  
                  $A^*(h_2) = 1,641$  nodes

# Effective branching factor

---

- Effective branching factor  $b^*$ 
  - Is the branching factor that a uniform tree of depth  $d$  would have in order to contain  $N+1$  nodes.

$$N + 1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d$$

- Measure is fairly constant for sufficiently hard problems.
    - Can thus provide a good guide to the heuristic's overall usefulness.

# Effectiveness of different heuristics

|     | Search Cost |            |            | Effective Branching Factor |            |            |
|-----|-------------|------------|------------|----------------------------|------------|------------|
| $d$ | IDS         | $A^*(h_1)$ | $A^*(h_2)$ | IDS                        | $A^*(h_1)$ | $A^*(h_2)$ |
| 2   | 10          | 6          | 6          | 2.45                       | 1.79       | 1.79       |
| 4   | 112         | 13         | 12         | 2.87                       | 1.48       | 1.45       |
| 6   | 680         | 20         | 18         | 2.73                       | 1.34       | 1.30       |
| 8   | 6384        | 39         | 25         | 2.80                       | 1.33       | 1.24       |
| 10  | 47127       | 93         | 39         | 2.79                       | 1.38       | 1.22       |
| 12  | 3644035     | 227        | 73         | 2.78                       | 1.42       | 1.24       |
| 14  | —           | 539        | 113        | —                          | 1.44       | 1.23       |
| 16  | —           | 1301       | 211        | —                          | 1.45       | 1.25       |
| 18  | —           | 3056       | 363        | —                          | 1.46       | 1.26       |
| 20  | —           | 7276       | 676        | —                          | 1.47       | 1.27       |
| 22  | —           | 18094      | 1219       | —                          | 1.48       | 1.28       |
| 24  | —           | 39135      | 1641       | —                          | 1.48       | 1.26       |

- Results averaged over random instances of the 8-puzzle

# Inventing heuristics via “relaxed problems”

---

- A problem with fewer restrictions on the actions is called a **relaxed problem**
- The cost of an optimal solution to a relaxed problem is an admissible heuristic for the original problem
- If the rules of the 8-puzzle are relaxed so that a tile can move **anywhere**, then  $h_1(n)$  gives the shortest solution
- If the rules are relaxed so that a tile can move to **any adjacent square**, then  $h_2(n)$  gives the shortest solution
- Can be a useful way to generate heuristics
  - E.g., ABSOLVER (Prieditis, 1993) discovered the first useful heuristic for the Rubik’s cube puzzle

## More on heuristics

---

- $h(n) = \max\{ h_1(n), h_2(n), \dots, h_k(n) \}$ 
  - Assume all  $h$  functions are admissible
  - Always choose the least optimistic heuristic (most accurate) at each node
  - Could also learn a convex combination of features
    - Weighted sum of  $h(n)$ 's, where weights sum to 1
    - Weights learned via repeated puzzle-solving
- Could try to learn a heuristic function based on “features”
  - E.g.,  $x_1(n)$  = number of misplaced tiles
  - E.g.,  $x_2(n)$  = number of goal-adjacent-pairs that are currently adjacent
  - $h(n) = w_1 x_1(n) + w_2 x_2(n)$ 
    - Weights could be learned again via repeated puzzle-solving
    - Try to identify which features are predictive of path cost



# Summary

---

- Uninformed search methods have their limits
- Informed (or heuristic) search uses problem-specific heuristics to improve efficiency
  - Best-first
  - A\*
  - RBFS
  - SMA\*
  - Techniques for generating heuristics
- Can provide significant speed-ups in practice
  - e.g., on 8-puzzle
  - But can still have worst-case exponential time complexity
- Next lecture: local search techniques
  - Hill-climbing, genetic algorithms, simulated annealing, etc

## Assignment 2

### Topic: Adversarial Search

The task of this assignment is to implement a game player which uses adversarial search algorithms to play a two-player game. The game that will be used is Mancala

(<https://www.mathplayground.com/mancala.html> provides an online implementation. You are strongly encouraged to have a look at this site to know the rules)

The basic implementation includes minimax algorithm with alpha-beta pruning. You can experiment with different search strategies. For example, you can experiment with iterative deepening search. You can also carry out experiments by varying depth-limits and changing move-ordering.

You need to implement various heuristics and find out which one is better by running experiments. For example, you can determine the win-loss ratio by running 100 games with computer vs computer auto-play.

A heuristic function estimates how good a particular state is for a player. A number of factors determine whether a given state of the game is good for a player.

#### **How to Play:**

In Mancala, the board consists of six (6) bins on each side, and a home position (called storage) on the right of the bins. The board is laid out typically starting with four stones in each bin and both storage bins empty.

The following link contains a description of rules of Mancala.

<https://www.thesprucecrafts.com/how-to-play-mancala-409424>

#### **Mancala heuristics:**

For Mancala, the following strategic factors can be used to design a good heuristic to determine how favorable a particular position is for a player:

- 1) First valid move [furthest valid bin (a bin on my side which is not empty) from my storage]
- 2) How far ahead of my opponent I am now [the difference in stone count between my storage and opponent's storage]
- 3) How close I am to winning (If my storage is already close to containing half of total number of stones)
- 4) How close opponent is to winning
- 5) The total number of stones residing in the six bins of my side
- 6) The total number of stones residing in the six bins of opponent's side
- 7) Number of stones close to my storage ( a stone, although residing in a bin on my side, is

not

8) close to my storage, if it is going to be overflowed to my opponent's side )

9) Number of stones close to my opponent's storage

10) Have I earned an extra move

11) Have I captured any stone

You can experiment with the following four heuristics (and some of your own heuristics):

**heuristic-1:** The evaluation function is

$(\text{stones\_in\_my\_storage} - \text{stones\_in\_opponents\_storage})$

**heuristic-2:** The evaluation function is

$W1 * (\text{stones\_in\_my\_storage} - \text{stones\_in\_opponents\_storage}) + W2 * (\text{stones\_on\_my\_side} - \text{stones\_on\_opponents\_side})$

**heuristic-3:** The evaluation function is

$W1 * (\text{stones\_in\_my\_storage} - \text{stones\_in\_opponents\_storage}) + W2 * (\text{stones\_on\_my\_side} - \text{stones\_on\_opponents\_side}) + W3 * (\text{additional\_move\_earned})$

**heuristic-4:** The evaluation function is

$W1 * (\text{stones\_in\_my\_storage} - \text{stones\_in\_opponents\_storage}) + W2 * (\text{stones\_on\_my\_side} - \text{stones\_on\_opponents\_side}) + W3 * (\text{additional\_move\_earned}) + W4 * (\text{stones\_captured})$

**Chapter 5:**  
**Adversarial Search**  
**&**  
**Game Playing**

## Typical assumptions

---

- Two agents whose actions alternate
- Utility values for each agent are the opposite of the other
  - creates the adversarial situation
- Fully observable environments
- In game theory terms:
  - “Deterministic, turn-taking, zero-sum games of perfect information”
- Can generalize to stochastic games, multiple players, non zero-sum, etc

# Search versus Games

---

- Search – no adversary
  - Solution is (heuristic) method for finding goal
  - Heuristics and CSP techniques can find *optimal* solution
  - Evaluation function: estimate of cost from start to goal through given node
  - Examples: path planning, scheduling activities
- Games – adversary
  - Solution is strategy (strategy specifies move for every possible opponent reply).
  - Time limits force an *approximate* solution
  - Evaluation function: evaluate “goodness” of game position
  - Examples: chess, checkers, Othello, backgammon

# Types of Games

---

|                       | deterministic                   | chance                                 |
|-----------------------|---------------------------------|----------------------------------------|
| perfect information   | chess, checkers,<br>go, othello | backgammon<br>monopoly                 |
| imperfect information |                                 | bridge, poker, scrabble<br>nuclear war |

# Game Setup

---

- Two players: **MAX** and **MIN**
- MAX moves first and they take turns until the game is over
  - Winner gets award, loser gets penalty.
- Games as search:
  - **Initial state**: e.g. board configuration of chess
  - **Successor function**: list of (move, state) pairs specifying legal moves.
  - **Terminal test**: Is the game finished?
  - **Utility function**: Gives numerical value of terminal states. E.g. **win (+1)**, **lose (-1)** and **draw (0)** in tic-tac-toe or chess
- MAX uses search tree to determine next move.



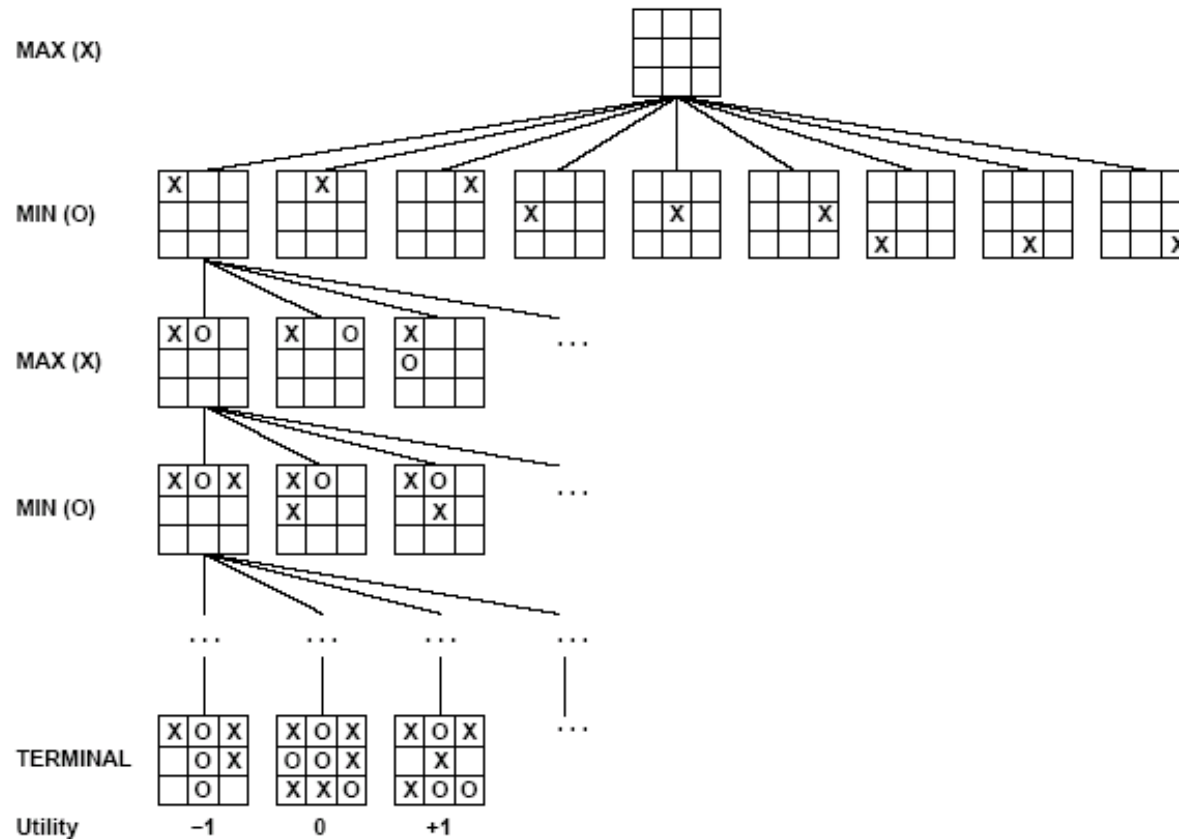
# Size of search trees

---

- $b$  = branching factor
- $d$  = number of moves by both players
- Search tree is  $O(b^d)$
- Chess
  - $b \sim 35$
  - $d \sim 100$ 
    - search tree is  $\sim 10^{154}$  (!!)
    - completely impractical to search this
- Game-playing emphasizes being able to **make optimal decisions** in a **finite amount of time**
  - Somewhat realistic as a model of a real-world agent
  - Even if games themselves are artificial

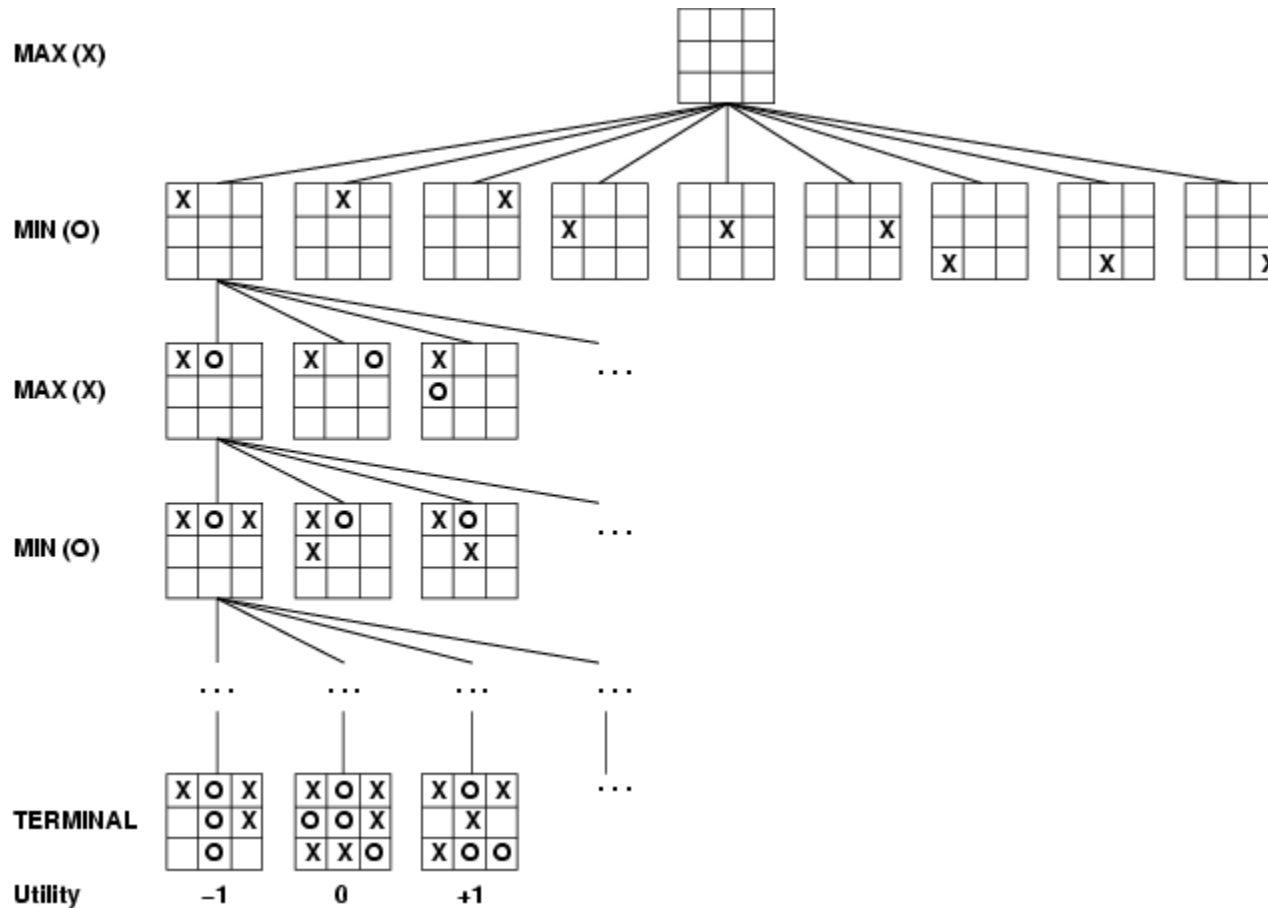
# Partial Game Tree for Tic-Tac-Toe

---



# Game tree (2-player, deterministic, turns)

---



How do we search this tree to find the optimal move?

# Minimax strategy

---

- Find the optimal *strategy* for MAX assuming an infallible MIN opponent
  - Need to compute this all the down the tree
- Assumption: Both players play optimally!
- Given a game tree, the optimal strategy can be determined by using the minimax value of each node:

**MINIMAX-VALUE( $n$ )=**

**UTILITY( $n$ )**

**$\max_{s \in \text{successors}(n)} \text{MINIMAX-VALUE}(s)$**

**$\min_{s \in \text{successors}(n)} \text{MINIMAX-VALUE}(s)$**

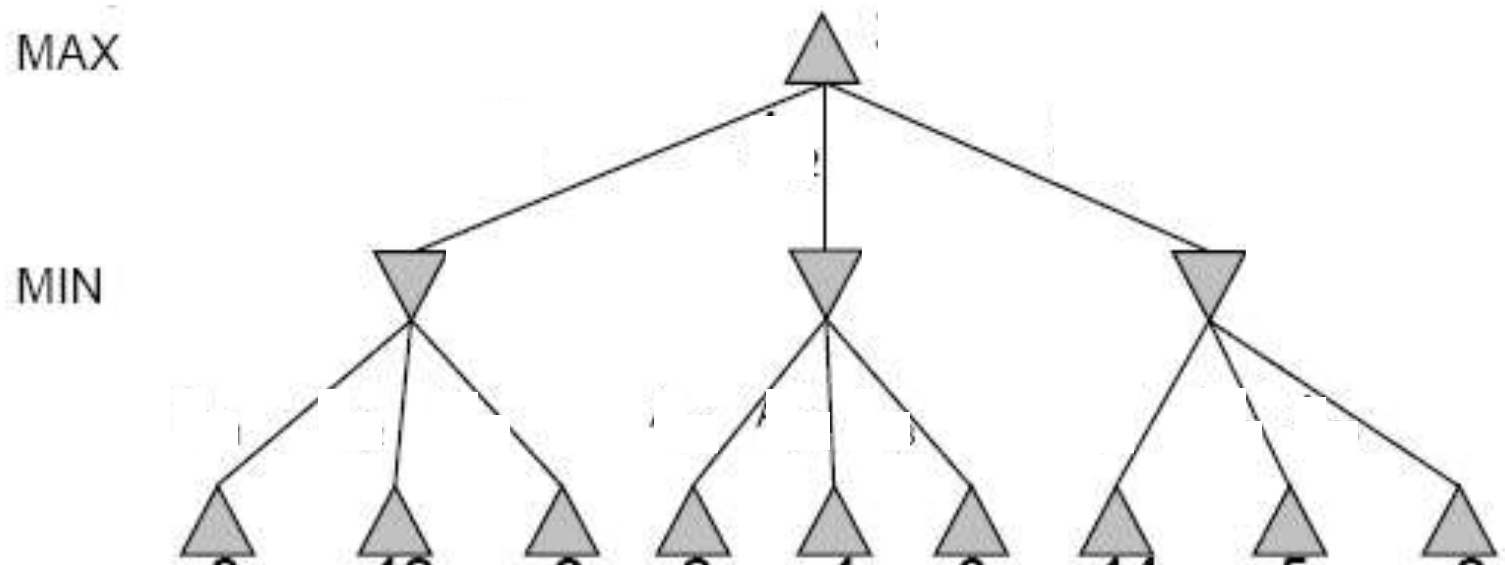
**If  $n$  is a terminal**

**If  $n$  is a max node**

**If  $n$  is a min node**

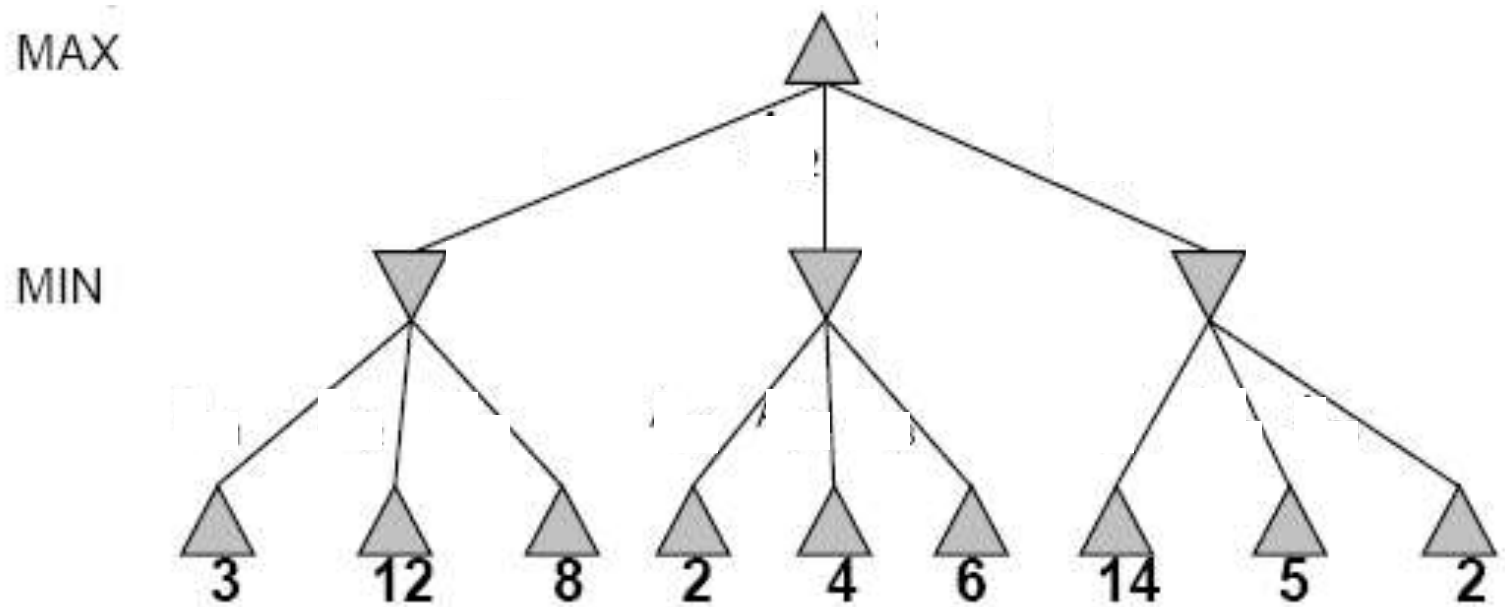
# Two-Ply Game Tree

---



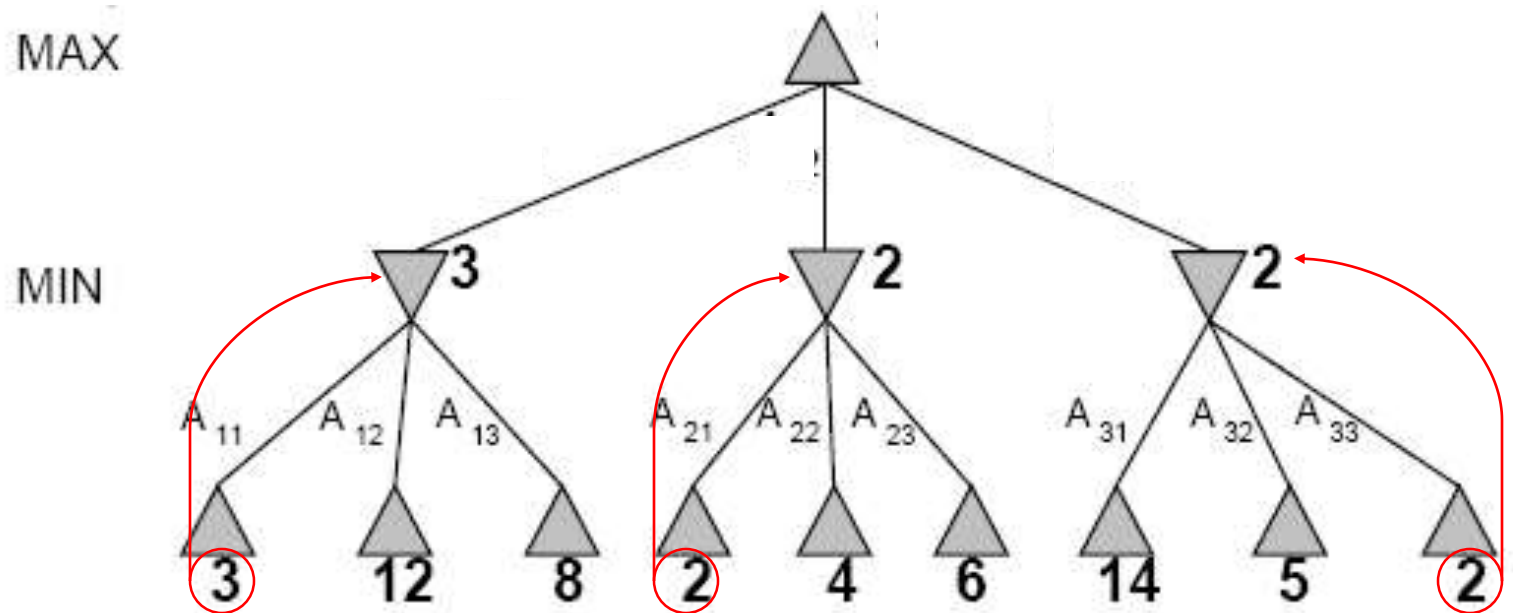
## Two-Ply Game Tree

---



# Two-Ply Game Tree

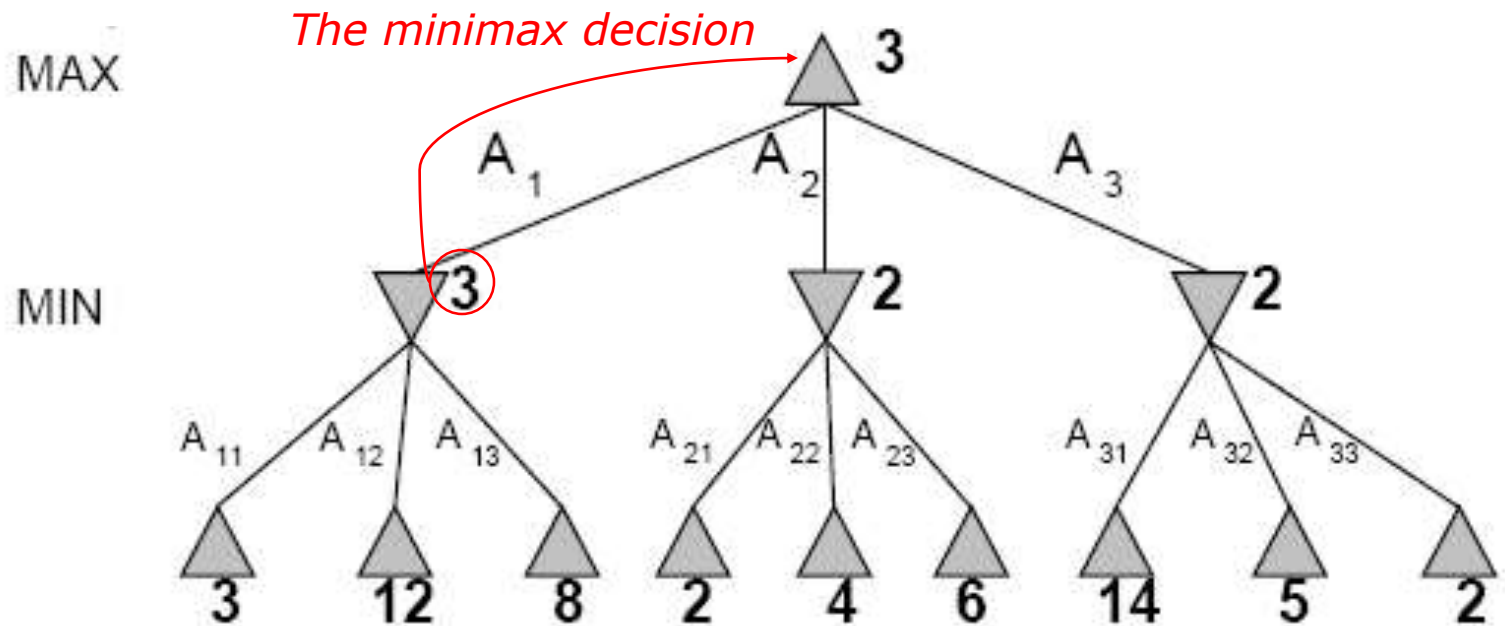
---



# Two-Ply Game Tree

---

Minimax maximizes the utility for the worst-case outcome for max





## What if MIN does not play optimally?

---

- Definition of optimal play for MAX assumes MIN plays optimally:
  - maximizes worst-case outcome for MAX
- But if MIN does not play optimally, MAX will do even better
  - Can prove this (Problem 6.2)

# Minimax Algorithm

---

- Complete depth-first exploration of the game tree
- Assumptions:
  - Max depth =  $d$ ,  $b$  legal moves at each point
  - E.g., Chess:  $d \sim 100$ ,  $b \sim 35$

| Criterion | Minimax     |
|-----------|-------------|
| Time      | $O(b^m)$ ☹️ |
| Space     | $O(bm)$ 😊   |

# Pseudocode for Minimax Algorithm

---

**function** MINIMAX-DECISION(*state*) **returns** *an action*

**inputs:** *state*, current state in game

$v \leftarrow \text{MAX-VALUE}(\text{state})$

**return** the *action* in  $\text{SUCCESSORS}(\text{state})$  with value  $v$

---

**function** MAX-VALUE(*state*) **returns** *a utility value*

**if**  $\text{TERMINAL-TEST}(\text{state})$  **then return**  $\text{UTILITY}(\text{state})$

$v \leftarrow \infty$

**for**  $a, s$  in  $\text{SUCCESSORS}(\text{state})$  **do**

$v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s))$

**return**  $v$

---

**function** MIN-VALUE(*state*) **returns** *a utility value*

**if**  $\text{TERMINAL-TEST}(\text{state})$  **then return**  $\text{UTILITY}(\text{state})$

$v \leftarrow \infty$

**for**  $a, s$  in  $\text{SUCCESSORS}(\text{state})$  **do**

$v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s))$

**return**  $v$

# Multiplayer games

---

- Games allow more than two players
- Single minimax values become vectors

to move

A

(1, 2, 6)

B

(1, 2, 6)

(-1, 5, 2)

C

(1, 2, 6)

X

(6, 1, 2)

(-1, 5, 2)

(5, 4, 5)

A

(1, 2, 6)

(4, 2, 3)

(6, 1, 2)

(7, 4, -1)

(5, -1, -1)

(-1, 5, 2)

(7, 7, -1)

(5, 4, 5)

## Aspects of multiplayer games

---

- Previous slide (standard minimax analysis) assumes that each player operates to maximize only their own utility
- In practice, players make alliances
  - E.g, C strong, A and B both weak
  - May be best for A and B to attack C rather than each other
- If game is not zero-sum (i.e.,  $\text{utility}(A) = - \text{utility}(B)$ ) then alliances can be useful even with 2 players
  - e.g., both cooperate to maximum the sum of the utilities

# Practical problem with minimax search

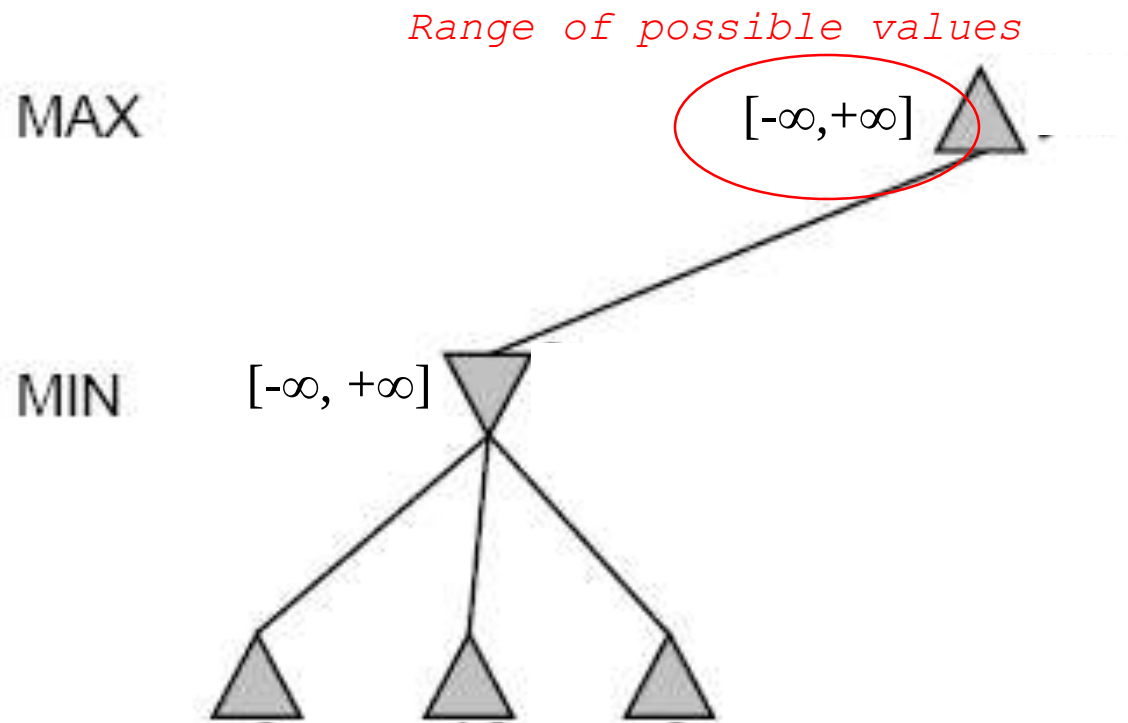
---

- Number of game states is **exponential** in the number of moves.
  - Solution: **Do not examine every node**  
=> **pruning**
    - Remove branches that do not influence final decision
- Revisit example ...

# Alpha-Beta Example

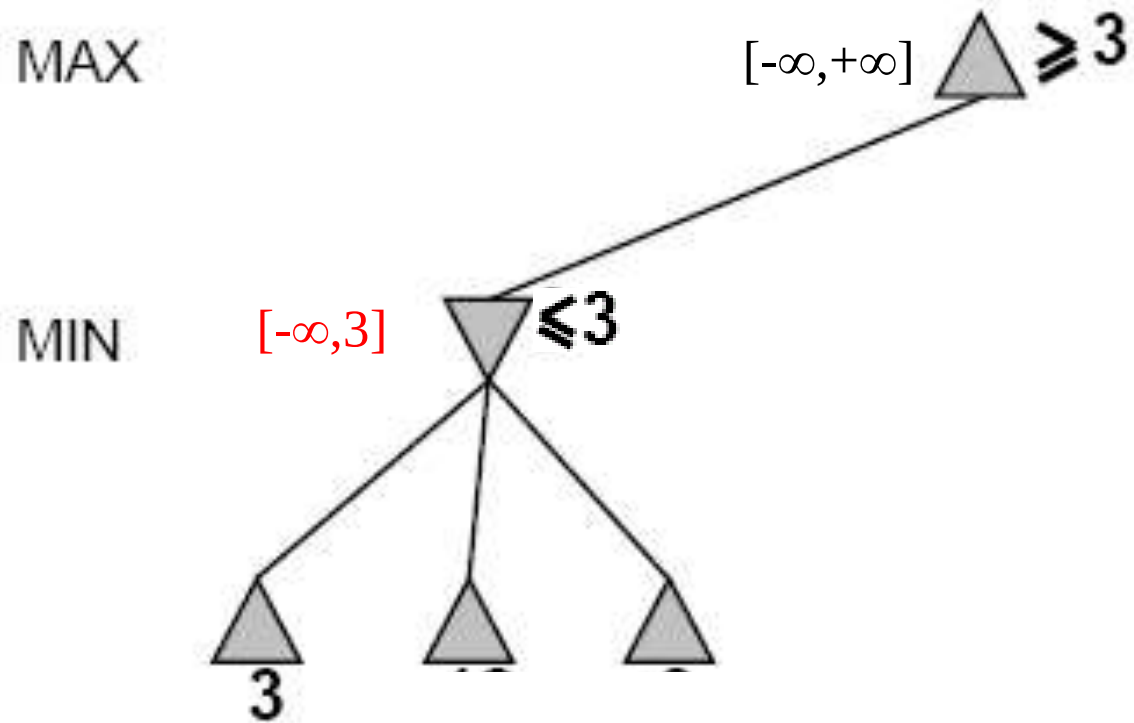
---

Do DF-search until first leaf



## Alpha-Beta Example (continued)

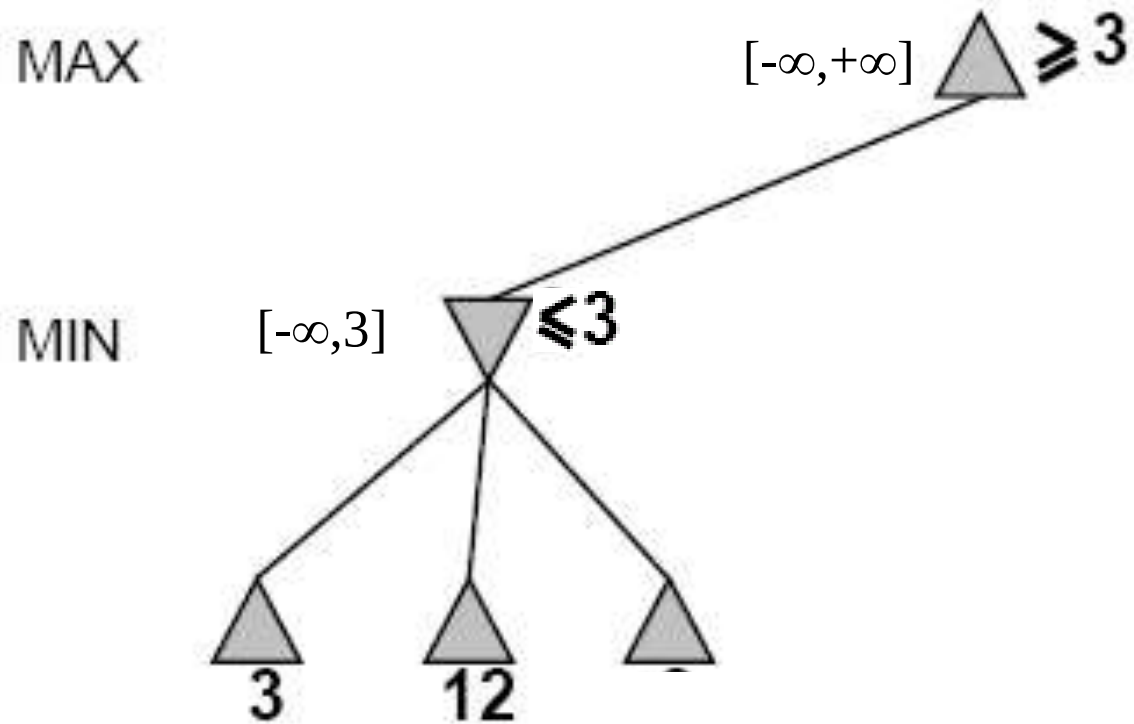
---





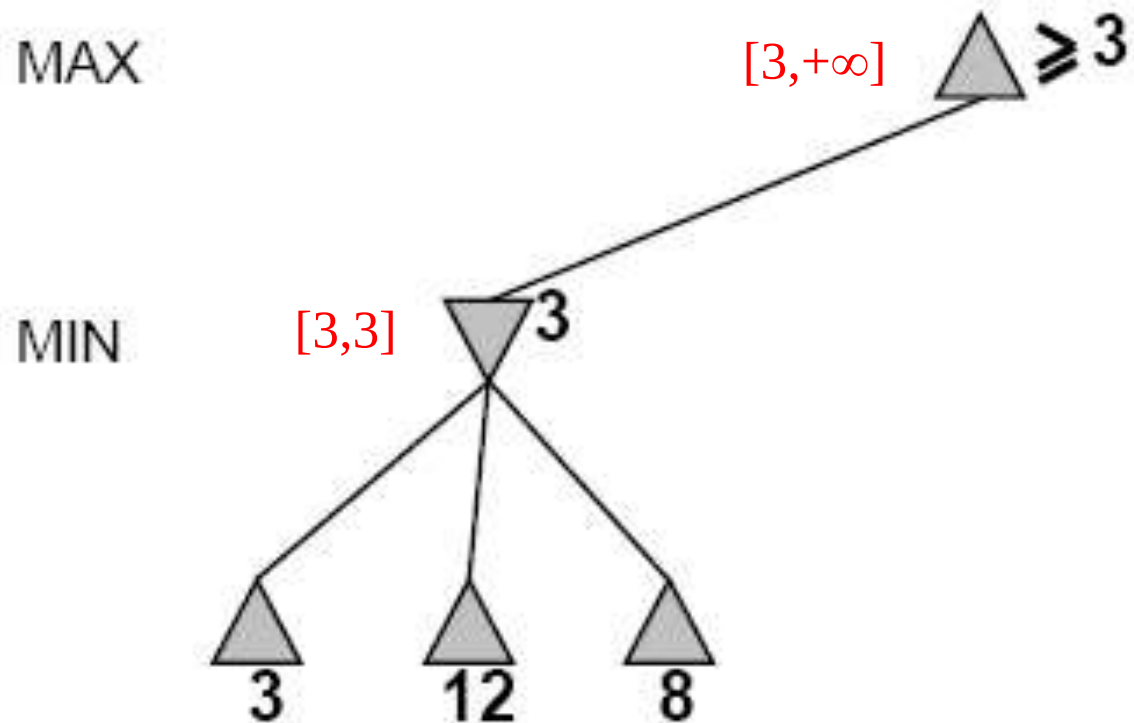
## Alpha-Beta Example (continued)

---



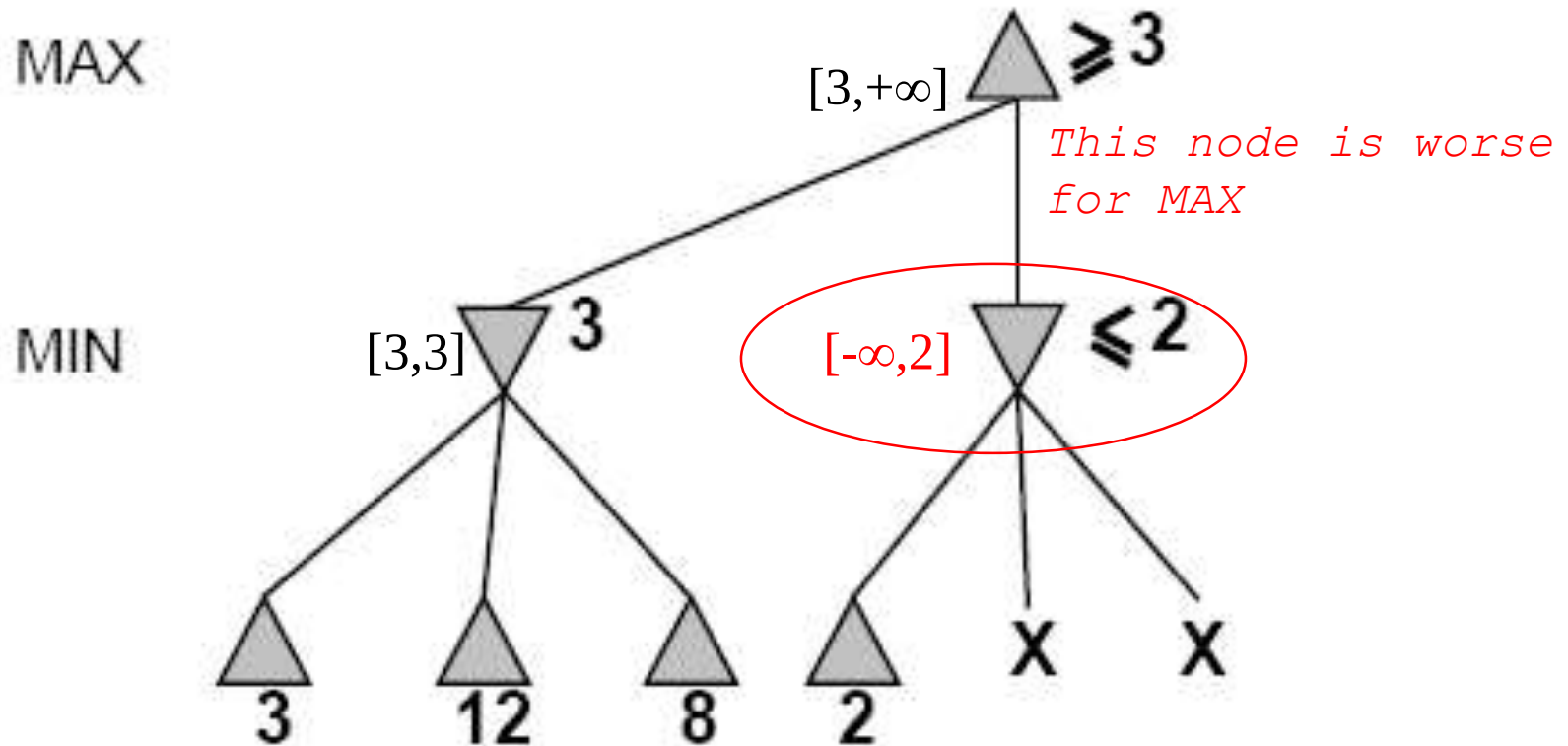
## Alpha-Beta Example (continued)

---



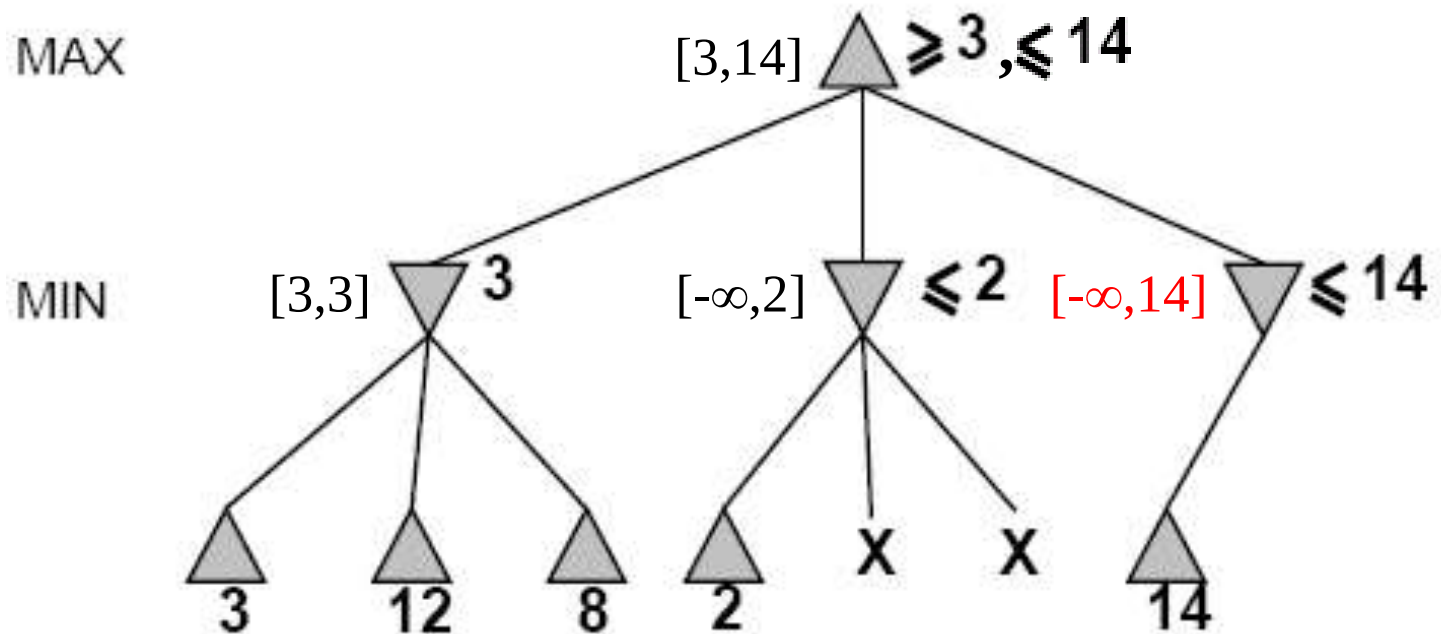
## Alpha-Beta Example (continued)

---



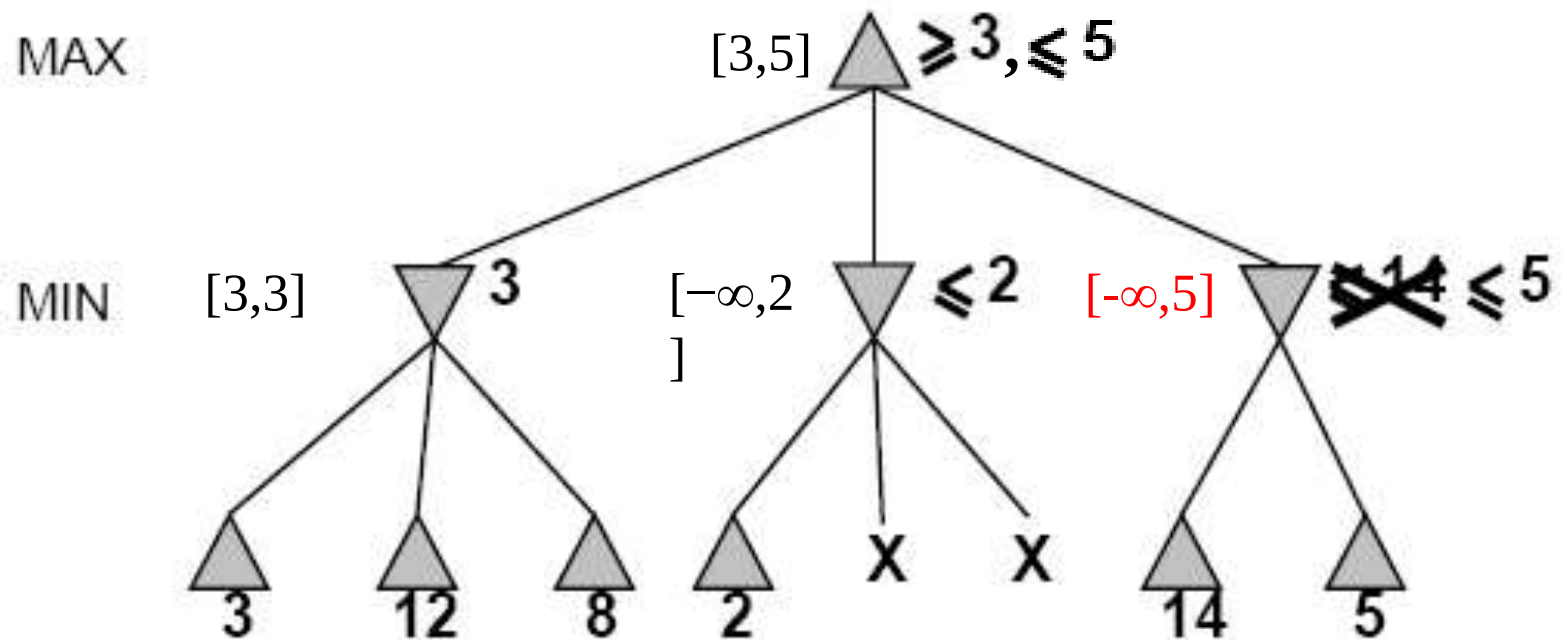
## Alpha-Beta Example (continued)

---



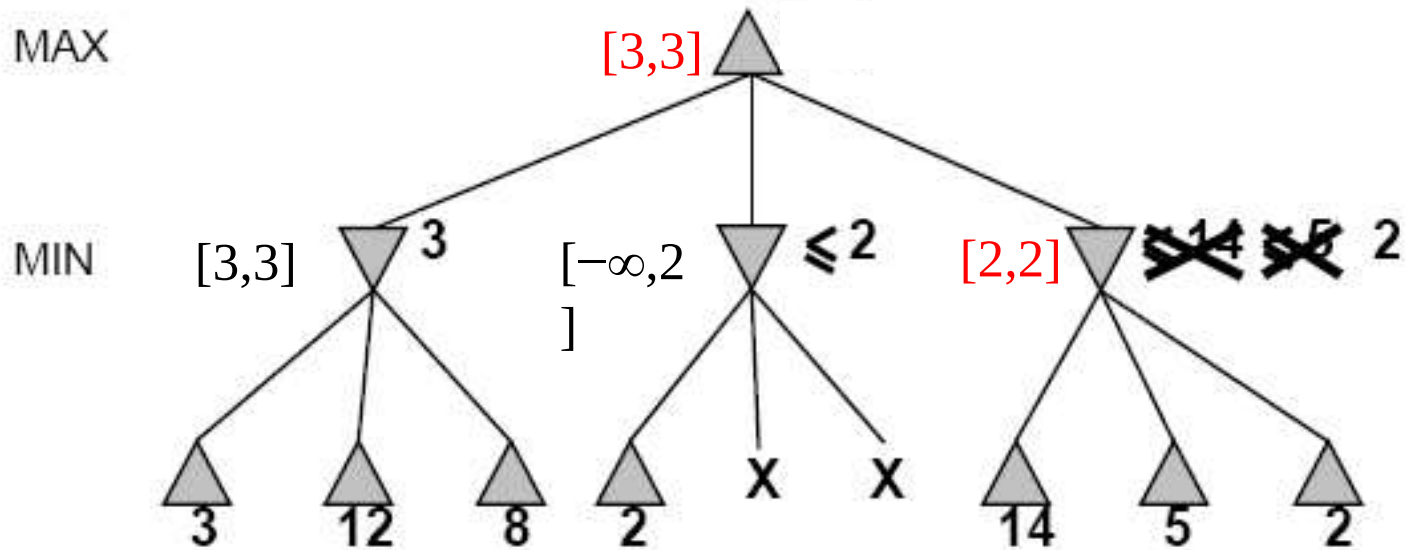
## Alpha-Beta Example (continued)

---



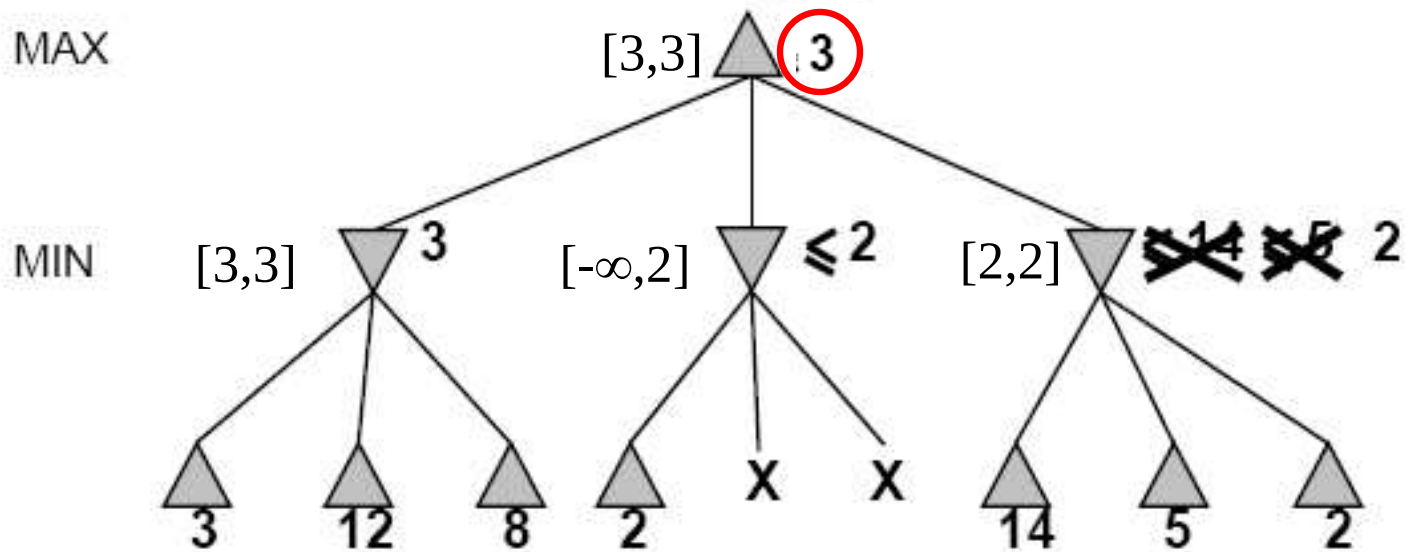
## Alpha-Beta Example (continued)

---



## Alpha-Beta Example (continued)

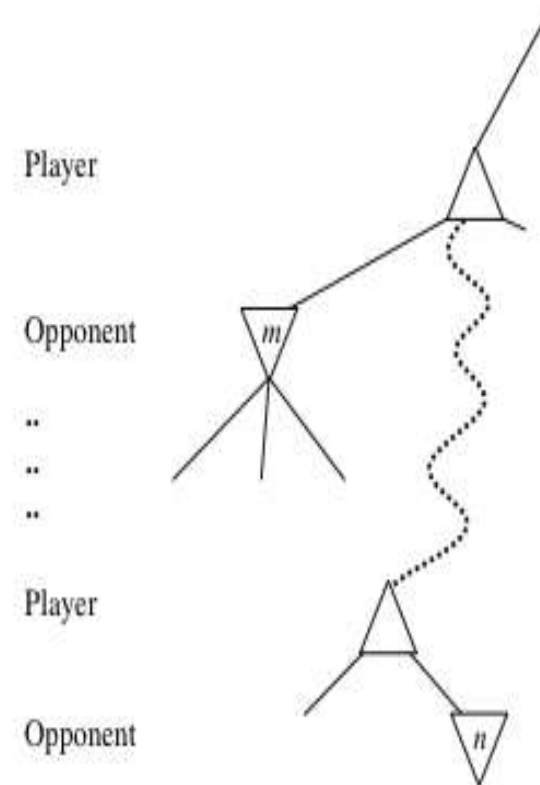
---



# General alpha-beta pruning

---

- Consider a node  $n$  somewhere in the tree
- If player has a better choice at
  - Parent node of  $n$
  - Or any choice point further up
- $n$  will **never** be reached in actual play.
- Hence when enough is known about  $n$ , it can be pruned.





# Alpha-beta Algorithm

---

- Depth first search – only considers nodes along a single path at any time

$\alpha$  = highest-value choice we have found at any choice point along the path for MAX

$\beta$  = lowest-value choice we have found at any choice point along the path for MIN

- update values of  $\alpha$  and  $\beta$  during search and prunes remaining branches as soon as the value is known to be worse than the current  $\alpha$  or  $\beta$  value for MAX or MIN

# Effectiveness of Alpha-Beta Search

---

- Worst-Case
  - branches are ordered so that no pruning takes place. In this case alpha-beta gives no improvement over exhaustive search
- Best-Case
  - each player's best move is the left-most alternative (i.e., evaluated first)
  - in practice, performance is closer to best rather than worst-case
- In practice often get  $O(b^{(d/2)})$  rather than  $O(b^d)$ 
  - this is the same as having a branching factor of  $\sqrt{b}$ ,
    - since  $(\sqrt{b})^d = b^{(d/2)}$
    - i.e., we have effectively gone from  $b$  to square root of  $b$
  - e.g., in chess go from  $b \sim 35$  to  $b \sim 6$ 
    - this permits much deeper search in the same amount of time

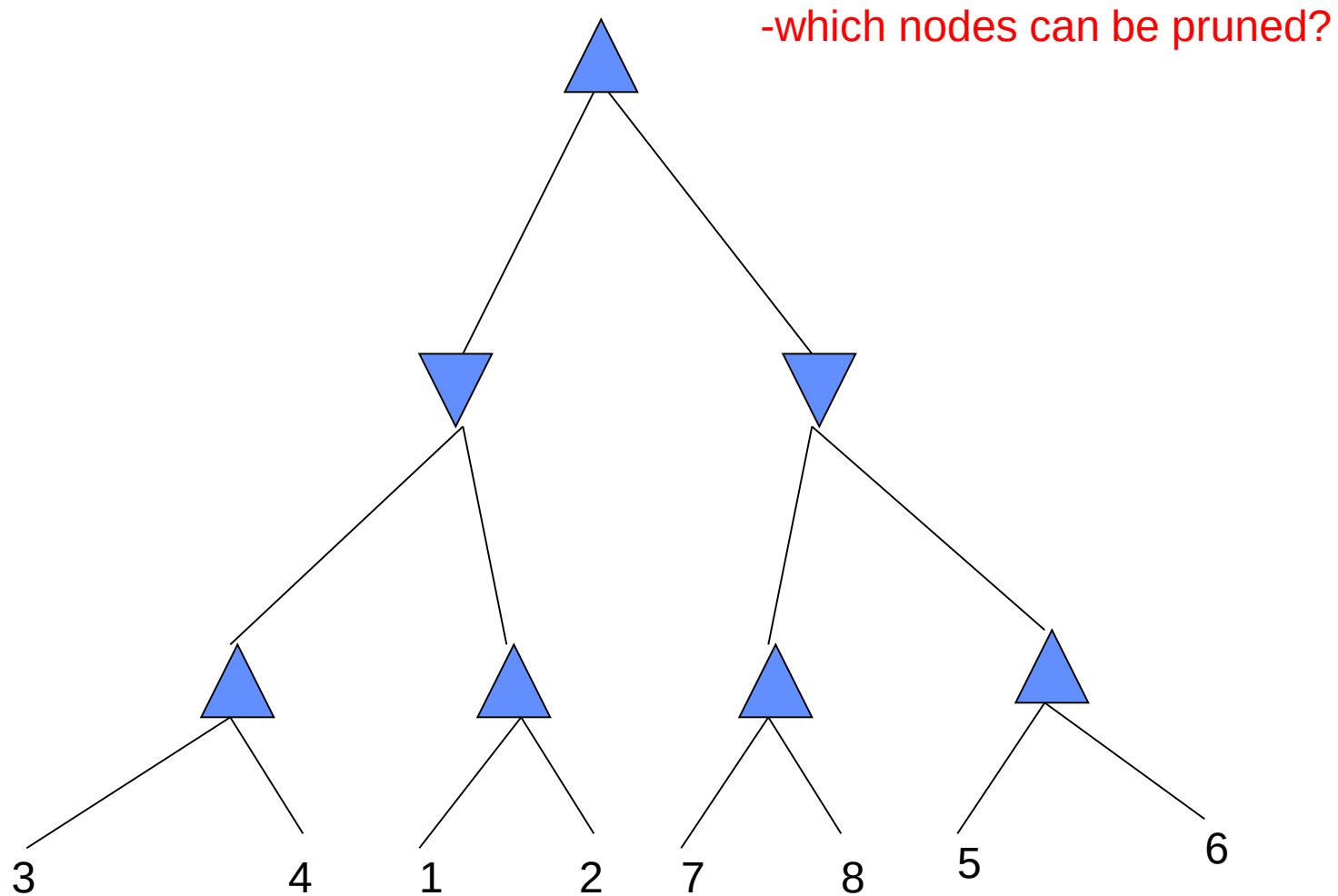
## Final Comments about Alpha-Beta Pruning

---

- Pruning does not affect final results
- Entire subtrees can be pruned.
- Good move *ordering* improves effectiveness of pruning
- Repeated states are again possible.
  - Store them in memory = transposition table

## Example

---



# Practical Implementation

---

How do we make these ideas practical in real game trees?

Standard approach:

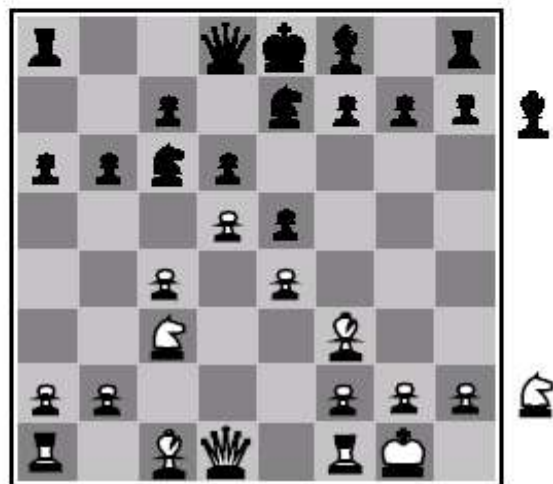
- **cutoff test:** (where do we stop descending the tree)
  - depth limit
  - better: iterative deepening
  - cutoff only when no big changes are expected to occur next (**quiescence search**).
- **evaluation function**
  - When the search is cut off, we evaluate the current state by estimating its utility. This estimate is captured by the evaluation function.

# Static (Heuristic) Evaluation Functions

---

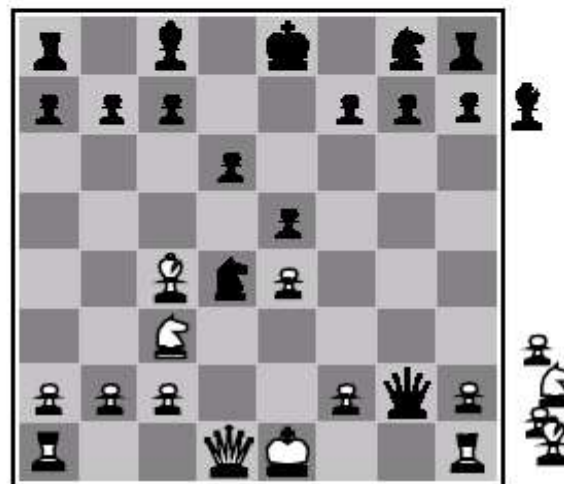
- An Evaluation Function:
  - estimates how good the current board configuration is for a player.
  - Typically, one figures how good it is for the player, and how good it is for the opponent, and subtracts the opponents score from the players
  - Othello: Number of white pieces - Number of black pieces
  - Chess: Value of all white pieces - Value of all black pieces
- Typical values from -infinity (loss) to +infinity (win) or  $[-1, +1]$ .
- If the board evaluation is  $X$  for a player, it's  $-X$  for the opponent
- Example:
  - Evaluating chess boards,
  - Checkers
  - Tic-tac-toe

## Evaluation functions



Black to move

White slightly better



White to move

Black winning

For chess, typically *linear* weighted sum of *features*

$$Eval(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

e.g.,  $w_1 = 9$  with

$f_1(s) = (\text{number of white queens}) - (\text{number of black queens}), \text{ etc.}$

## Iterative (Progressive) Deepening

---

- In real games, there is usually a time limit  $T$  on making a move
- How do we take this into account?
  - using alpha-beta we cannot use “partial” results with any confidence unless the full breadth of the tree has been searched
  - So, we could be conservative and set a conservative depth-limit which guarantees that we will find a move in time  $< T$ 
    - disadvantage is that we may finish early, could do more search
- In practice, iterative deepening search (IDS) is used
  - IDS runs depth-first search with an increasing depth-limit
  - when the clock runs out we use the solution found at the previous depth limit



# Heuristics and Game Tree Search

---

- The Horizon Effect
  - sometimes there's a major "effect" (such as a piece being captured) which is just "below" the depth to which the tree has been expanded
  - the computer cannot see that this major event could happen
  - it has a "limited horizon"
  - there are heuristics to try to follow certain branches more deeply to detect to such important events
  - this helps to avoid catastrophic losses due to "short-sightedness"
- Heuristics for Tree Exploration
  - it may be better to explore some branches more deeply in the allotted time
  - various heuristics exist to identify "promising" branches

# The State of Play

---

- Checkers:
  - Chinook ended 40-year-reign of human world champion Marion Tinsley in 1994.
- Chess:
  - Deep Blue defeated human world champion Garry Kasparov in a six-game match in 1997.
- Othello:
  - human champions refuse to compete against computers: they are too good.
- Go:
  - human champions refuse to compete against computers: they are too bad
  - $b > 300$  (!)
- See (e.g.) <http://www.cs.ualberta.ca/~games/> for more information

# Deep Blue

---

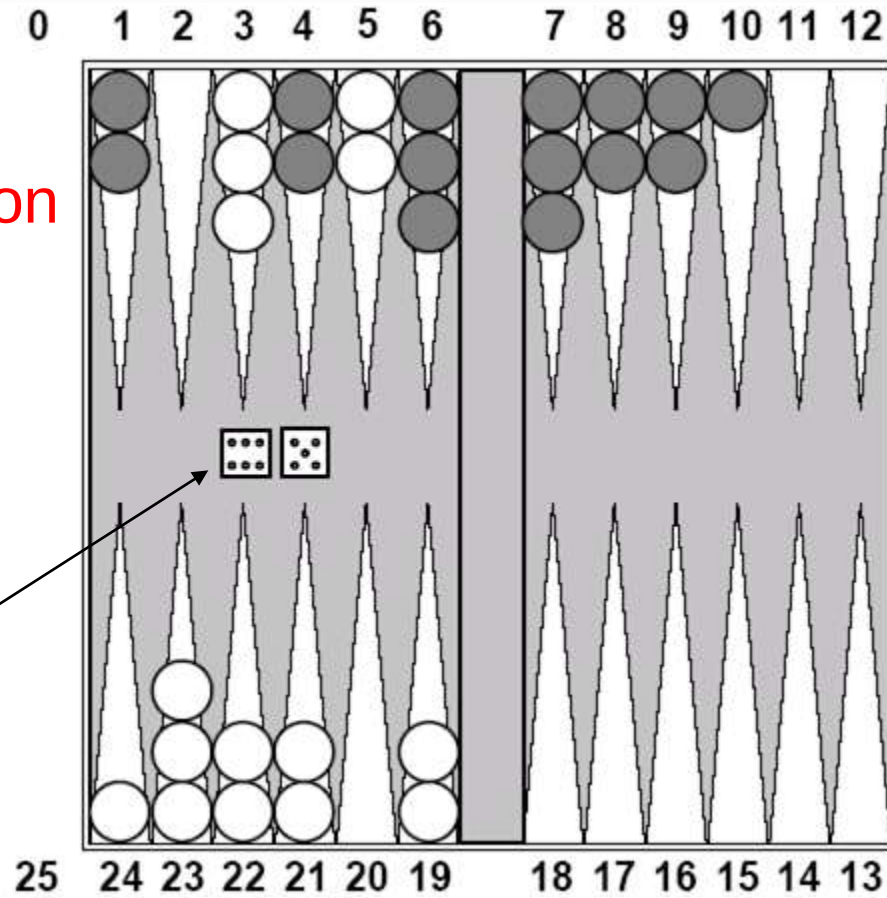
- 1957: Herbert Simon
  - “within 10 years a computer will beat the world chess champion”
- 1997: Deep Blue beats Kasparov
- Parallel machine with 30 processors for “software” and 480 VLSI processors for “hardware search”
- Searched 126 million nodes per second on average
  - Generated up to 30 billion positions per move
  - Reached depth 14 routinely
- Uses iterative-deepening alpha-beta search with transpositioning
  - Can explore beyond depth-limit for interesting moves

## Chance Games.

---

Backgammon

your element of  
chance



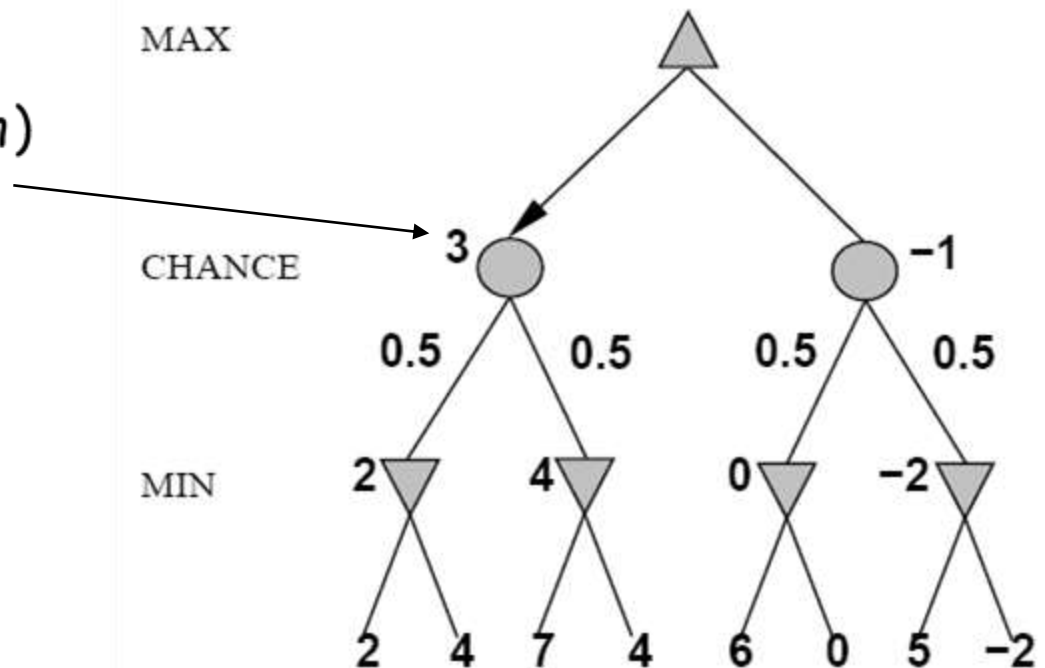
# Expected Minimax

$$v = \sum_{\text{chance nodes}} P(n) \times \text{Minimax}(n)$$

$$3 = 0.5 \times 4 + 0.5 \times 2$$

Interleave chance nodes  
with min/max nodes

Again, the tree is constructed  
bottom-up



## Summary

---

- Game playing can be effectively modeled as a search problem
- Game trees represent alternate computer/opponent moves
- Evaluation functions estimate the quality of a given board configuration for the Max player.
- Minimax is a procedure which chooses moves by assuming that the opponent will always choose the move which is best for them
- Alpha-Beta is a procedure which can prune large parts of the search tree and allow search to go deeper
- For many well-known games, computer algorithms based on heuristic search match or outperform human world experts.

## Solving the Travelling Salesman Problem (TSP) by local search

The motivation behind the Travelling Salesman Problem (also known as Travelling Salesperson Problem) is the problem faced by a salesperson who needs to visit a number of customers located in different cities and tries to find the shortest round trip accomplishing this task. In a more general and abstract formulation, the TSP is, given a directed, edge-weighted graph, to find a shortest cyclic path that visits every node in this graph exactly once. Often, the TSP is defined in such a way that the underlying graphs are always complete graphs, that is, any pair of vertices is connected by an edge.

In this assignment task, we assume that TSP instances are specified as complete graphs. In addition, we assume the vertices correspond to the points in a Euclidean space, and the edge weight between any two vertices corresponds to the Euclidean distance between the respective points.

In local search paradigm, we start at some position in search space and iteratively move from the current position to a neighbouring position. In constructive search, the search space contains partial candidate solutions and the search step consists of doing extension with one or more solution components. In perturbative search, the search space contains complete candidate solutions and the search step consists of doing modification of one or more solution components.

Both the constructive and the perturbative search can be described under the local search paradigm. While in the literature, the term local search is mostly used for perturbative search methods, it also applies to constructive search methods. Perturbative search methods start from some complete candidate solution. The task of generating such candidate solutions is commonly accomplished by constructive search methods or construction heuristics. Constructive search methods iteratively extend an initially empty candidate solution by one or several solution components until a complete candidate solution is obtained. Constructive search methods can thus be seen as operating in a search space of partial candidate solutions.

### Constructive search methods

The *Nearest Neighbour Heuristic (NNH)* starts tour construction from some randomly chosen vertex  $u_1$  in the given graph and then iteratively extends the current partial tour  $p = (u_1, \dots, u_k)$  with an unvisited vertex  $u_{k+1}$  that is connected to  $u_k$  by a minimum weight edge ( $u_{k+1}$  is called a *nearest neighbour* of  $u_k$ ); when all vertices have been visited, a complete tour is obtained by extending  $p$  with the initial vertex,  $u_1$ . The tours constructed by the NNH are called *nearest neighbour tours*.

*Insertion heuristics* construct tours in a way that is different from that underlying the NNH; in each step, they extend the current partial tour  $p$  by inserting a heuristically chosen vertex at a position that typically leads to a minimal length increase. Several variants of these heuristics exist, including

- (i) *Nearest insertion* construction heuristics, where the next vertex to be inserted is a vertex  $u_i$  with minimum distance to any vertex  $u_j$  in  $p$ ;
- (ii) *Cheapest insertion*, which inserts a vertex that leads to the minimum increase of the weight of  $p$  over all vertices not yet in  $p$ ;
- (iii) *Farthest insertion*, where the next vertex to be inserted is a vertex  $u_i$  for which the minimum distance to a vertex in  $p$  is maximal;
- (iv) *Random insertion*, where the next vertex to be inserted is chosen randomly.

The **Greedy Heuristic**: The construction heuristics discussed so far build a complete tour by iteratively extending a connected partial tour. An alternative approach is to iteratively build several tour fragments that are ultimately patched together into a complete tour. One example for a construction heuristic of this type is the *Greedy Heuristic*, which works as follows. First, all edges in the given graph  $G$  are sorted according to increasing weight. Then, this list is scanned, starting from the minimum weight edge, in linear order. An edge  $e$  is added to the current partial candidate solution  $p$  if inserting it into  $G'$ , the graph that contains all vertices of  $G$  and the edges in  $p$ , does not result in any vertices of degree greater than two or any cycles of length less than  $n$  edges.

For construction heuristics, you may also consider the simple Minimum Spanning Tree (MST) based heuristic and the Christofides heuristic.

## Perturbative search methods

**2-opt** and **3-opt** are iterative improvement algorithms based on  $k$ -exchange neighbourhoods. Various speed-up techniques, including fixed-radius near neighbour search and candidate lists may be considered. **Or-opt** and **2.5-opt** are closely related to 2-opt and 3-opt algorithms.

In addition, **Node Shift** and **Node Swap** are simple iterative improvement algorithms. A Node Shift neighborhood is generated by shifting a city to other position in tour, while a node swap neighborhood is generated by swapping two cities in tour.

In this assignment, you need to implement at least three heuristics from each of the constructive and the perturbative category and present the result of the combinations. You should run experiments on the provided 21 benchmark data sets. For experimentation, you may consider semi-greedy versions in the constructive category. For example, instead of the nearest candidate, you may randomly select any of the top 3 or top 5 vertices in Nearest Neighbour Heuristic (NNH). Similar semi-greedy versions may be considered for nearest/cheapest/farthest insertion heuristic where you may select any of the top 3 or top 5 of the nearest/cheapest/farthest candidates for insertion for extending the current partial solution.



**Chapter 4 (Section 4.3, ...) 2<sup>nd</sup> Edition**  
**or**  
**Chapter 4 (3<sup>rd</sup> Edition)**  
**Local Search and Optimization**

# Outline

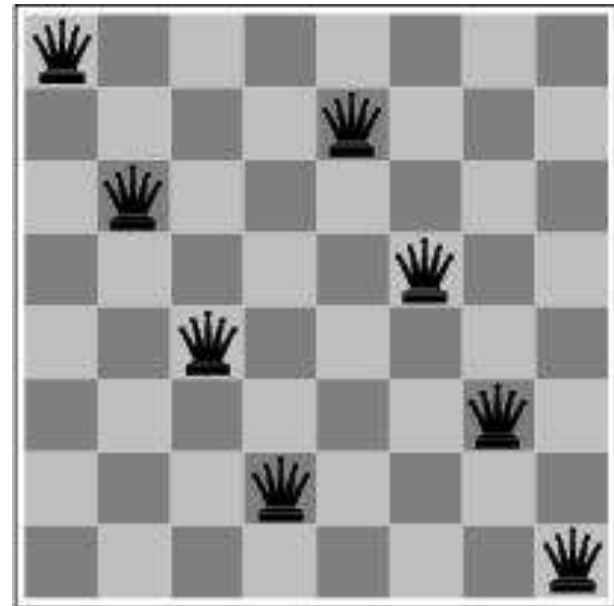
---

- Local search techniques and optimization
  - Hill-climbing
  - Gradient methods
  - Simulated annealing
  - Genetic algorithms
  - Issues with local search

# Local search and optimization

---

- Previously: systematic exploration of search space.
  - Path to goal is solution to problem
- YET, for some problems path is irrelevant.
  - E.g 8-queens
- Different algorithms can be used
  - Local search



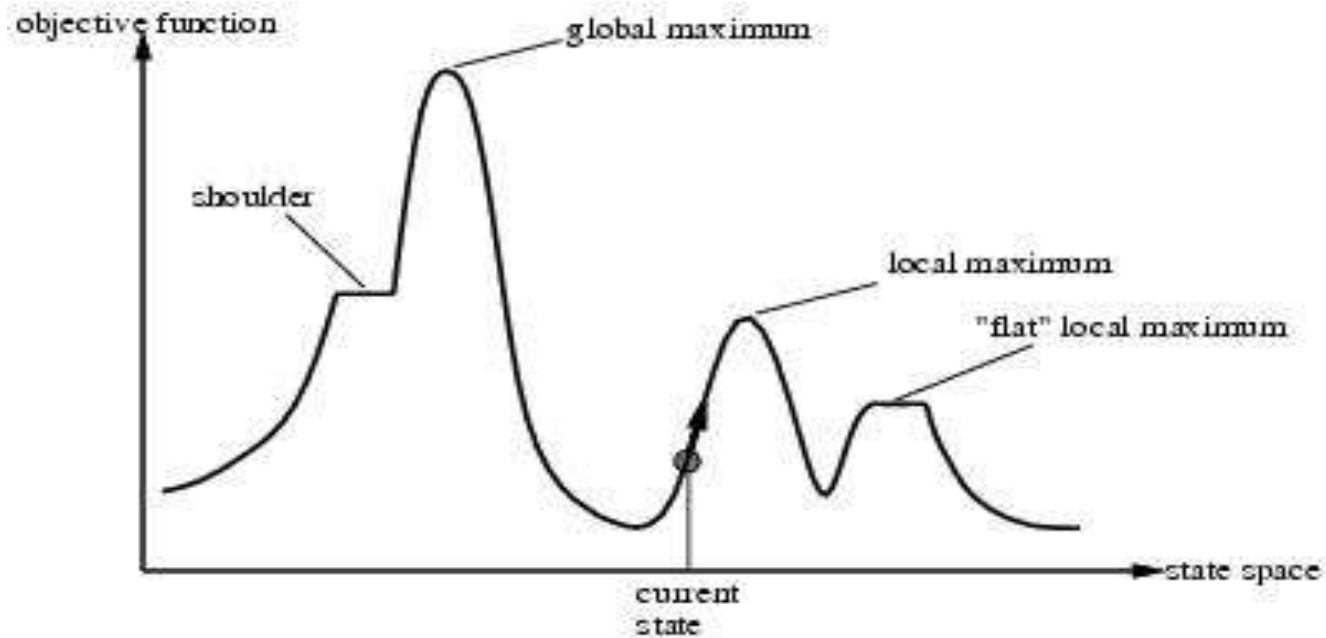
# Local search and optimization

---

- Local search
  - Keep track of single current state
  - Move only to neighboring states
  - Ignore paths
- Advantages:
  - Use very little memory
  - Can often find reasonable solutions in large or infinite (continuous) state spaces.
- “Pure optimization” problems
  - All states have an objective function
  - Goal is to find state with max (or min) objective value
  - Does not quite fit into path-cost/goal-state formulation
  - Local search can do quite well on these problems.

# "Landscape" of search

---



# Hill-climbing search

---

**function** HILL-CLIMBING( *problem*) **return** a state that is a local maximum

**input:** *problem*, a problem

**local variables:** *current*, a node.  
                    *neighbor*, a node.

*current*  $\leftarrow$  MAKE-NODE(INITIAL-STATE[*problem*])

**loop do**

*neighbor*  $\leftarrow$  a highest valued successor of *current*

**if** VALUE [*neighbor*]  $\leq$  VALUE[*current*] **then return** STATE[*current*]

*current*  $\leftarrow$  *neighbor*

# Hill-climbing search

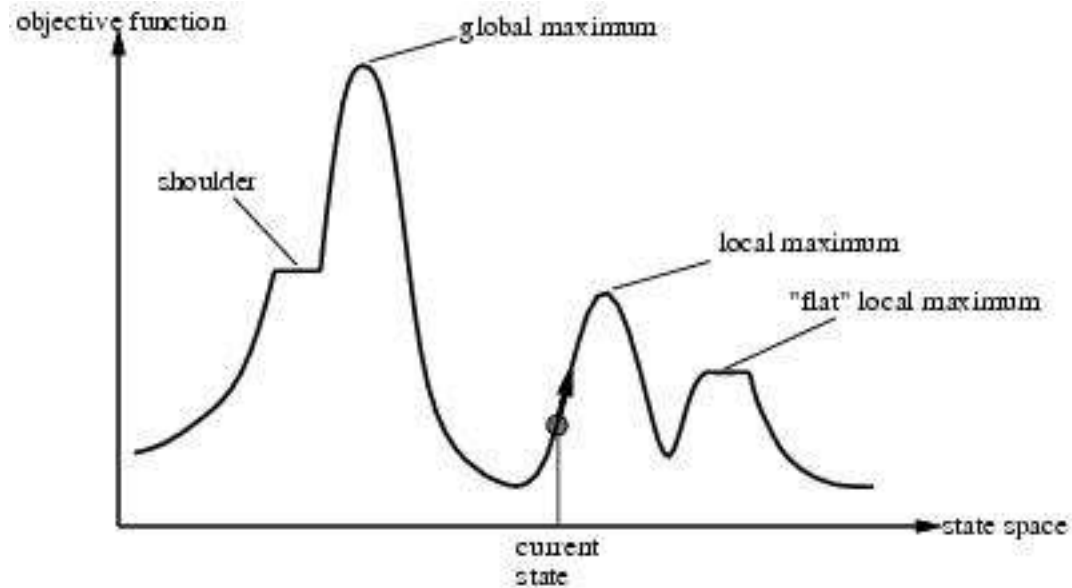
---

- “a loop that continuously moves in the direction of increasing value”
  - terminates when a peak is reached
  - Aka greedy local search
- Value can be either
  - Objective function value
  - Heuristic function value (minimized)
- Hill climbing does not look ahead of the immediate neighbors of the current state.
- Can randomly choose among the set of best successors, if multiple have the best value
- Characterized as “trying to find the top of Mount Everest while in a thick fog”

# Hill climbing and local maxima

---

- When local maxima exist, hill climbing is suboptimal
- Simple (often effective) solution
  - Multiple random restarts





# Hill-climbing example

---

- 8-queens problem, complete-state formulation
  - All 8 queens on the board in some configuration
- Successor function:
  - move a single queen to another square in the same column.
- Example of a heuristic function  $h(n)$ :
  - the number of pairs of queens that are attacking each other (directly or indirectly)
  - (so we want to minimize this)

# Hill-climbing example

---

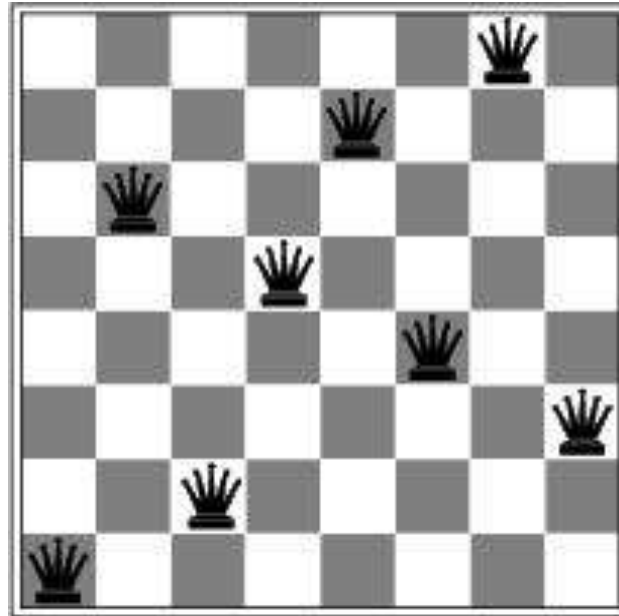
|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 18 | 12 | 14 | 13 | 13 | 12 | 14 | 14 |
| 14 | 16 | 13 | 15 | 12 | 14 | 12 | 16 |
| 14 | 12 | 18 | 13 | 15 | 12 | 14 | 14 |
| 15 | 14 | 14 | ♔  | 13 | 16 | 13 | 16 |
| ♔  | 14 | 17 | 15 | ♔  | 14 | 16 | 16 |
| 17 | ♔  | 16 | 18 | 15 | ♔  | 15 | ♔  |
| 18 | 14 | ♔  | 15 | 15 | 14 | ♔  | 16 |
| 14 | 14 | 13 | 17 | 12 | 14 | 12 | 18 |

Current state:  $h=17$

Shown is the  $h$ -value for each possible successor in each column

# A local minimum for 8-queens

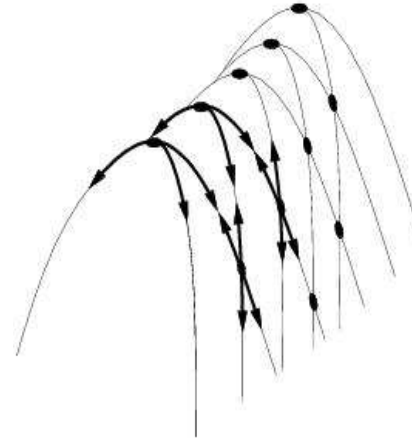
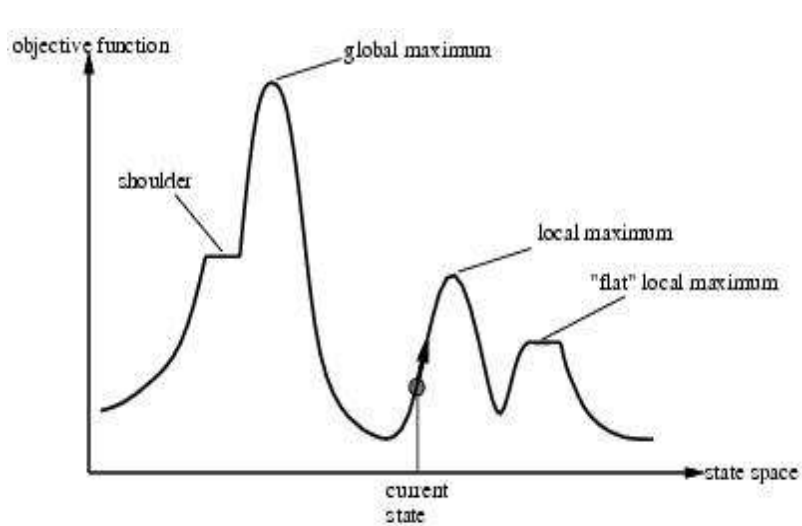
---



A local minimum in the 8-queens state space ( $h=1$ )

# Other drawbacks

---



- Ridge = sequence of local maxima difficult for greedy algorithms to navigate
- Plateau = an area of the state space where the evaluation function is flat.

# Performance of hill-climbing on 8-queens

---

- Randomly generated 8-queens starting states...
- 14% the time it solves the problem
- 86% of the time it get stuck at a local minimum
- However...
  - Takes only 4 steps on average when it succeeds
  - And 3 on average when it gets stuck
  - (for a state space with  $\sim 17$  million states)

# Possible solution...sideways moves

---

- If no downhill (uphill) moves, allow sideways moves in hope that algorithm can escape
  - Need to place a limit on the possible number of sideways moves to avoid infinite loops
- For 8-queens
  - Now allow sideways moves with a limit of 100
  - Raises percentage of problem instances solved from 14 to 94%
  - However....
    - 21 steps for every successful solution
    - 64 for each failure

# Hill-climbing variations

---

- Stochastic hill-climbing
  - Random selection among the uphill moves.
  - The selection probability can vary with the steepness of the uphill move.
- First-choice hill-climbing
  - stochastic hill climbing by generating successors randomly until a better one is found
  - Useful when there are a very large number of successors
- Random-restart hill-climbing
  - Tries to avoid getting stuck in local maxima.

# Hill-climbing with random restarts

---

- Different variations
  - For each restart: run until termination v. run for a fixed time
  - Run a fixed number of restarts or run indefinitely
- Analysis
  - Say each search has probability  $p$  of success
    - E.g., for 8-queens,  $p = 0.14$  with no sideways moves
  - Expected number of restarts?
  - Expected number of steps taken?



## Expected number of restarts

---

- Probability of Success =  $p$
- Number of restarts =  $1 / p$
- This means 1 successful iteration after  $(1/p - 1)$  failed iterations
- Let avg. number of steps in a failure iteration =  $f$   
and avg. number of steps in a successful iteration =  $s$

Therefore, expected number of steps in random-restart hill climbing =  $1 * s + (1/p - 1) f$

So for 8-queens,  $p = 14\%$ ,  $s = 4$ ,  $f = 3$ ,

$$\text{Expected no of moves} = 1 * 4 + (1/0.14 - 1) * 3 = 22$$

With sideways moves,  $p = 94\%$ ,  $s = 21$ ,  $f = 64$

$$\text{Expected no of moves} = 1 * 21 + (1/0.94 - 1) * 64 = 25$$

# Local beam search

---

- Keep track of  $k$  states instead of one
  - Initially:  $k$  randomly selected states
  - Next: determine all successors of  $k$  states
  - If any of successors is goal  $\rightarrow$  finished
  - Else select  $k$  best from successors and repeat.
- Major difference with random-restart search
  - Information is shared among  $k$  search threads.
- Can suffer from lack of diversity.
  - Stochastic beam search
    - choose  $k$  successors proportional to state quality.

# Gradient Descent

---

Assume we have some cost-function:  $\mathcal{C}(x_1, \dots, x_n)$   
and we want minimize over continuous variables  $x_1, x_2, \dots, x_n$

1. Compute the *gradient* :  $\frac{\partial}{\partial x_i} \mathcal{C}(x_1, \dots, x_n) \quad \forall i$

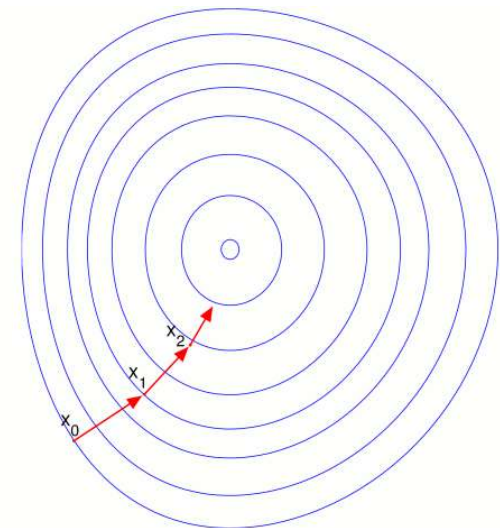
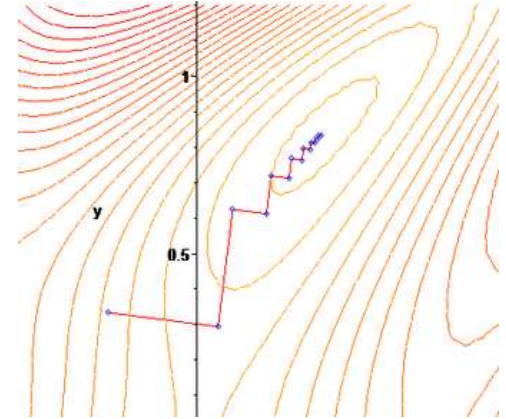
2. Take a small step downhill in the direction of the gradient:

$$x_i \rightarrow x'_i = x_i - \lambda \frac{\partial}{\partial x_i} \mathcal{C}(x_1, \dots, x_n) \quad \forall i$$

3. Check if  $\mathcal{C}(x_1, \dots, x'_i, \dots, x_n) < \mathcal{C}(x_1, \dots, x_i, \dots, x_n)$

4. If true then accept move, if not reject.

5. Repeat.



# Learning as optimization

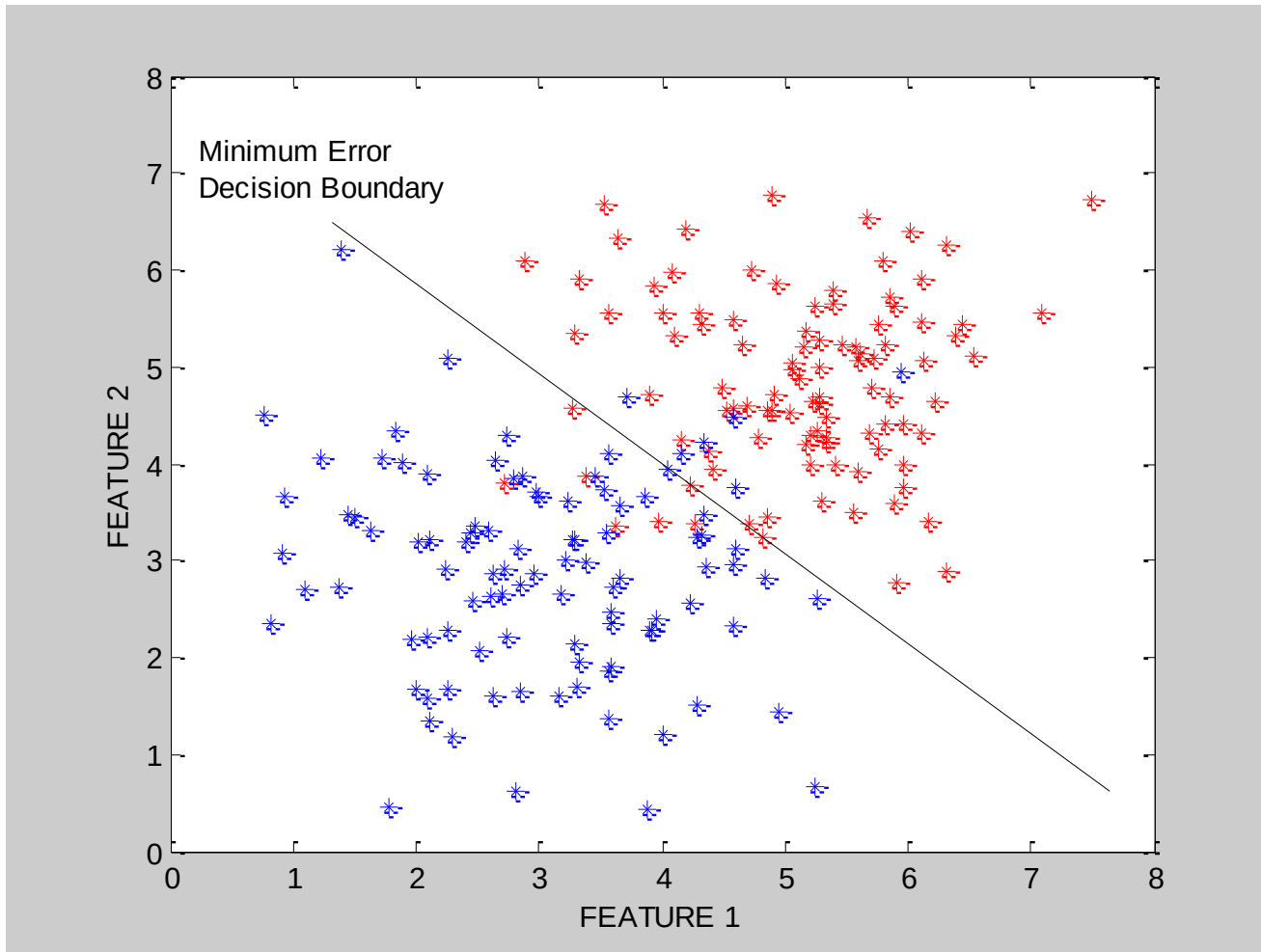
---

- Many machine learning problems can be cast as optimization
- Example:
  - Training data  $D = \{(\underline{x}_1, c_1), \dots, (\underline{x}_n, c_n)\}$   
where  $\underline{x}_i$  = feature or attribute vector  
and  $c_i$  = class label (say binary-valued)
  - We have a model (a function or classifier) that maps from  $x$  to  $c$   
e.g.,  $\text{sign}(\underline{w} \cdot \underline{x}') = \{-1, +1\}$
  - We can measure the error  $E(\underline{w})$  for any setting of the weights  $\underline{w}$ ,  
and given a training data set  $D$
  - Optimization problem: find the weight vector that minimizes  $E(\underline{w})$

(general idea is “empirical error minimization”)

# Learning a minimum error decision boundary

---



# Search using Simulated Annealing

---

- Simulated Annealing = hill-climbing with non-deterministic search
- Basic ideas:
  - like hill-climbing identify the quality of the local improvements
  - instead of picking the best move, pick one randomly
  - say the change in objective function is  $\delta$
  - if  $\delta$  is positive, then move to that state
  - otherwise:
    - move to this state with probability proportional to  $\delta$
    - thus: worse moves (very large negative  $\delta$ ) are executed less often
  - however, there is always a chance of escaping from local maxima
  - over time, make it less likely to accept locally bad moves
  - (Can also make the size of the move random as well, i.e., allow “large” steps in state space)

# Physical Interpretation of Simulated Annealing

---

- A Physical Analogy:
  - imagine letting a ball roll downhill on the function surface
    - this is like hill-climbing (for minimization)
  - now imagine shaking the surface, while the ball rolls, gradually reducing the amount of shaking
    - this is like simulated annealing
- Annealing = physical process of cooling a liquid or metal until particles achieve a certain frozen crystal state
  - simulated annealing:
    - free variables are like particles
    - seek “low energy” (high quality) configuration
    - get this by slowly reducing temperature  $T$ , which particles move around randomly

# Simulated annealing

---

**function** SIMULATED-ANNEALING( *problem*, *schedule*) **return** a solution state

**input:** *problem*, a problem

*schedule*, a mapping from time to temperature

**local variables:** *current*, a node.

*next*, a node.

*T*, a “temperature” controlling the probability of downward steps

*current*  $\leftarrow$  MAKE-NODE(INITIAL-STATE[*problem*])

**for** *t*  $\leftarrow$  1 **to**  $\infty$  **do**

*T*  $\leftarrow$  *schedule*[*t*]

**if** *T* = 0 **then return** *current*

*next*  $\leftarrow$  a randomly selected successor of *current*

$\Delta E \leftarrow$  VALUE[*next*] - VALUE[*current*]

**if**  $\Delta E > 0$  **then** *current*  $\leftarrow$  *next*

**else** *current*  $\leftarrow$  *next* only with probability  $e^{\Delta E / T}$



# More Details on Simulated Annealing

---

- Lets say there are 3 moves available, with changes in the objective function of  $d1 = -0.1$ ,  $d2 = 0.5$ ,  $d3 = -5$ . (Let  $T = 1$ ).
- pick a move randomly:
  - if  $d2$  is picked, move there.
  - if  $d1$  or  $d3$  are picked, probability of move =  $\exp(d/T)$
  - move 1:  $\text{prob1} = \exp(-0.1) = 0.9$ ,
    - i.e., 90% of the time we will accept this move
  - move 3:  $\text{prob3} = \exp(-5) = 0.05$ 
    - i.e., 5% of the time we will accept this move
- $T$  = "temperature" parameter
  - high  $T \Rightarrow$  probability of "locally bad" move is higher
  - low  $T \Rightarrow$  probability of "locally bad" move is lower
  - typically,  $T$  is decreased as the algorithm runs longer
    - i.e., there is a "temperature schedule"

# Simulated Annealing in Practice

---

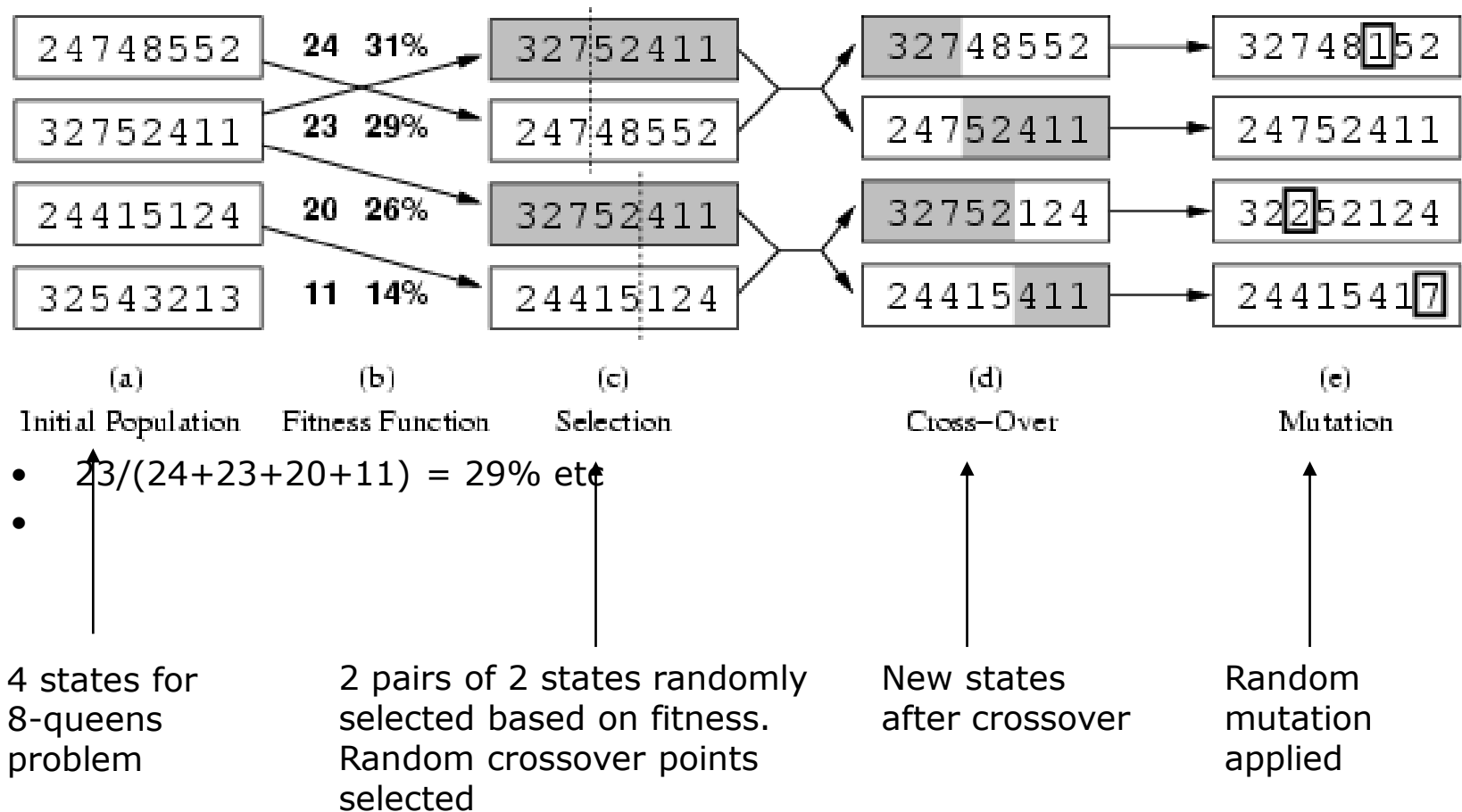
- method proposed in 1983 by IBM researchers for solving VLSI layout problems (Kirkpatrick et al, *Science*, 220:671-680, 1983).
  - theoretically will always find the global optimum (the best solution)
- useful for some problems, but can be very slow
  - slowness comes about because  $T$  must be decreased very gradually to retain optimality
  - In practice how do we decide the rate at which to decrease  $T$ ? (this is a practical problem with this method)

# Genetic algorithms

---

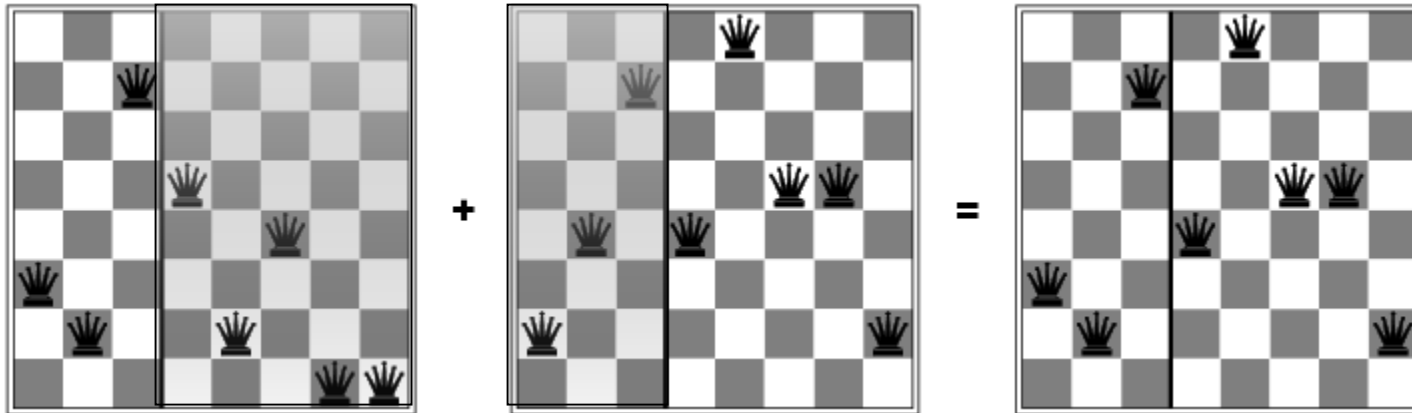
- Different approach to other search algorithms
  - A successor state is generated by combining two parent states
  -
- A state is represented as a string over a finite alphabet (e.g. binary)
  - 8-queens
    - State = position of 8 queens each in a column  
=>  $8 \times \log(8)$  bits = 24 bits (for binary representation)
- Start with  $k$  randomly generated states (**population**)
- 
- Evaluation function (**fitness function**).
  - Higher values for better states.
  - 
  - Opposite to heuristic function, e.g., # non-attacking pairs in 8-queens
- Produce the next generation of states by “simulated evolution”
  - Random selection
  - Crossover
  - Random mutation
  -

# Genetic algorithms



# Genetic algorithms

---



Has the effect of “jumping” to a completely different new part of the search space (quite non-local)

# Genetic algorithm pseudocode

---

```
function GENETIC_ALGORITHM( population, FITNESS-FN) return an individual
  input: population, a set of individuals
           FITNESS-FN, a function which determines the quality of the individual
  repeat
    new_population  $\leftarrow$  empty set
    loop for i from 1 to SIZE(population) do
      x  $\leftarrow$  RANDOM_SELECTION(population, FITNESS_FN)
      y  $\leftarrow$  RANDOM_SELECTION(population, FITNESS_FN)
      child  $\leftarrow$  REPRODUCE(x,y)
      if (small random probability) then child  $\leftarrow$  MUTATE(child )
      add child to new_population
    population  $\leftarrow$  new_population
  until some individual is fit enough or enough time has elapsed
  return the best individual
```

# Comments on genetic algorithms

---

- Positive points
  - Random exploration can find solutions that local search can't
    - (via crossover primarily)
  - Appealing connection to human evolution
    - E.g., see related area of genetic programming
- Negative points
  - Large number of “tunable” parameters
    - Difficult to replicate performance from one problem to another
  - Lack of good empirical studies comparing to simpler methods
  - Useful on some (small?) set of problems but no convincing evidence that GAs are better than hill-climbing w/random restarts in general

# Summary

---

- Local search techniques and optimization
  - Hill-climbing
  - Gradient methods
  - Simulated annealing
  - Genetic algorithms
  - Issues with local search



#### **Assignment 4: Decision tree learning for car evaluation**

In this assignment, you will implement a decision-tree algorithm and apply it to car evaluation (please see the supplied dataset).

1. Implement the decision tree learning algorithm. You may program in C/C++ or Java. The attribute selection criteria should be a parameter. The value will be one of the following: information gain and Gini impurity. Please see the link for details: [Decision tree learning - Wikipedia](#)
2. Divide the data set randomly between training (80%) and testing (20%) sets. Use your algorithm to train a decision tree classifier using the training data and calculate it's performance on the test data. Run this experiment 20 times randomly choosing the training and testing set. Then calculate the average accuracy of your algorithm. Compare the accuracy between information gain and Gini impurity measures.
3. Implement steps 1 and 2 with the following modifications: instead of selecting the best attribute at each step, select one randomly from the top three attributes. Compare the accuracy between information gain and Gini impurity measures.
4. Print a table as shown below.

|                                                   | Average accuracy over 20 runs |               |
|---------------------------------------------------|-------------------------------|---------------|
| Attribute selection strategy                      | Information gain              | Gini impurity |
| Always select the best attribute                  |                               |               |
| Select one randomly from the top three attributes |                               |               |

5. Submit your source code and the performance result (above table). Only C++/Java programs are accepted for this assignment.

Chapter 18 (AIAMA)

# Learning From Examples-02

Sukarna Barua

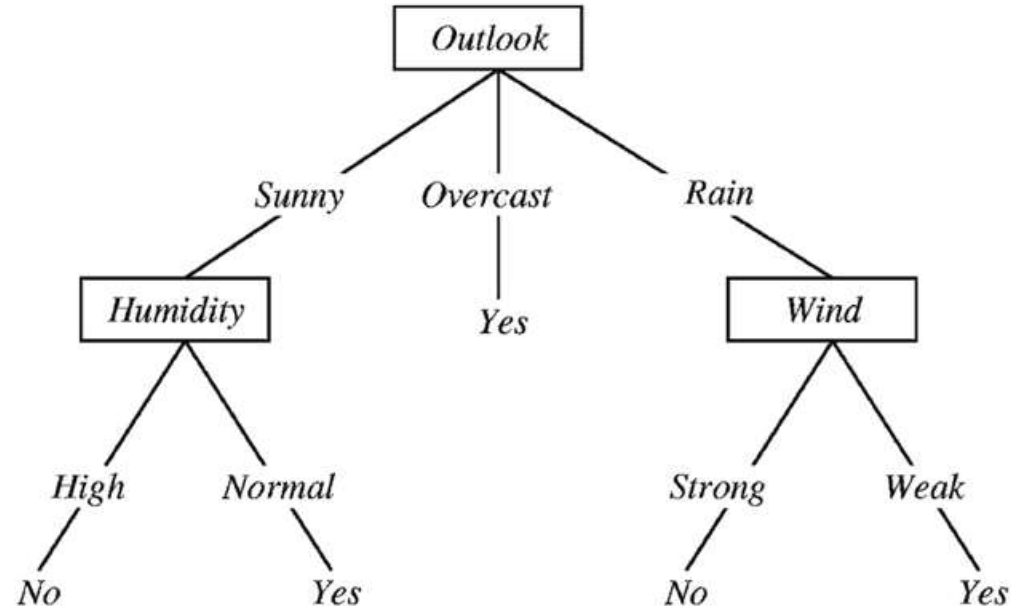
Associate Professor, CSE, BUET

# Decision Tree

- **A decision tree:**
  - Represents a function that takes as input a vector of attribute values
  - Returns a “decision”—a single output value.
  - The input and output values can be discrete or continuous.
  - For now we will concentrate on problems where
    - Inputs have discrete values
    - Output has exactly two possible values; this is Boolean classification, where each example input will be classified as *true* (a **positive** example) or *false* (a **negative** example).

# Decision Tree

- A decision tree for rain forecasting.
  - Input: {Outlook=Sunny, Humidity=Normal, Wind=Strong}
  - Output: Yes [Rainy]



# Decision Tree

- **In a decision tree:**
  - Each **internal node** in the tree corresponds to a test of the value of one of the input attributes,  $A_i$
  - The **branches** from the node are labeled with the possible values of the attribute,  $A_i = v_{ik}$ .
  - Each **leaf node** in the tree is marked with a outcome to be returned by the function.

# Decision Tree: Example

- **Problem:** Build a decision tree to decide whether to wait at a restaurant.
- **Learning Goal:**
  - WillWait - to decide whether to wait for a table at a restaurant.
  - Goal is binary valued (i.e., binary classification task)
    - Values: {Yes, No}

# Decision Tree: Example

- **Input Attributes**

1. *Alternate*: whether there is a suitable alternative restaurant nearby.
2. *Bar* : whether the restaurant has a comfortable bar area to wait in.
3. *Fri/Sat*: *true* on Fridays and Saturdays, *false* otherwise.
4. *Hungry*: whether we are hungry.
5. *Patrons*: how many people are in the restaurant (values are None, Some, and Full ).
6. *Price*: the restaurant's price range (\$, \$\$, \$\$\$).

# Decision Tree: Example

- **Input Attributes**

- 7. *Raining*: whether it is raining outside.

- 8. *Reservation*: whether we made a reservation.

- 9. *Type*: the kind of restaurant (French, Italian, Thai, or burger).

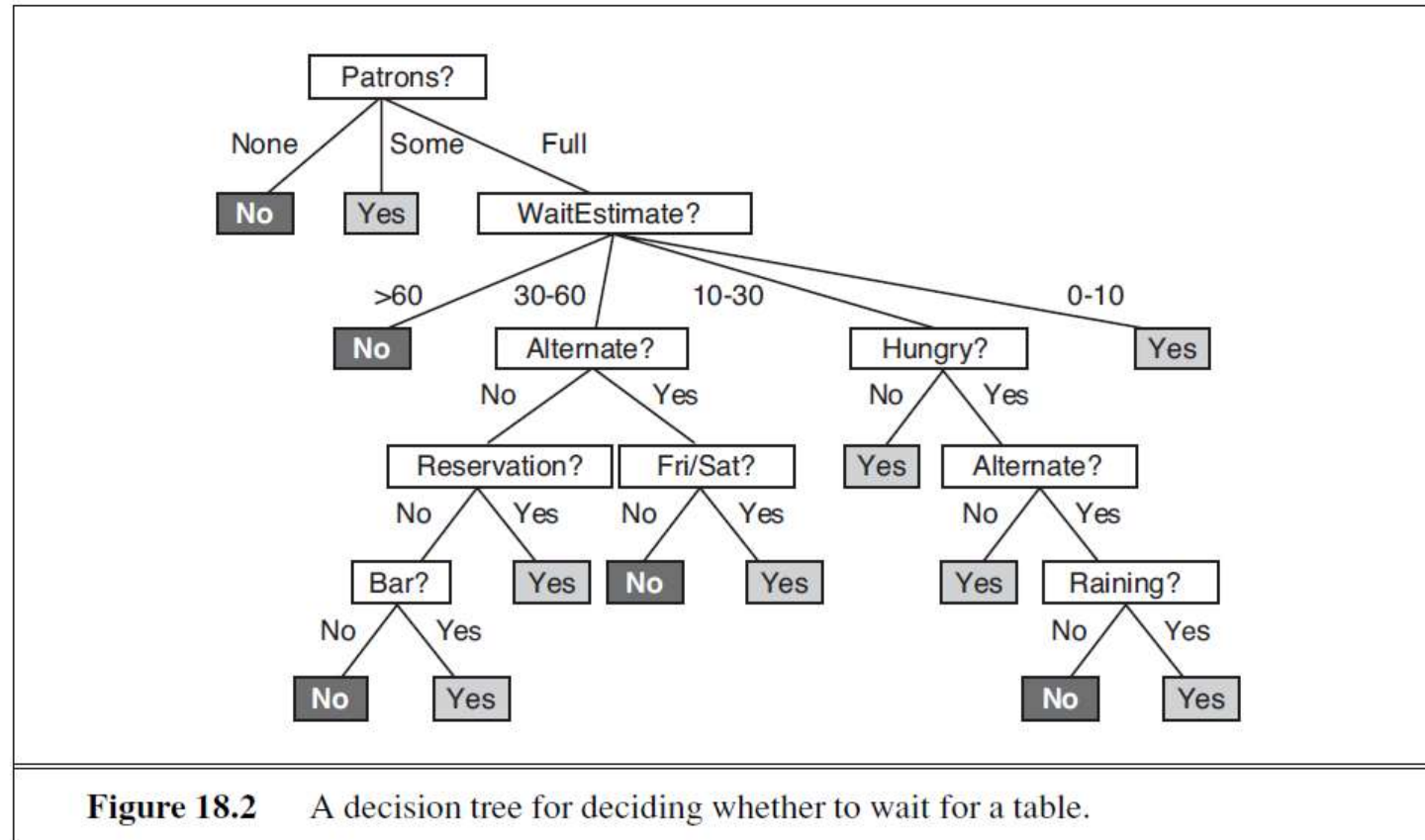
- 10. *WaitEstimate*: the wait estimated by the host (0–10 minutes, 10–30, 30–60, or >60).

- Note that every variable has a small set of possible values; the value of *WaitEstimate*, for example, is not an integer, rather it is one of the four discrete values 0–10, 10–30, 30–60, or >60.



# Decision Tree: Example

- An example decision tree for the restaurant problem



# Expressiveness of Decision Trees

- - In a Boolean decision tree, the goal attribute is true if and only if the input attributes satisfy one of the paths leading to a leaf with value true.
  - $Goal \Leftrightarrow (Path1 \vee Path2 \vee \dots)$ , where each  $Path$  is a conjunction of attribute-value tests required to follow that path.
  - Thus, the whole expression is equivalent to *disjunctive normal form*, which means that any function in propositional logic can be expressed as a decision tree.

# Expressiveness of Decision Trees

- - Any Boolean function can be represented by a decision tree.
    - Consider the following Boolean function:  $f(A, B) = A + B$ .
    - Draw a decision tree for the function:

# Expressiveness of Decision Trees

- For a wide variety of problems, the decision tree format yields a nice, concise result.
- But some functions cannot be represented concisely:
  - For example, the majority function, which returns true if and only if more than half of the inputs are true, requires an exponentially large decision tree.
- Decision trees are good for some kinds of functions and bad for others.

# Expressiveness of Decision Trees

- **How many different decision trees can be obtained for a Boolean function with  $n$  variables?**
  - A truth table over  $n$  attributes has  $2^n$  rows, one for each combination of values of the attributes.
  - There are  $2^{2^n}$  different functions.
  - Each function can be represented by a decision tree. Hence, number of decision trees at least  $2^{2^n}$  [why more?]

# Expressiveness of Decision Trees

- - How many different decision trees can be obtained for a Boolean function with  $n$  variables?
    - For 10 Boolean attributes of our restaurant problem there are  $2^{1024}$  or about  $10^{308}$  different functions to choose from.
    - Number of possible decision trees  $\geq 10^{308}$

# Inducing Decision Trees

- Training data: A set of examples where each example is a pair  $(\mathbf{x}, y)$  where  $\mathbf{x}$  is a vector of values for the input attributes,  $y$  is a single Boolean output value.

| Example           | Input Attributes |            |            |            |             |               |             |            |                |               | Goal                  |
|-------------------|------------------|------------|------------|------------|-------------|---------------|-------------|------------|----------------|---------------|-----------------------|
|                   | <i>Alt</i>       | <i>Bar</i> | <i>Fri</i> | <i>Hun</i> | <i>Pat</i>  | <i>Price</i>  | <i>Rain</i> | <i>Res</i> | <i>Type</i>    | <i>Est</i>    | <i>WillWait</i>       |
| $\mathbf{x}_1$    | <i>Yes</i>       | <i>No</i>  | <i>No</i>  | <i>Yes</i> | <i>Some</i> | <i>\$\$\$</i> | <i>No</i>   | <i>Yes</i> | <i>French</i>  | <i>0-10</i>   | $y_1 = \text{Yes}$    |
| $\mathbf{x}_2$    | <i>Yes</i>       | <i>No</i>  | <i>No</i>  | <i>Yes</i> | <i>Full</i> | <i>\$</i>     | <i>No</i>   | <i>No</i>  | <i>Thai</i>    | <i>30-60</i>  | $y_2 = \text{No}$     |
| $\mathbf{x}_3$    | <i>No</i>        | <i>Yes</i> | <i>No</i>  | <i>No</i>  | <i>Some</i> | <i>\$</i>     | <i>No</i>   | <i>No</i>  | <i>Burger</i>  | <i>0-10</i>   | $y_3 = \text{Yes}$    |
| $\mathbf{x}_4$    | <i>Yes</i>       | <i>No</i>  | <i>Yes</i> | <i>Yes</i> | <i>Full</i> | <i>\$</i>     | <i>Yes</i>  | <i>No</i>  | <i>Thai</i>    | <i>10-30</i>  | $y_4 = \text{Yes}$    |
| $\mathbf{x}_5$    | <i>Yes</i>       | <i>No</i>  | <i>Yes</i> | <i>No</i>  | <i>Full</i> | <i>\$\$\$</i> | <i>No</i>   | <i>Yes</i> | <i>French</i>  | <i>&gt;60</i> | $y_5 = \text{No}$     |
| $\mathbf{x}_6$    | <i>No</i>        | <i>Yes</i> | <i>No</i>  | <i>Yes</i> | <i>Some</i> | <i>\$\$</i>   | <i>Yes</i>  | <i>Yes</i> | <i>Italian</i> | <i>0-10</i>   | $y_6 = \text{Yes}$    |
| $\mathbf{x}_7$    | <i>No</i>        | <i>Yes</i> | <i>No</i>  | <i>No</i>  | <i>None</i> | <i>\$</i>     | <i>Yes</i>  | <i>No</i>  | <i>Burger</i>  | <i>0-10</i>   | $y_7 = \text{No}$     |
| $\mathbf{x}_8$    | <i>No</i>        | <i>No</i>  | <i>No</i>  | <i>Yes</i> | <i>Some</i> | <i>\$\$</i>   | <i>Yes</i>  | <i>Yes</i> | <i>Thai</i>    | <i>0-10</i>   | $y_8 = \text{Yes}$    |
| $\mathbf{x}_9$    | <i>No</i>        | <i>Yes</i> | <i>Yes</i> | <i>No</i>  | <i>Full</i> | <i>\$</i>     | <i>Yes</i>  | <i>No</i>  | <i>Burger</i>  | <i>&gt;60</i> | $y_9 = \text{No}$     |
| $\mathbf{x}_{10}$ | <i>Yes</i>       | <i>Yes</i> | <i>Yes</i> | <i>Yes</i> | <i>Full</i> | <i>\$\$\$</i> | <i>No</i>   | <i>Yes</i> | <i>Italian</i> | <i>10-30</i>  | $y_{10} = \text{No}$  |
| $\mathbf{x}_{11}$ | <i>No</i>        | <i>No</i>  | <i>No</i>  | <i>No</i>  | <i>None</i> | <i>\$</i>     | <i>No</i>   | <i>No</i>  | <i>Thai</i>    | <i>0-10</i>   | $y_{11} = \text{No}$  |
| $\mathbf{x}_{12}$ | <i>Yes</i>       | <i>Yes</i> | <i>Yes</i> | <i>Yes</i> | <i>Full</i> | <i>\$</i>     | <i>No</i>   | <i>No</i>  | <i>Burger</i>  | <i>30-60</i>  | $y_{12} = \text{Yes}$ |

**Figure 18.3** Examples for the restaurant domain.

# Inducing Decision Trees

- **Build a decision tree that is -**
  - Consistent with the examples [*Not always expected though, generalization may suffer*]
  - Is as small as possible. [*Occam's razor*]



# Inducing Decision Trees

- **Question:** Can we always find a consistent decision tree given a set of training examples?.
- **Answer:** Yes.
  - However, if two training examples  $(\mathbf{x}, y)$  and  $(\mathbf{x}', y')$  have different outputs but same input attributes ( $\mathbf{x} = \mathbf{x}'$ ), then a consistent decision tree is not possible.

# Inducing Decision Trees

- Unfortunately, it is an intractable problem to find the smallest consistent tree; there is no way to efficiently search through *hypotheses space*.
  - An NP-hard problem!
- What can we do if cannot find the smallest decision tree?
- **Solution:** Use heuristics to find a closest one.
  - With some simple heuristics, we can find a good approximate solution: a small (*but not the smallest*) consistent tree.
  - This is a greedy approach. [Remember what is *greedy*]

# Decision Tree Learning

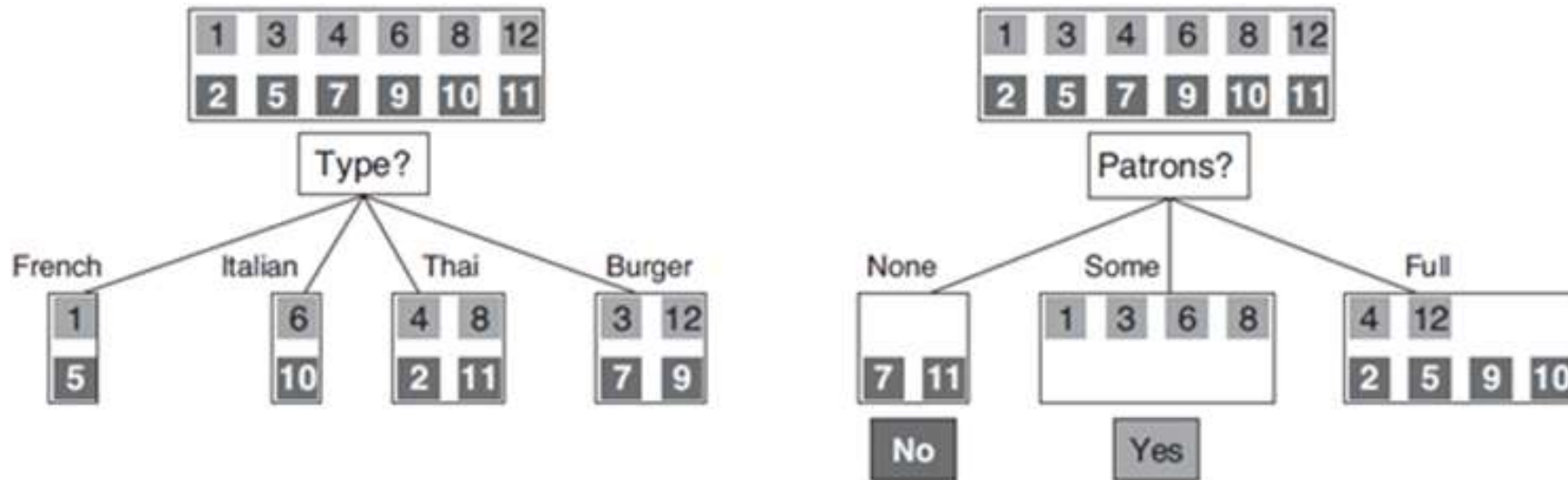
- **Greedy approach to build a decision tree:**
  - Start with empty decision tree.
  - Select an attribute to test at the next level [node]:
    - *Always select the most important attribute to test first. [greedy strategy]*
  - The test creates new branches and divides the problem into smaller subproblems.
  - Recurse on each child (created for each branch)

# Decision Tree Learning

- **Greedy strategy:** *Always select the most important attribute to test first.*
  - *Most important attribute* implies the one that makes the most difference to the classification of an example. [*Get leaves as early as possible*]
    - Get correct classification with a small number of test
    - All paths in the tree will be short
    - Tree as a whole will be shallow.
  - *Above greedy strategy is a local optimal choice, may not necessarily leads to the globally smallest tree!*

# Decision Tree Learning

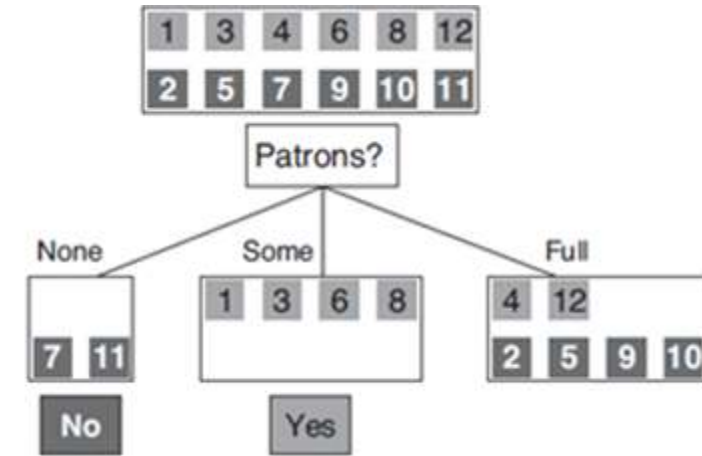
- Which attribute to test at root? Type vs Patron?
  - Type: all subsets (i.e., branches) needs further exploration.
  - Patron: two branches become leaves, only one needs further exploration.



# Decision Tree Learning Algorithm

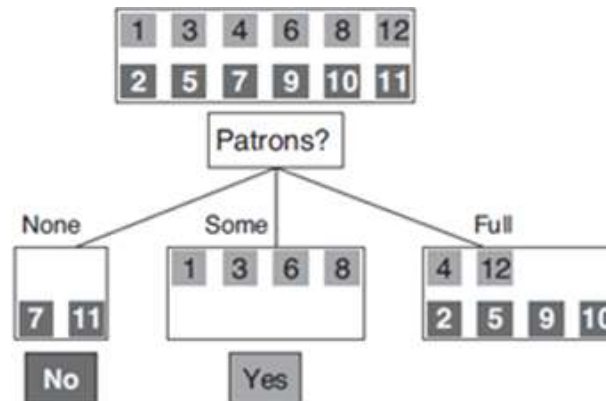
- **Decision tree construction:**

- Step 1: Test an attribute at each node.
- Step 2: Partition the examples according to values and create child nodes with relevant examples.
- Step 3: Now consider child node for further tests of attributes except the one which have already been tested in the hierarchy (*recursive operation*)



# Decision Tree Learning Algorithm

- **Four cases to consider at a child node:**
  - Case 1: All examples are positive (or all negative) → we are done; we can answer Yes or No. [e.g., *Patron=None*]
  - Case 2: Some positive and some negative examples → then choose the best attribute to split them recursively. [e.g., *Patron=Full*]



# Decision Tree Learning Algorithm

- **Four cases to consider at a child node:**
  - Case 3: No examples left → No example has been observed for this combination of attribute values, and we return a default value (e.g., *plurality of parent node*)
  - Case 4: No attributes left, but both positive and negative examples remain → These examples have exactly the same description, but different classifications. This can happen because there is an error or **noise** in the data.
    - *In this case return the plurality classification of the remaining examples.*  
[tree will not be consistent]



# Decision Tree Learning Algorithm

- **Algorithm pseudocode**

```
function DECISION-TREE-LEARNING(examples, attributes, parent_examples) returns  
a tree  
  
  if examples is empty then return PLURALITY-VALUE(parent_examples)  
  else if all examples have the same classification then return the classification  
  else if attributes is empty then return PLURALITY-VALUE(examples)  
  else  
     $A \leftarrow \operatorname{argmax}_{a \in \text{attributes}} \text{IMPORTANCE}(a, \text{examples})$   
    tree  $\leftarrow$  a new decision tree with root test A  
    for each value  $v_k$  of A do  
       $\text{exs} \leftarrow \{e : e \in \text{examples} \text{ and } e.A = v_k\}$   
      subtree  $\leftarrow$  DECISION-TREE-LEARNING(exs, attributes – A, examples)  
      add a branch to tree with label (A =  $v_k$ ) and subtree subtree  
  return tree
```

- Function PLURALITY-VALUE selects the most common class/output among the examples

# Choosing the Most Important Attribute

- **Which attribute is the most important now?**
  - **Perfect attribute:** One that splits into subsets where each subset contain either all positive or all negative examples. [*all branches become leaf nodes*]
  - **Useless attribute:** One that splits into subsets where each subset contain fairly equal mix of positive and negative examples. [*all branches need recursive exploration*]

# Choosing the Most Important Attribute

## ▪ Perfect vs. useless? How to measure?

- A formal measure of perfect vs useless: Entropy
- The fundamental quantity in information theory (*Shannon and Weaver, 1949*).
- Entropy is a measure of the uncertainty of a random variable.
- Acquisition of information corresponds to a reduction in entropy.
- A random variable with only one value—a coin that always comes up heads—has no uncertainty and thus its entropy is defined as zero; thus, we gain no information by observing its value.

# Choosing the Most Important Attribute

- **Entropy:** Average number of bits per symbol to encode information.
  - The roll of a fair *four*-sided die has 2 bits of entropy, because it takes two bits to describe one of four equally probable choices.
  - An unfair coin that comes up heads 99% of the time.
    - This coin has less uncertainty than the fair coin—if we guess heads we'll be wrong only 1% of the time—it's entropy measure should be close to zero, but positive.

# Entropy Measure

- - In general, the entropy  $H(V)$  of a random variable  $V$  with values  $v_k$ , each with probability  $P(v_k)$ , is defined as :

$$H(V) = \sum_k P(v_k) \log_2 \frac{1}{P(v_k)} = - \sum_k P(v_k) \log_2 P(v_k)$$

# Entropy Measure

- **Verify that entropies measures are correct.**

We can check that the entropy of a fair coin flip is indeed 1 bit:

$$H(\textit{Fair}) = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5) = 1 .$$

If the coin is loaded to give 99% heads, we get

$$H(\textit{Loaded}) = -(0.99 \log_2 0.99 + 0.01 \log_2 0.01) \approx 0.08 \text{ bits.}$$

# Entropy Measure

- Define  $B(q)$  as the entropy of a Boolean random variable that is true with probability  $q$ :

$$B(q) = -(q \log_2 q + (1 - q) \log_2 (1 - q))$$

- If a training set contains  $p$  positive examples and  $n$  negative examples, then the entropy of the goal attribute on the whole set is:

$$H(Goal) = B\left(\frac{p}{p + n}\right) .$$

# Entropy Measure in Decision Tree

- **Decision tree contest:** Entropy represents an impurity measure of the set.
  - A set with 5 positive and 5 negative examples: Most impure, entropy should be highest.
  - A set with 10 positive and 0 negative examples: Purest, entropy should be the lowest.



# Entropy Before Split

- A set with  $p$  positive and  $n$  negative examples.

- Entropy of the set is:  $B\left(\frac{p}{p+n}\right)$

# Entropy After Splitting

- An attribute  $A$  with  $d$  distinct values divides the training set  $E$  into subsets  $E_1, E_2, \dots, E_d$ .
- Each subset  $E_k$  has  $p_k$  positive examples and  $n_k$  negative examples, with entropy of  $B(p_k/(p_k + n_k))$  bits of information. [ $E_k$  contains examples with  $A = kth$  value]
- A randomly chosen example from the training set has the  $kth$  value for the attribute with probability  $(p_k + n_k)/(p + n)$ , so the expected entropy (*weighted average*) of the  $d$  subsets after splitting on attribute  $A$  is:

$$Remainder(A) = \sum_{k=1}^d \frac{p_k + n_k}{p + n} B\left(\frac{p_k}{p_k + n_k}\right)$$

# Information Gain

- **Information Gain:** The amount of information gain is the amount of entropy reduction after the split on attribute A.
- Assume  $E_{BS}$  = Entropy before split,  $E_{AS}$  = Entropy after split [*weighted average*]
  - Hence, information gain ( = *reduction of entropy*):

$$Gain = E_{BS} - E_{AS} \text{ [reduction of entropy]}$$

- Hence, this is simply:

$$Gain(A) = B\left(\frac{p}{p+n}\right) - Remainder(A)$$

# Information Gain

- **Information Gain:**  $Gain(A) = B(\frac{p}{p+n}) - Remainder(A)$
- **Compute the information gain for the attributes Patron and Type:**

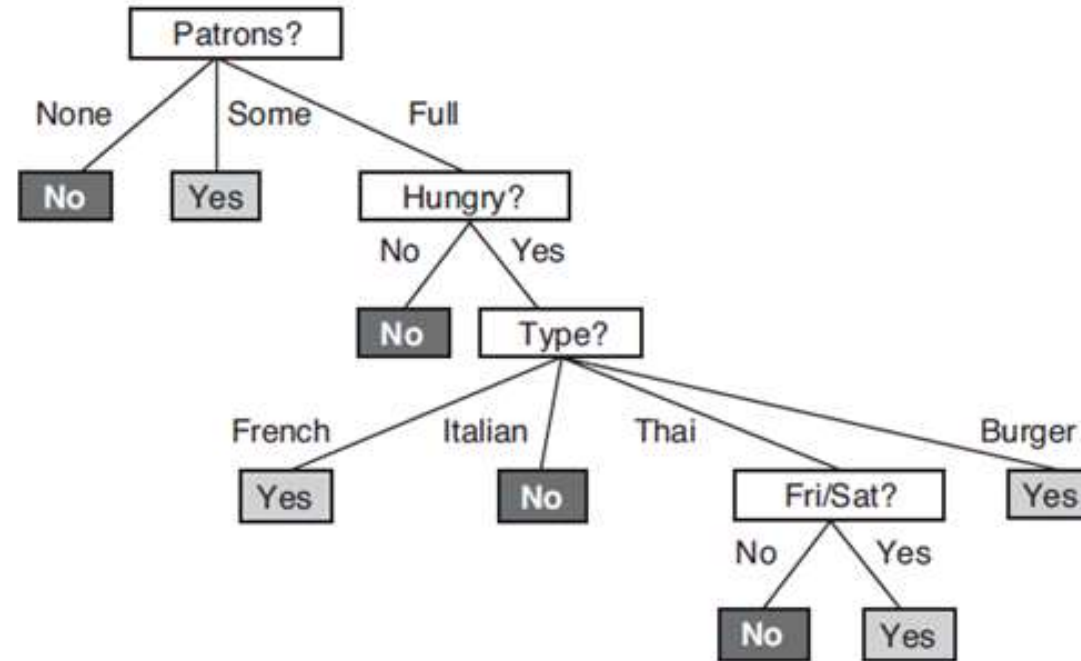
$$Gain(Patrons) = 1 - [\frac{2}{12}B(\frac{0}{2}) + \frac{4}{12}B(\frac{4}{4}) + \frac{6}{12}B(\frac{2}{6})] \approx 0.541 \text{ bits},$$

$$Gain(Type) = 1 - [\frac{2}{12}B(\frac{1}{2}) + \frac{2}{12}B(\frac{1}{2}) + \frac{4}{12}B(\frac{2}{4}) + \frac{4}{12}B(\frac{2}{4})] = 0 \text{ bits},$$

- **Patron is a better attribute than Gain! Hence, choose Patron over Type.**
- *Choose the attribute which gives the highest information gain!*

# Final Decision Tree

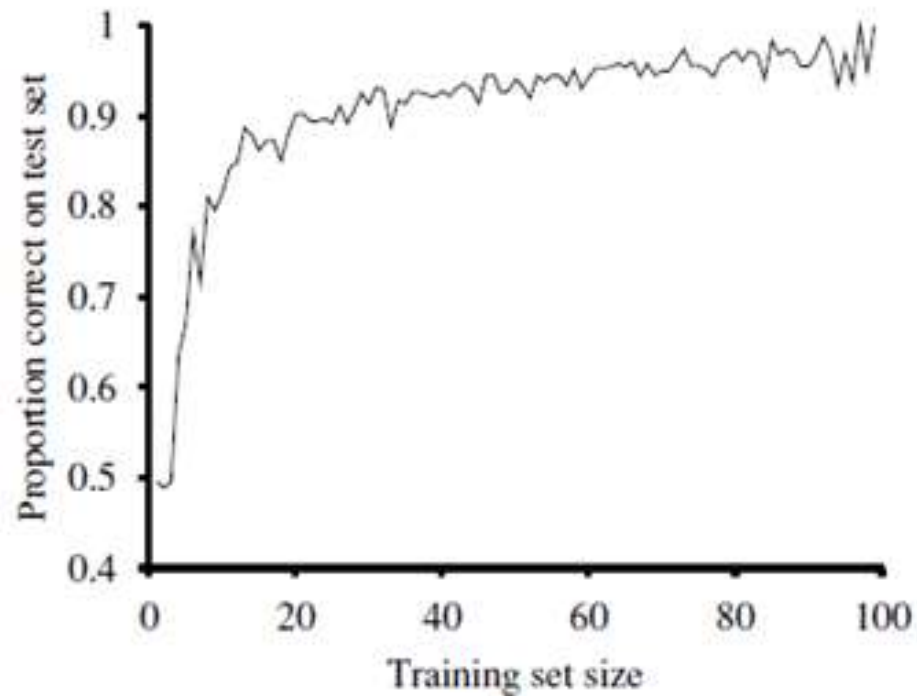
- Final decision tree constructed from given examples.



- Question: What is the information gain for Type attribute at level 3?

# Decision Tree Learning Curve

- Experiment with 100 examples: Construct decision trees
  - Split into train and test (e.g., 1 and 99, 2 and 98, etc.)
  - Random split 20 times and report average accuracy on test set
  - Note: As the training size grows, accuracy increases.



# Overfitting

- **Overfitting:** Too much importance on every training example
  - Complex tree
  - Suffers generalization: very low training error, but very high test error.
  - Misses important concepts
- A consistent decision tree over training data may result in a complex tree
  - Noisy data may also induce complexity in the tree

# Overfitting Solution

- **Pre-prune:** Prune the tree before it gets large
  - Early stopping: limit depth during tree construction
- **Post-pruning:** Prune the tree after construction
  - Remove irrelevant nodes
  - Replace internal nodes with most common class [*Only replace if test error do not increase*]



# Decision Tree Issues

- **Missing values:** Some examples have missing value in some attributes
  - Solution: Replace missing value with mean of the attribute over the entire dataset
- **Continuous-valued attribute:** For example, height is continuous-valued.
  - Convert to categorical attribute:
    - Height > 40cm: *Tall* and Height <40cm: *Short*

Solve n-puzzle using A\* search :

priority function,  $f(n) = g(n) + h(n)$ ,

$g(n) \rightarrow$  <sup>actual</sup> the cost from initial state to  $n$ th state

$h(n) \rightarrow$  estimated cost from node  $n$  to the goal state.

$\hookrightarrow$  this is called a heuristic

(i) Hamming distance: The numbers of blocks in the wrong position.

|   |   |   |
|---|---|---|
| 7 | 2 | 4 |
| 6 |   | 5 |
| 8 | 3 | 1 |

initial board

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 |   |

goal board

hamming distance: 7

| Digits            | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|-------------------|---|---|---|---|---|---|---|---|
| Is placed correct | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 |

(ii) Manhattan distance: The sum of the Manhattan distances (sum of vertical and horizontal distance) of the blocks to their goal positions.

| Digits       | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|--------------|---|---|---|---|---|---|---|---|
| Row dist.    | 2 | 0 | 2 | 1 | 0 | 0 | 2 | 0 |
| Column dist. | 2 | 0 | 1 | 2 | 1 | 2 | 0 | 1 |

$$\text{total} = 2+2+2+1+1+2+1+2+2+1 \\ = 16$$

(3) Euclidean heuristic: it calculates the straight-line distance between a tile's current position and its goal position.

current state:

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 0 | 7 | 8 |

goal state:

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | 0 |

for tile 7: current position: (2,1) ; Goal position: (2,0)

~~for tile~~ distance =  $\sqrt{(2-2)^2 + (1-0)^2} = 1$

for tile 8: current position: (2,2) ; Goal position: (2,1)

distance =  $\sqrt{(2-2)^2 + (2-1)^2} = 1$

$\therefore$  total euclidean heuristic is:  $1+1=2$

Inversion Count:

A pair of tiles form an inversion if the values of tiles are in reverse order of their appearance in goal state.

goal state: 1 2 3 5 4 7 8 6 -

current state: 2 3 4 7 6 8 1 5 -

inversion count for:  $2 \rightarrow 1, 3 \rightarrow 1, 4 \rightarrow 2, 7 \rightarrow 2, 6 \rightarrow 3, 8 \rightarrow 2$   
 $1 \rightarrow 0, 5 \rightarrow 0$  total: 13

## Solvability:

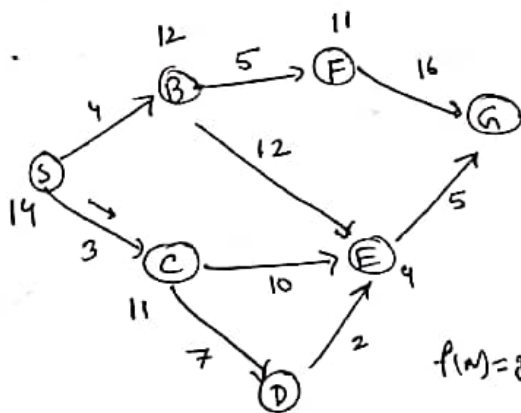
(1) if grid size  $k$  is odd, then no. of inversions should be even number.

(2) if grid size  $k$  is even, then

(a) the blank is on an even row counting from the bottom and number of inversion is odd.

(b) the blank is on an odd row counting from the bottom and number of inversion is even.

## Simulations



Start state: S, Goal state: G

$$S \rightarrow B: 4 + 12 = 16 \checkmark$$

$$S \rightarrow C: 3 + 11 = 14 \checkmark$$

$$S \rightarrow C \rightarrow D: 3 + 7 + 6 = 16 \checkmark$$

$$S \rightarrow C \rightarrow E: 3 + 10 + 4 = 17$$

$$S \rightarrow B \rightarrow E: 4 + 12 + 4 = 20$$

$$S \rightarrow B \rightarrow F: 4 + 5 + 11 = 20$$

$$S \rightarrow C \rightarrow D \rightarrow E: 3 + 7 + 2 + 4 = 16 \checkmark \checkmark$$

$S \rightarrow C \rightarrow D \rightarrow E \rightarrow G:$

$$3 + 7 + 2 + 5 = 17$$

Time complexity:  $O(b^d)$

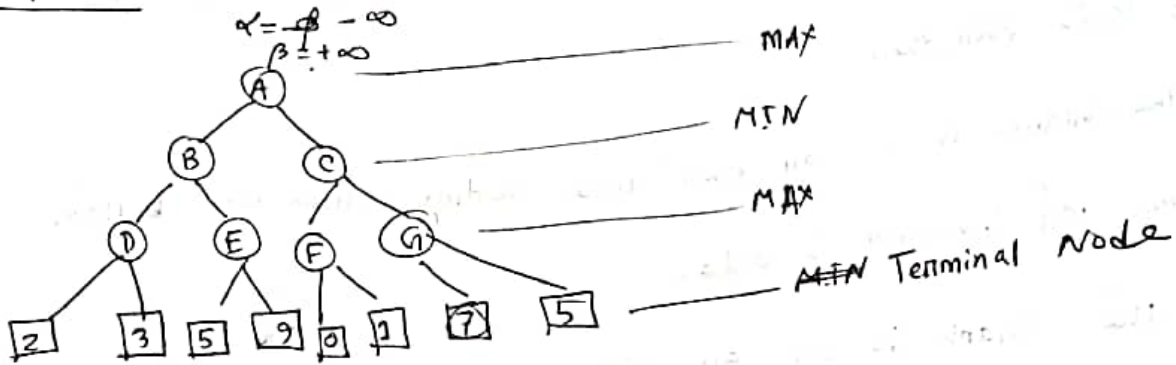
$b$  = branch factor

$d$  = depth

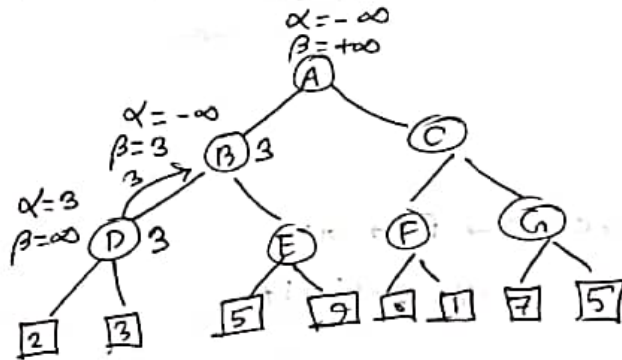
Space complexity:  $O(b^d)$

# Adversarial Search:

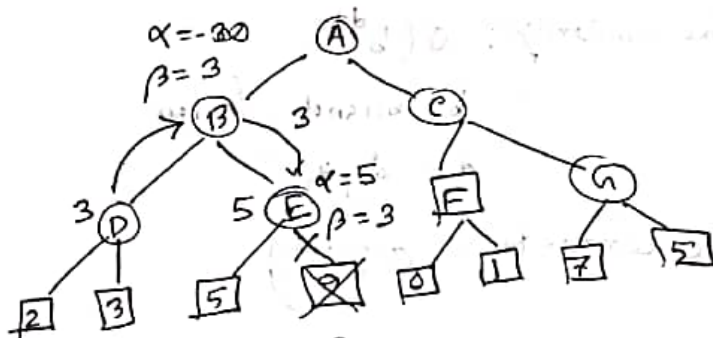
## Simulation:



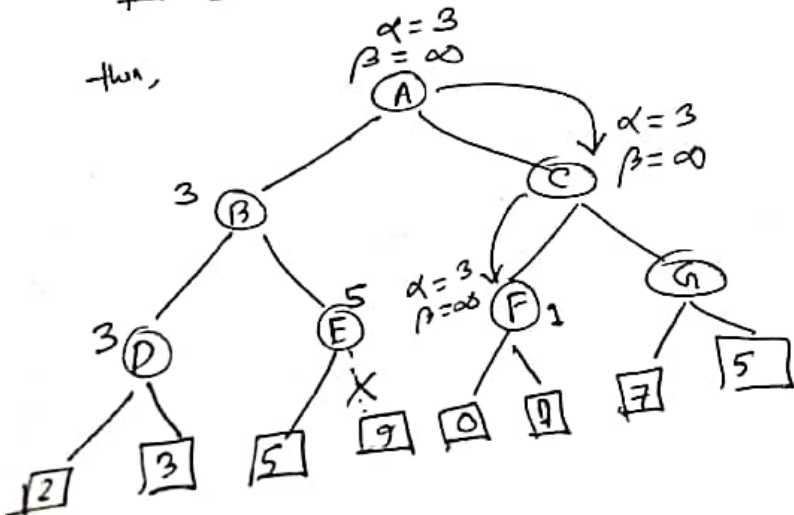
for node D:  $\max(2, 3) = 3$

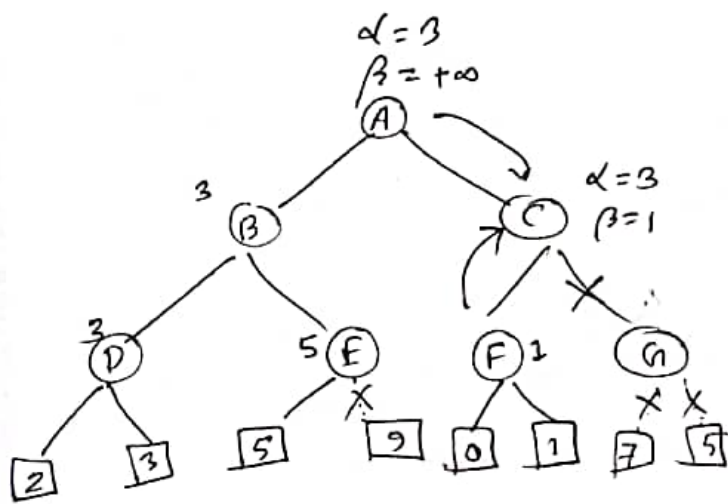


then,



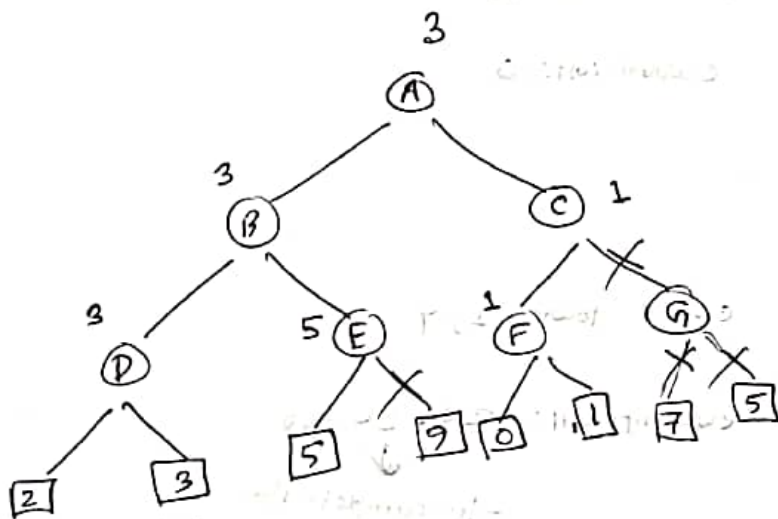
then,





as,  $\alpha \geq \beta$

Now C returns the value of 1 to A here the best value for A is  $\max\{3, 1\} = 3$

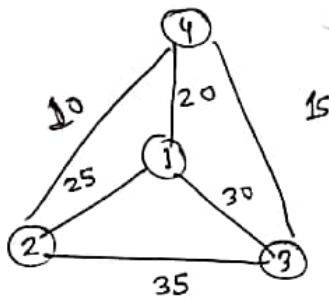


Worst ordering: Doesn't prune any of the leaves of the tree.  
time complexity:  $O(b^m)$

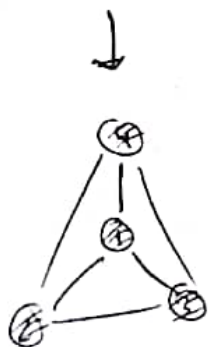
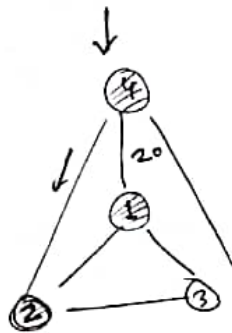
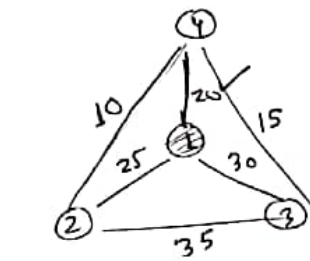
Ideal ordering: best moves occur at the left side of the tree.  
time complexity:  $O(b^{\frac{m}{2}})$



## Solving TSP by local search:



### 1. Nearest Neighbour (NN):



current tour: 1

current cost: 0

current tour: 1, 4

current cost:  $20 + 20 = 40$

to complete the  
tour come back  
to initial state

current tour: 1, 4, 2

current cost:  $20 + 10 + 25 = 55$

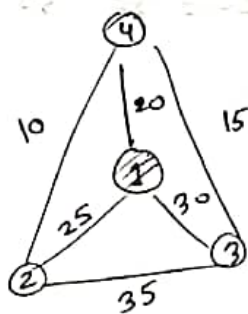
current tour: 1, 4, 2, 3

current cost:  $20 + 10 + 35 + 30 = 95$

- Pros:
1. Simple, Intuitive and relatively efficient.
  2. Empirically OK, esp. on Euclidean TSP

- Cons:
1. Greedy: can easily miss shortcut paths.

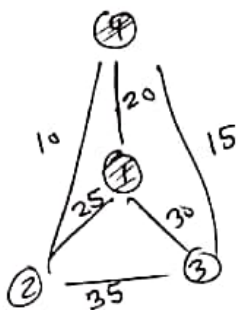
### Nearest Insertion (NI):



cur. tour: 1

cost: 0

↓



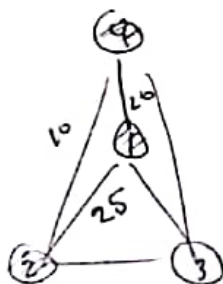
cur. tour: 1, 2, 3

cost:  $20 + 20 = 40$

Next vertex: 2

$$\text{After 1: } w(1,2) + w(2,4) - w(1,4) = 25 + 10 - 20 = 15$$

$$\text{After 4: } w(4,2) + w(2,1) - w(1,4) = 10 + 25 - 20 = 15$$

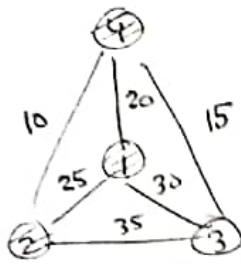


cur. tour: 1, 2, 4

cur. cost:  $25 + 10 + 20 = 55$



Next vertex: 3



current tour: 1, 2, 4  
cost:  $25 + 10 + 20 = 55$

After: 1  $w(1,3) + w(3,2) - w(1,2) = 30 + 35 - 25 = 40$

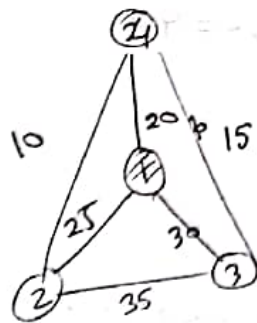
After: 2  $w(2,3) + w(3,4) - w(2,4) = 35 + 15 - 10 = 40$

After: 4  $w(4,3) + w(3,1) - w(4,1) = 15 + 30 - 20 = 25$

$\therefore$  Tour: 1, 2, 4, 3

cost:  $25 + 10 + 15 + 30 = 80$

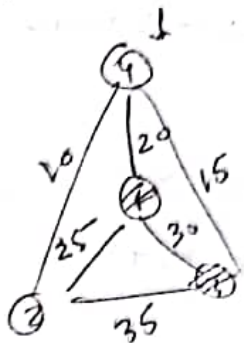
Farthest Insertion (FI) :



current tour: 1

current cost: 10

taking 3



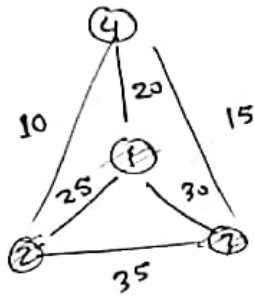
current tour: 1, 3

current cost:  $30 + 30 = 60$

taking 2

After: 1  $w(1,2) + w(2,3) - w(1,3) = 25 + 35 - 30 = 30$

After: 3  $w(3,2) + w(2,1) - w(1,3) = 35 + 25 - 30 = 30$



cut:  $w(1,2,3)$

cut:  $25 + 35 + 30 = 90$

taking 4

After: 1  $w(1,4) + w(4,2) - w(1,2) = 20 + 10 - 25 = 5$

After: 2  $w(2,4) + w(4,3) - w(2,3) = 10 + 15 - 35 = -10$

After: 3  $w(3,4) + w(4,1) - w(3,1) = 15 + 20 - 30 = 5$

cut: 1, 2, 4, 3

cut:  $25 + 10 + 15 + 30 = 80$

## Decision Tree:

Given dataset.

high, low, medium  
↓

| GPA | Studied | Passed |
|-----|---------|--------|
| L   | F       | F      |
| L   | T       | T      |
| M   | F       | F      |
| M   | T       | T      |
| H   | F       | T      |
| H   | T       | T      |

Q. What is the entropy  $H(\text{passed})$ ?

soln:

$$H(\text{passed}) = - \left( \frac{2}{6} \log_2 \frac{2}{6} + \frac{4}{6} \log_2 \frac{4}{6} \right)$$
$$= 0.92$$